Introduction to Artificial Intelligence

Chapter 3 - Solving Problems by Searching

Namyup Kim

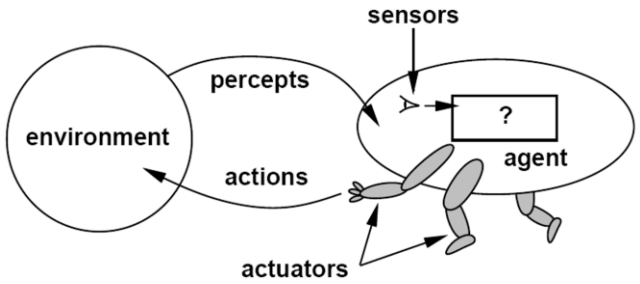
Soongsil University



Adapted from CS188 at UC Berkeley created by Dan Klein and Pieter Abbeel and Intro to AI at Soongsil Univ. created by Young-joon Han

Recap of Chapter 2

- An agent ***perceives*** through sensors and ***acts*** through actuators



  - A rational agent takes the action expected to maximize its performance measure.
- That bundle (PEAS) is the ***task environment.***
- Seven properties classify any environment, and each one makes a demand on the agent.
- Four architectures, each fixing what the previous could not:
  - Reflex → Model-based → **Goal-based (for this chapter)** → Utility-based

Contents

1. What is a search problem?
2. Tree search
3. Search space
4. Uninformed search strategies
5. Informed search strategies

Learning Objectives

- Understand what a search problem is, and how a goal-based agent uses one
- Understand the various measures that describe a search space
- Understand the uninformed and informed search algorithms, and their properties

Introduction to Artificial Intelligence

Chapter 3 - Part I: Search Problem, Tree Search, and Search Space

Namyup Kim

Soongsil University



1. Search Problem

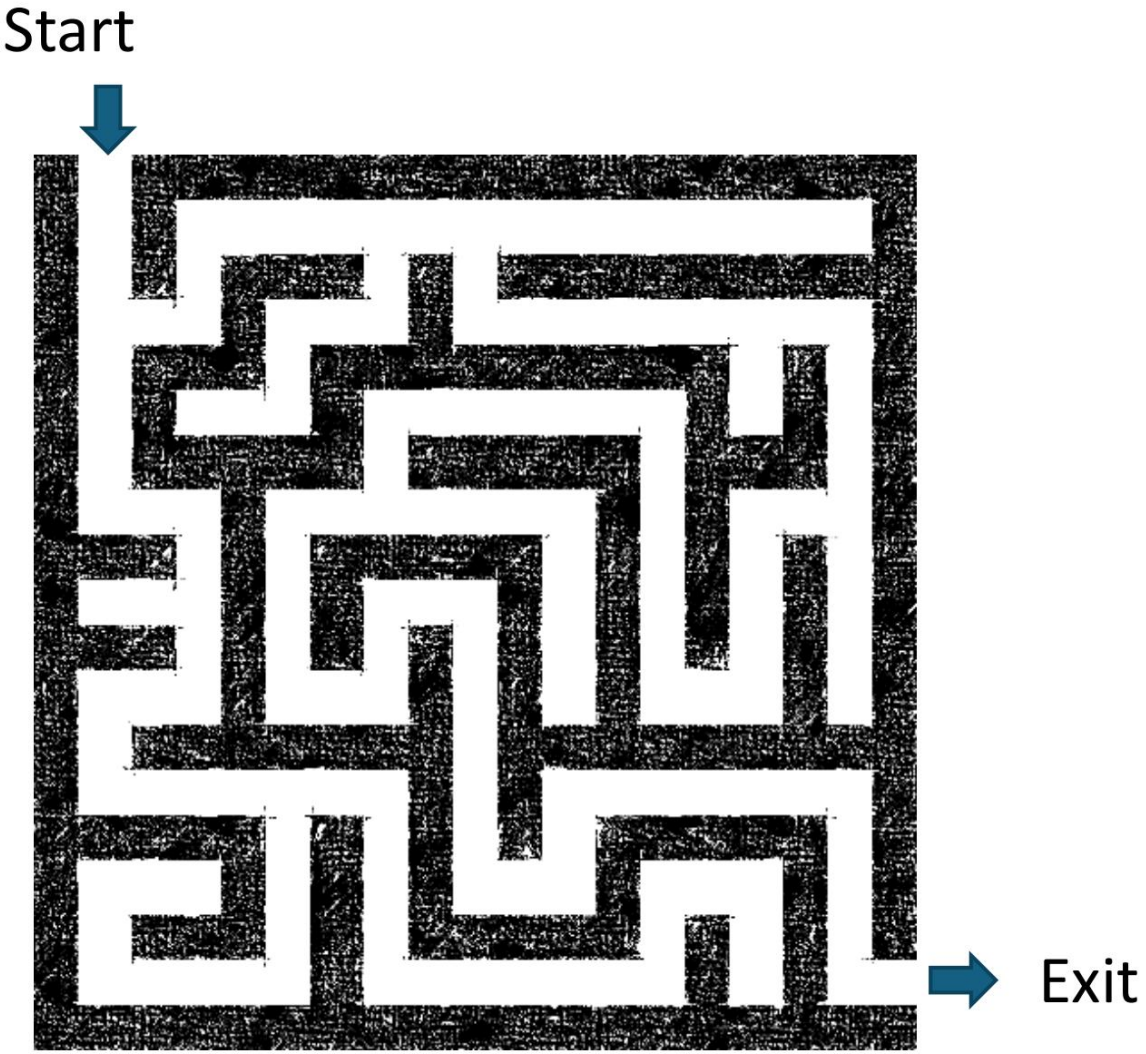
What has to be written down before any search can start?

- A search problem is five things: an initial state, actions, a transition model, goal states, and an action cost function
- Transition function: $ACTIONS(s)$ and $RESULT(s, a)$ are functions, not stored tables - states are generated on demand.
- Three worked examples: vacation in Romania, the vacuum-cleaner world, the sliding-tile puzzle
- Choosing the state representation is a design decision, and it is the hard part

Search Problem: What Is It?

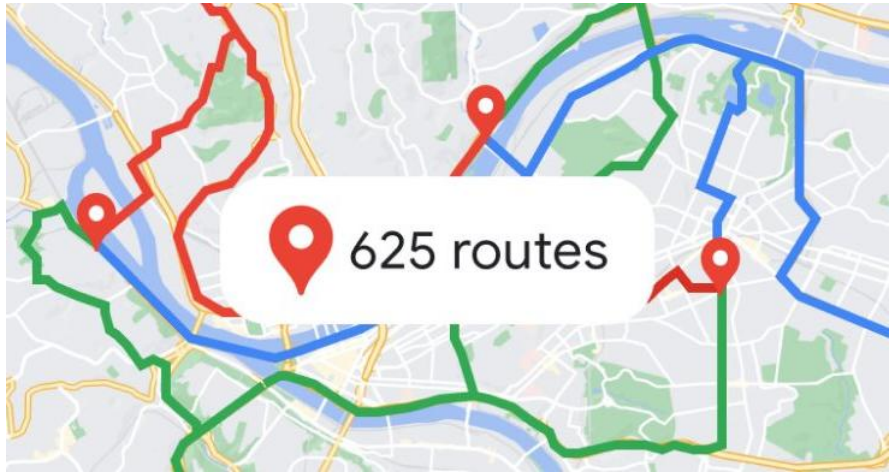
- We design ***goal-based agents*** for environments that are.
  - **Known:** the agent knows what its actions do
  - **Fully observable:** it can see the whole state
  - **Deterministic:** the same action always gives the same result

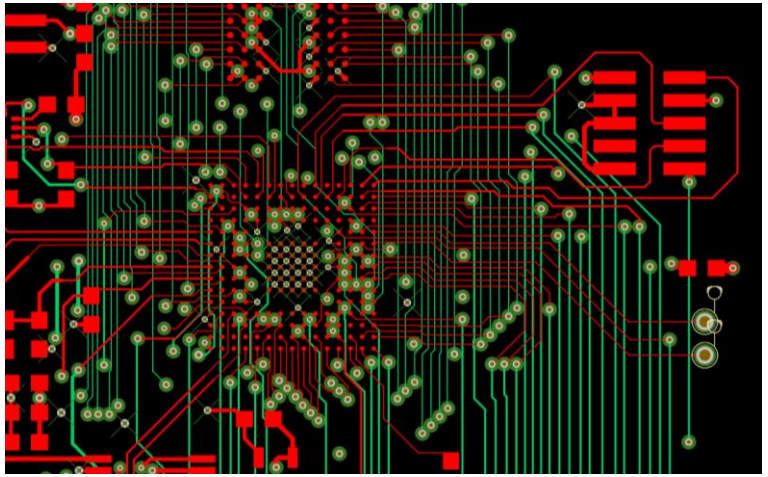
- *Example*: a robot in a maze — the maze is the environment

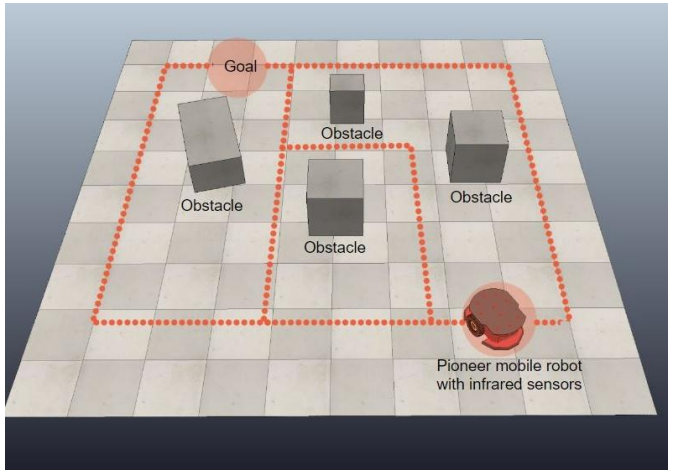


Why Search Matters

- The algorithms we'll discuss are incredibly widely applied.
- Pathing and routing problems, both for people and for robots
- Chip design (routing wires) and resource planning problems
- Robot motion planning and protein design
- Language analysis, machine translation, and speech recognition
- Video games

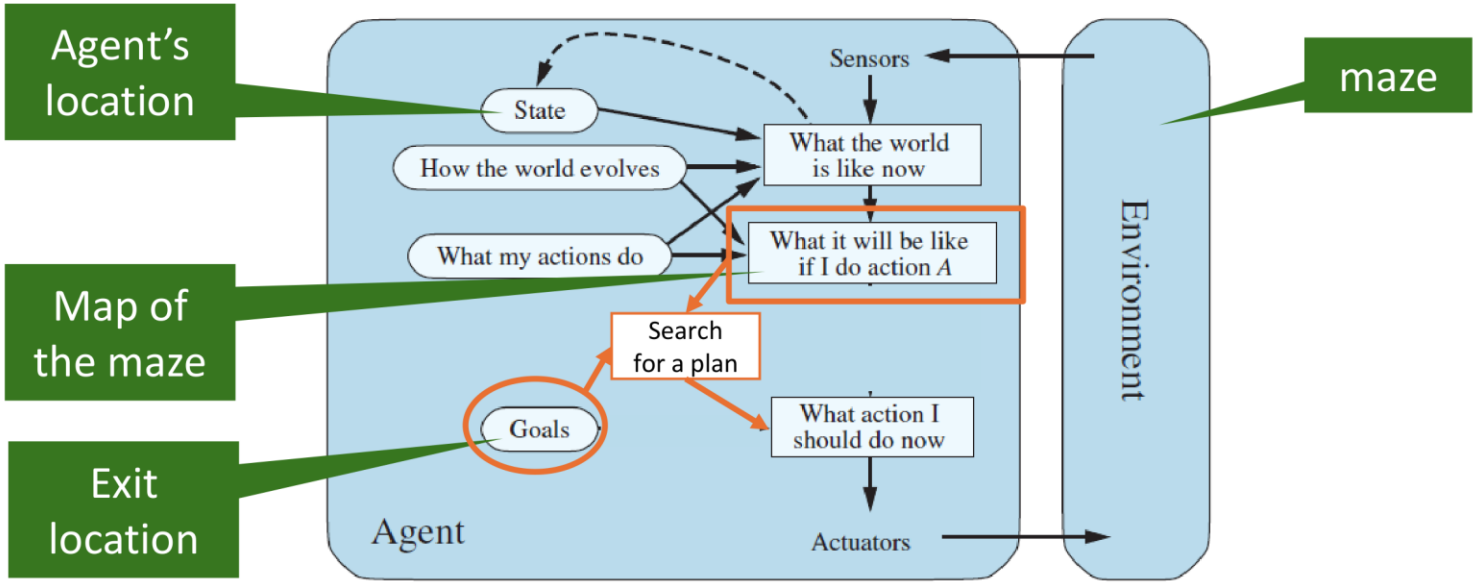






Search Problem: Recall the Goal-Based Agent

- The agent's task is to reach a defined goal state
- Its performance measure is normally the cost of getting there
- **The problem-solving agent** : a special kind of goal-based agent which produces a sequence of actions



$$a_i = \text{argmin}_{a_i \in A}\left[\sum_{t=i}^{T} c_t \;\middle|\; s_T \in S^{goal}\right]$$

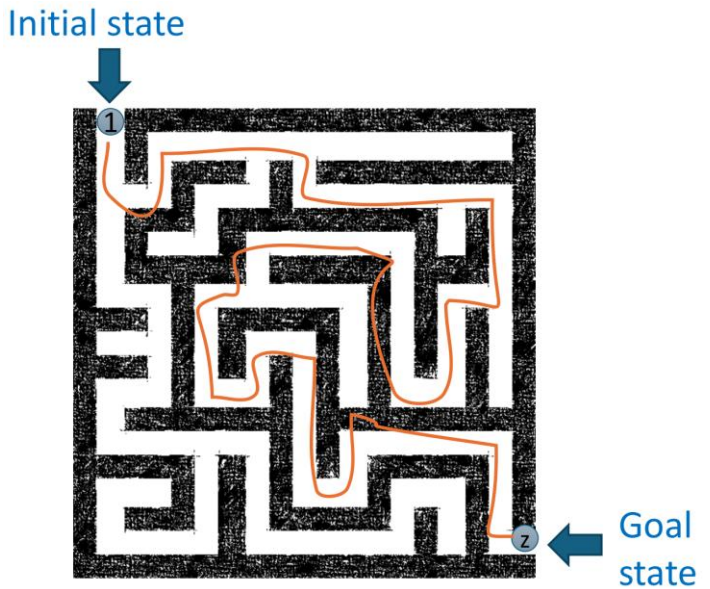
Search Problem: Problem Formulation

- We consider only discrete environments with an atomic state representation (1, 2, 3, …)
- The **state space** is **the set of all states** the environment can be in, including the initial state and the goal states.
- An **optimal solution** is the sequence of actions with the lowest path cost that reaches a goal state.

s	a	s'
1	S	2
2	N	1
2	S	3
…	…	…
4	E	a
4	S	5
4	N	3
…	…	…

state-space graph

1) Search, or planning: the process of finding that sequence
2) In a **known and deterministic environment,** the agent can ignore its **percepts** while executing the plan (an open-loop system)



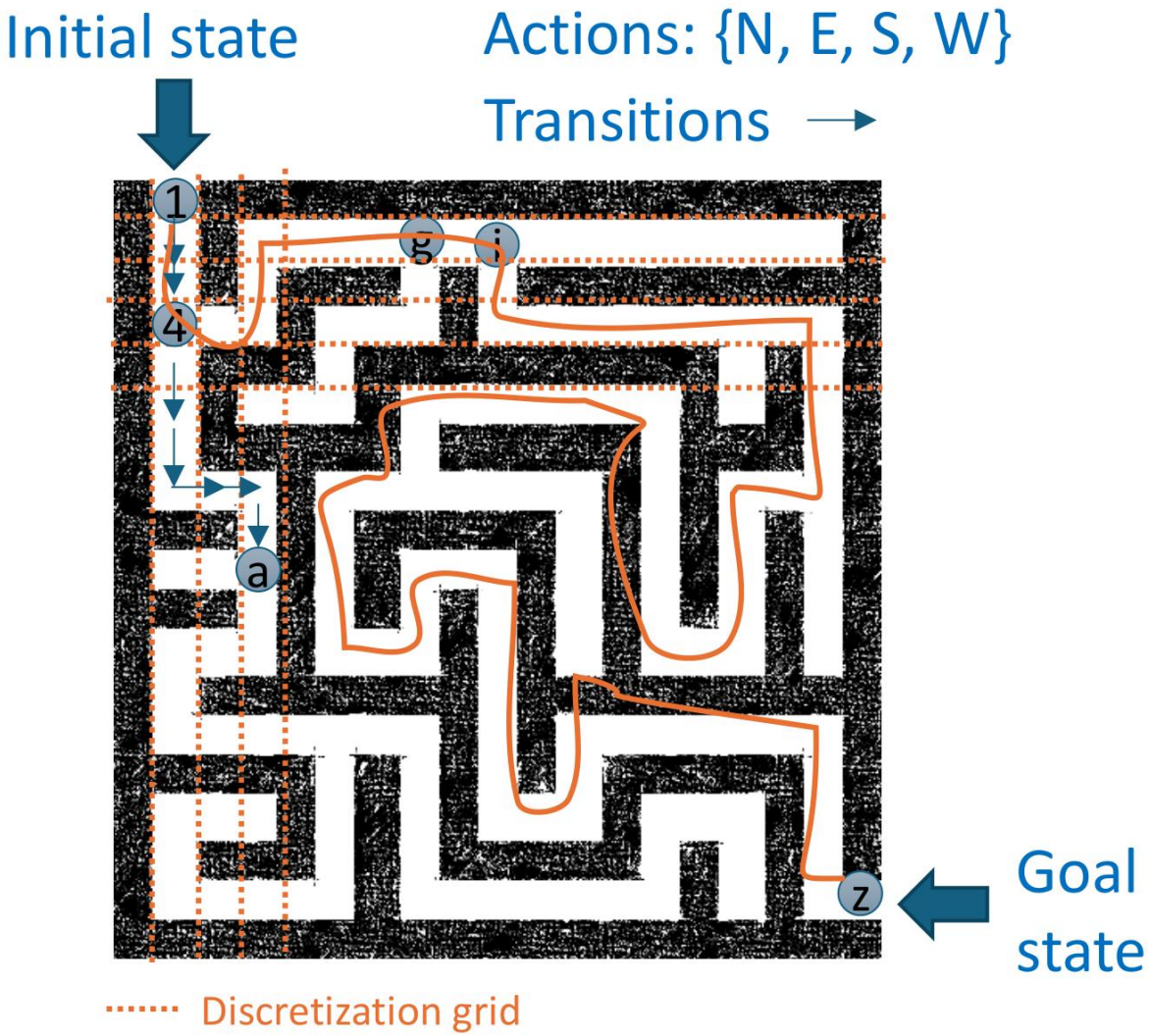
Actions: {N, E, S, W}

Search Problem: Problem Formulation (2)

- **Initial state**: where the agent starts
- **Actions**: the set A of possible actions
- **Transition model**: the function giving the next state when one action is taken in the current state
- **Goal state**: a set of one or more goal states
- **Action cost function**: what each action costs

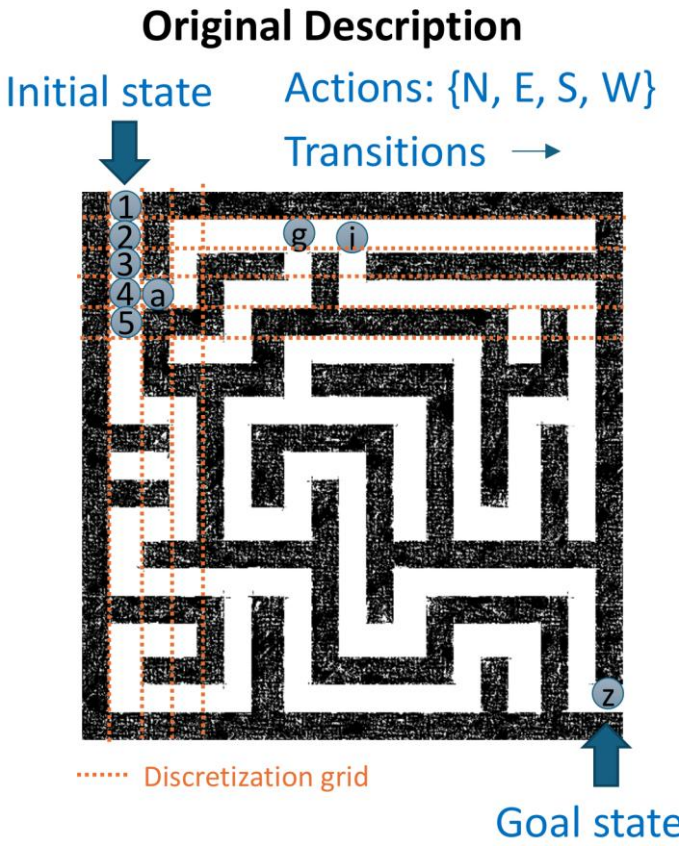
1) We define the initial state, the actions, the transition model and the goal states. And **NOT the state space** (**table**) itself, which is usually too large to enumerate, or continuous

2) **Path cost**: the sum of the action costs along the sequence from initial state to goal



Search Problem: Transition Function and Actions

- Transition function: $f : S \times A \rightarrow S$, or $s' = RESULT(s, a)$
- **A lookup table** of the transition model is a **state-space graph**: Each row is one edge.
- $ACTIONS(s)$ is the set of actions legal in state s. $ACTIONS(4) = \{E, S, N\}$

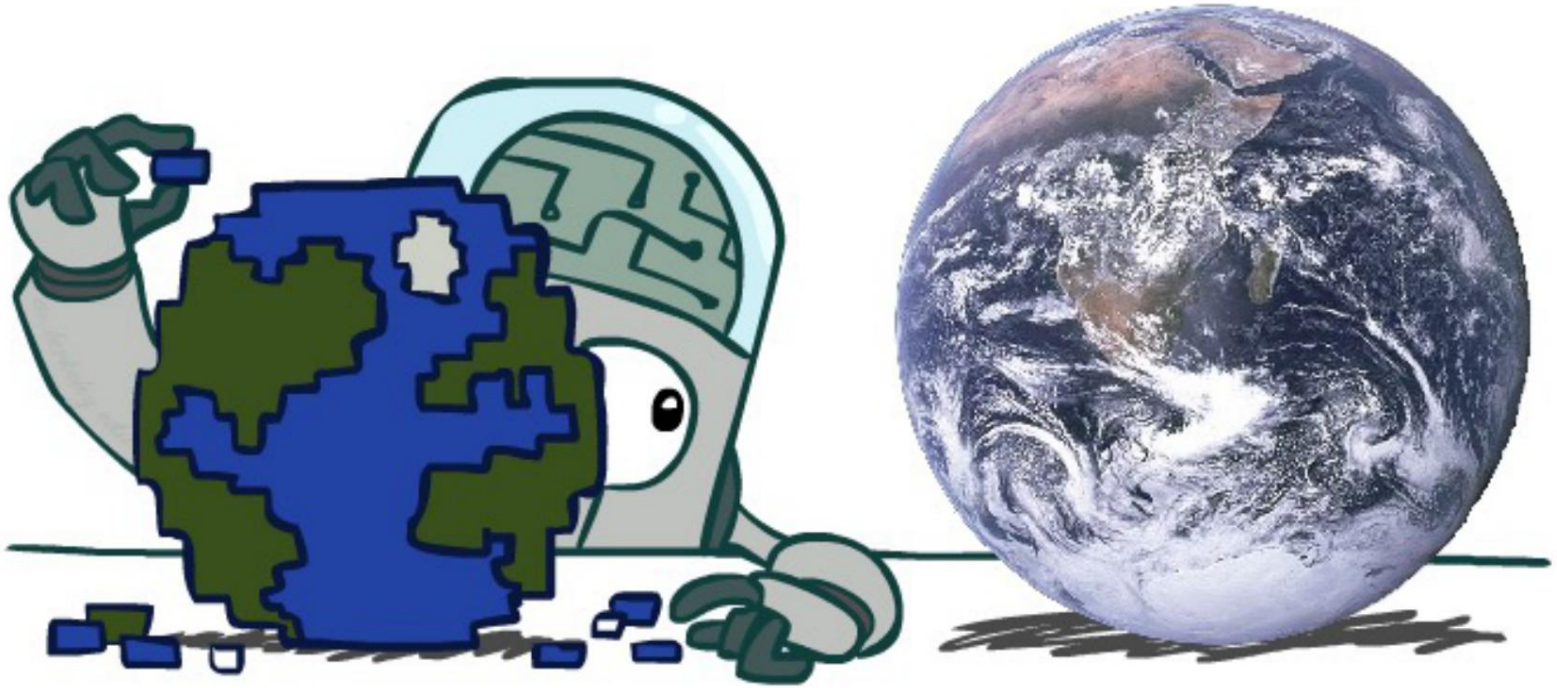


s	a	s'
1	S	2
2	N	1
2	S	3
...	...	...
4	E	a
4	S	5
4	N	3
...	...	...

$$Actions(4) = \{E, S, N\}$$

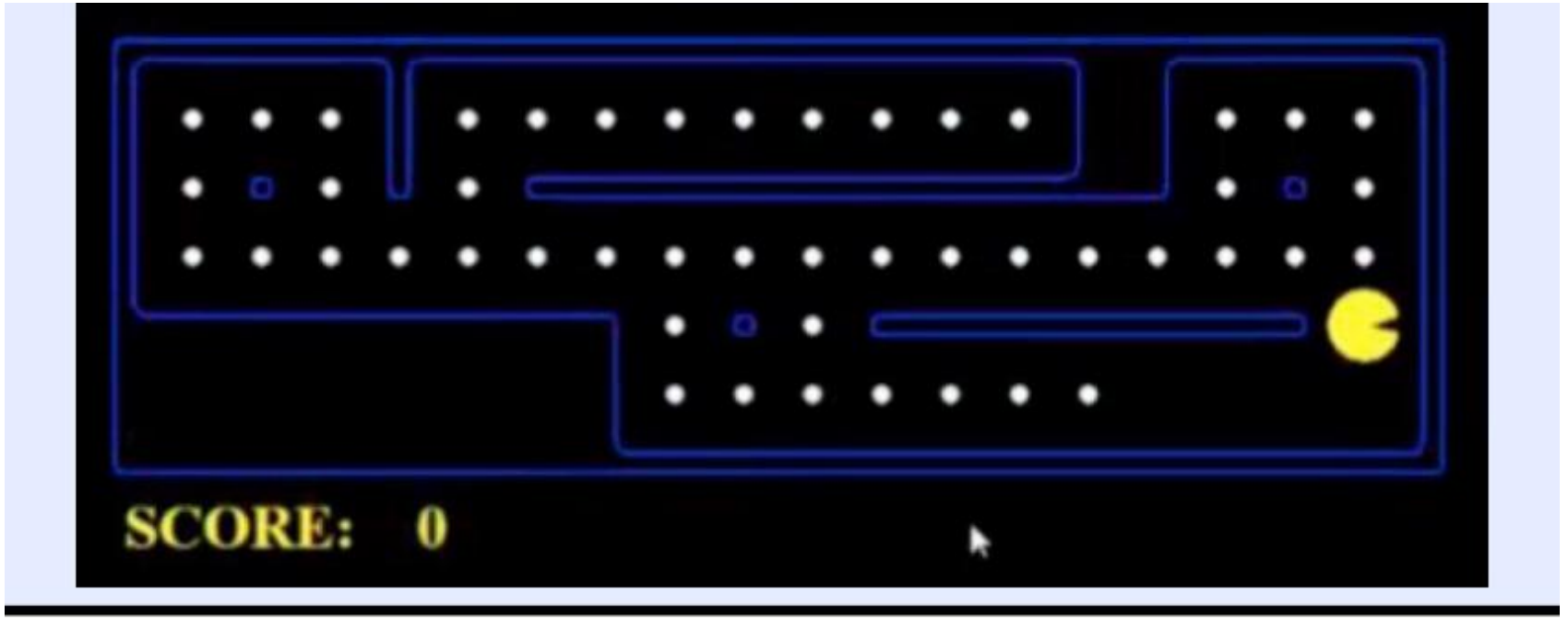
Search Problems Are Models

- A robot has built a blocky, pixelated model of the Earth and set it next to the real planet.
- Search never runs on the world itself - it runs on a model of the world.
- The model is a simplification on purpose, so a plan is only as good as the model it was found in.



What's in a State Space?

- **World state**: Every detail of the environment.
- **Search state**: only the details needed for planning - this is abstraction.
  - Problem 1: Pathing - states are (x,y) locations; actions are $\{N,S,E,W\}$; the transition model updates location only; every action costs 1; the goal states are those where (x,y) == END.
  - Problem 2: Eat-All-Dots - states are $\{(x,y), dot\ booleans\}$; actions are $\{N,S,E,W\}$; the transition model updates location and possibly a dot boolean; every action costs 1; the goal states are those where all dots are false.



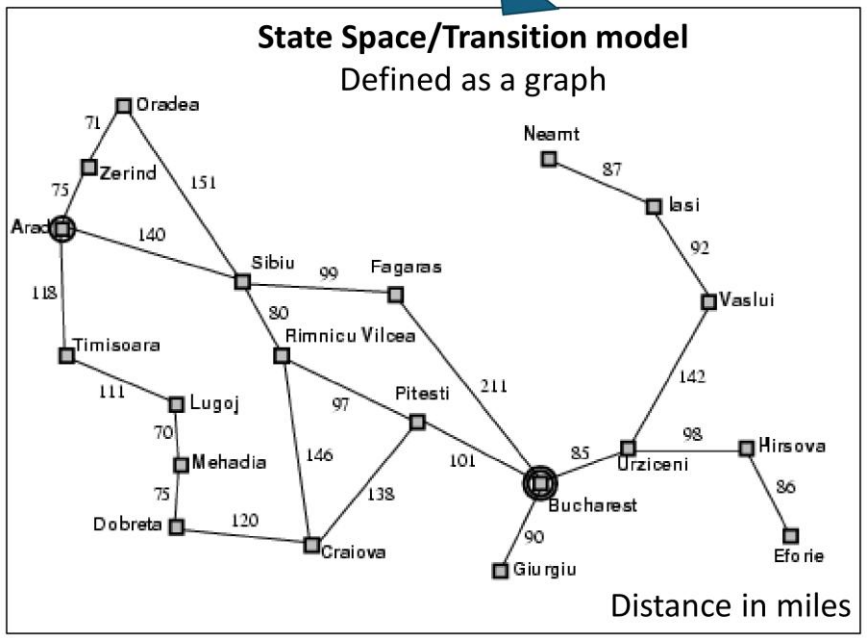
Example: Vacation in Romania

- *You are on holiday in Arad and fly out of Bucharest tomorrow.*

- **Initial state**: Arad
- **Actions**: drive from one city to another
- **Transition model**: leaving city A for city B lands you in B
- **Goal** state: Bucharest
- **Action cost**: distance travelled
- Set of actions (**Path) cost**: total distance to the goal

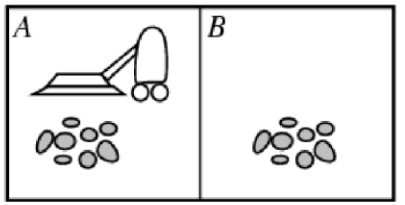
Original Description

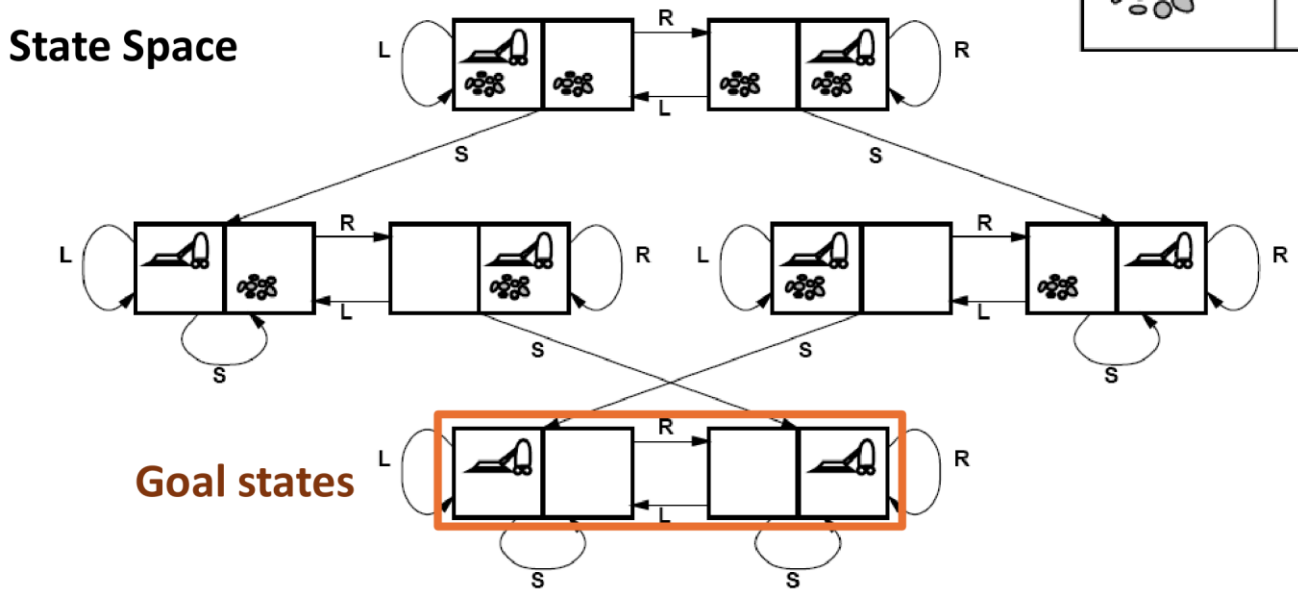




Example 2: The Vacuum-Cleaner World

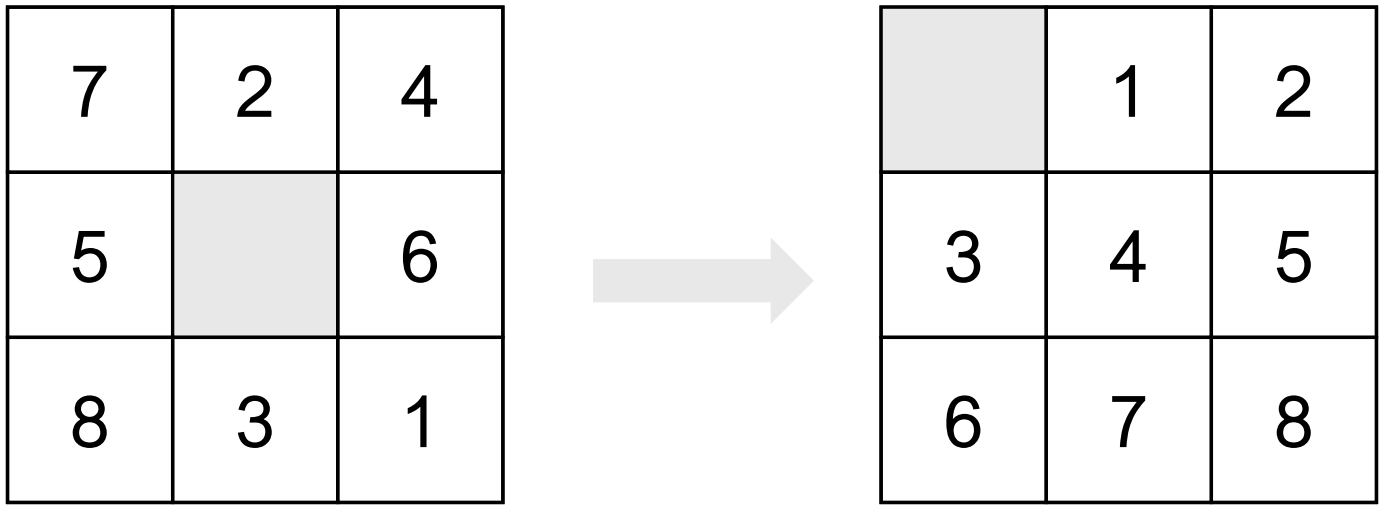
- States — which of the two rooms holds the agent, and where the dirt is
- Initial state — any of them
- Actions — Left, Right, Suck
- Transition model — move, or remove the dirt
- Goal state — both rooms clean
- Action cost — 1 per action, so path cost counts actions
- 8 atomic states can be represented





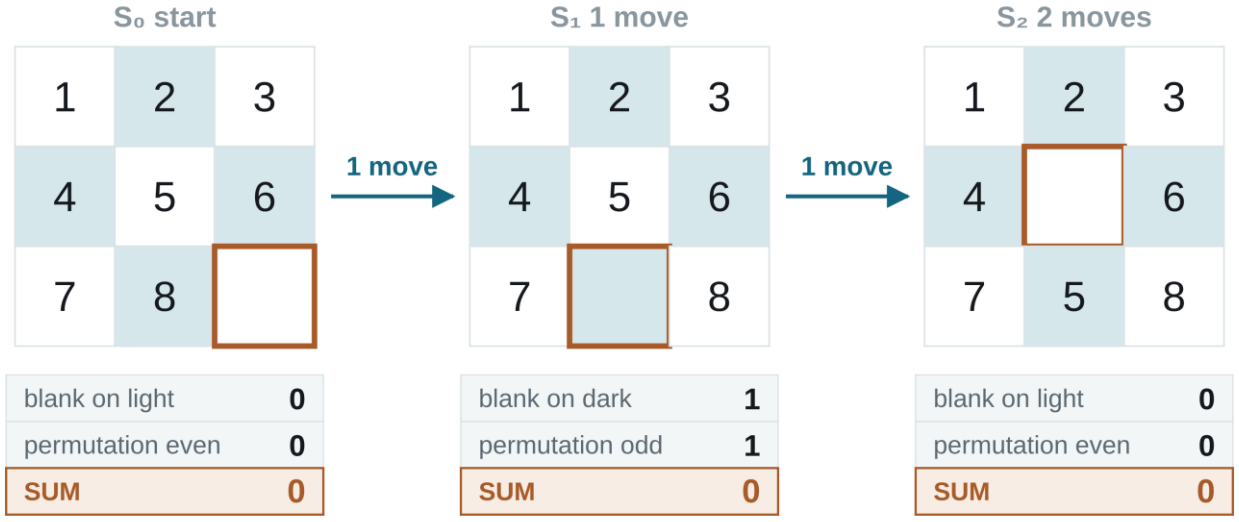
Example 3: The Sliding-Tile Puzzle

- States: tile positions
- Initial state: the left-bottom figure
- Actions: move the blank tile left, right, up or down.
- Transition model: : swap the blank with the position of the tile the moved into it.
- Goal state: the tiles in order as the right-bottom figure.
- Action cost: the tiles in order as the right-bottom figure.

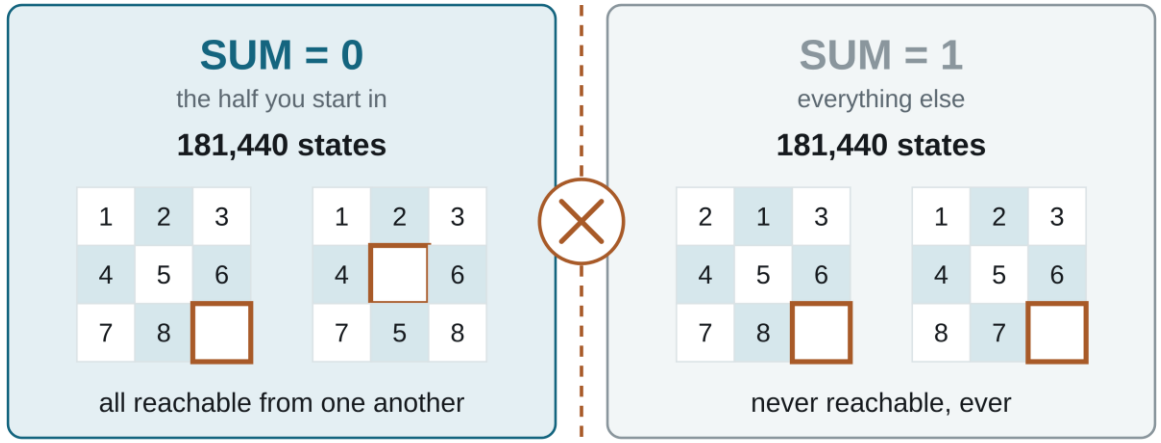


Example: How Big Is the Puzzle?

- Each state is a placement of the tiles, blank included
- Half of the permutations are unreachable from any given start
  - 8-puzzle — 9! / 2 = 181,440 states
  - 15-puzzle — 16! / 2 ≈ 10^{13} states
  - 24-puzzle — 25! / 2 ≈ 10^{25} states
  - Going from the 15-puzzle to the 24-puzzle (one more row and one more column) multiplies the state space by about 10^{12}



Both flip together, every single move — so the sum never changes.



9! = 362,880 splits in two, and no legal move crosses.

Introduction to Artificial Intelligence

Chapter 3 - Part I: Search Problem, Tree Search, and Search Space

Namyup Kim

Soongsil University



2. Tree Search

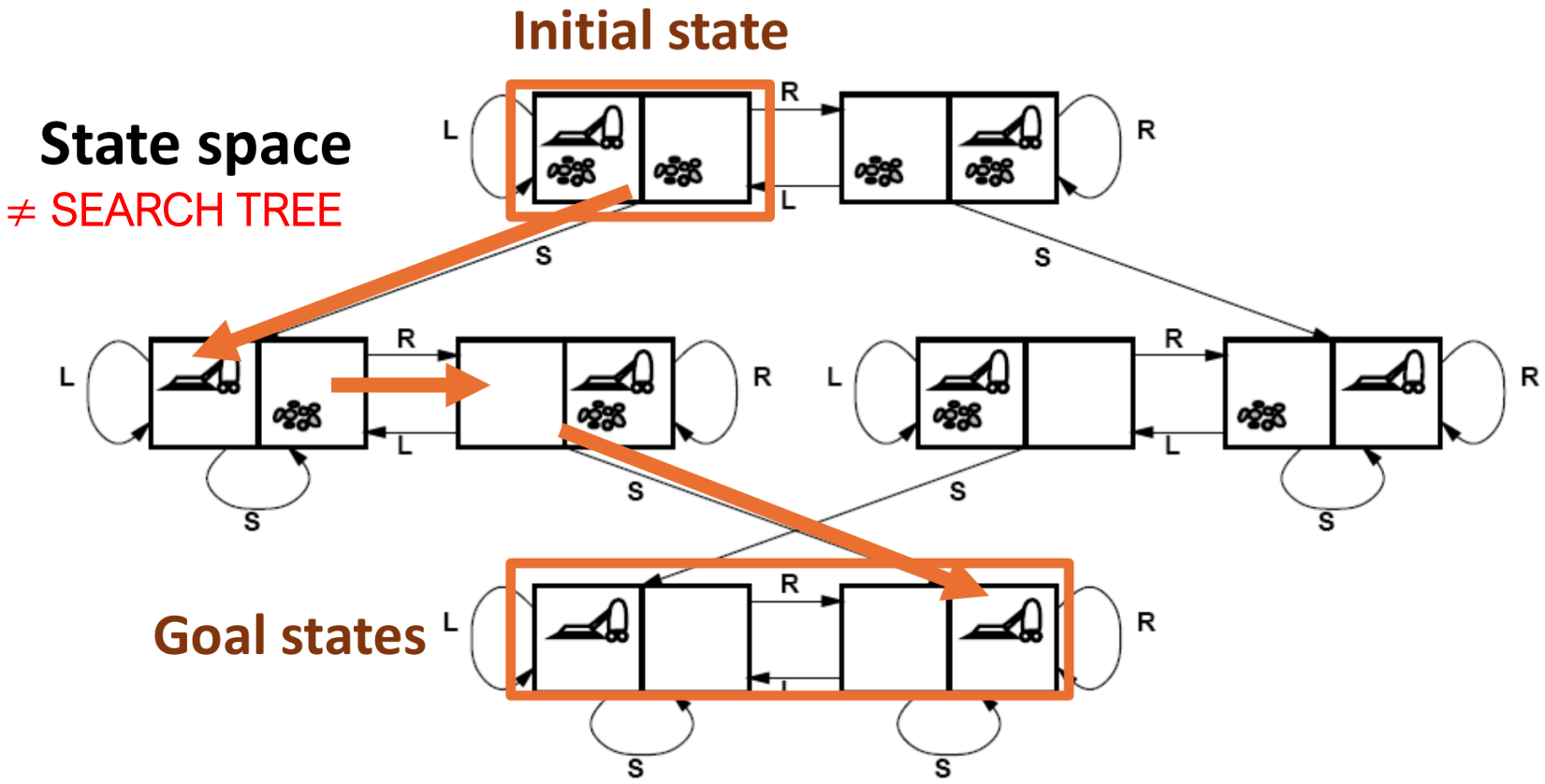
The state space is a graph. A search tree is not. What happens in between?

- Cycles and redundant paths, and why every algorithm today has to deal with them
  - In tree, Seoul ≠ Seoul → Busan → Seoul ≠ S → B → S → B → S
- The frontier known but not yet expanded, also called the open list
- In tree, nodes are paths, not states: eight states, an infinite tree
- One loop that every strategy in this chapter is a special case of
- The four questions asked of any search strategy: complete, optimal, time, space

Tree Search: Solving a Search Problem

- We have the problem formulation:
  - initial state, actions, transition model, goal states, action cost function
- So how do we actually find the optimal solution?

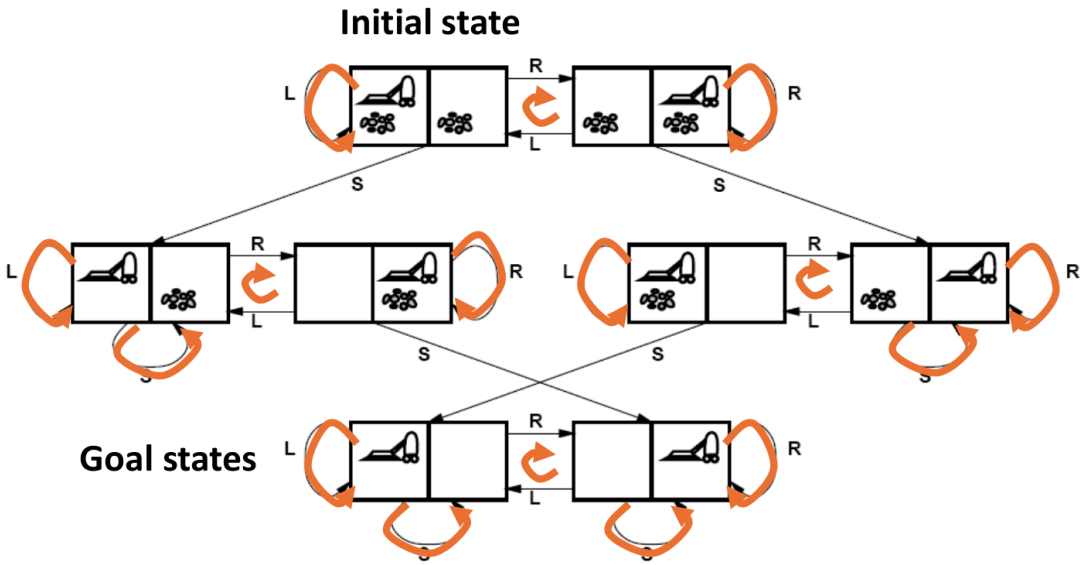
  → *Build **a search tree** over the state-space graph*



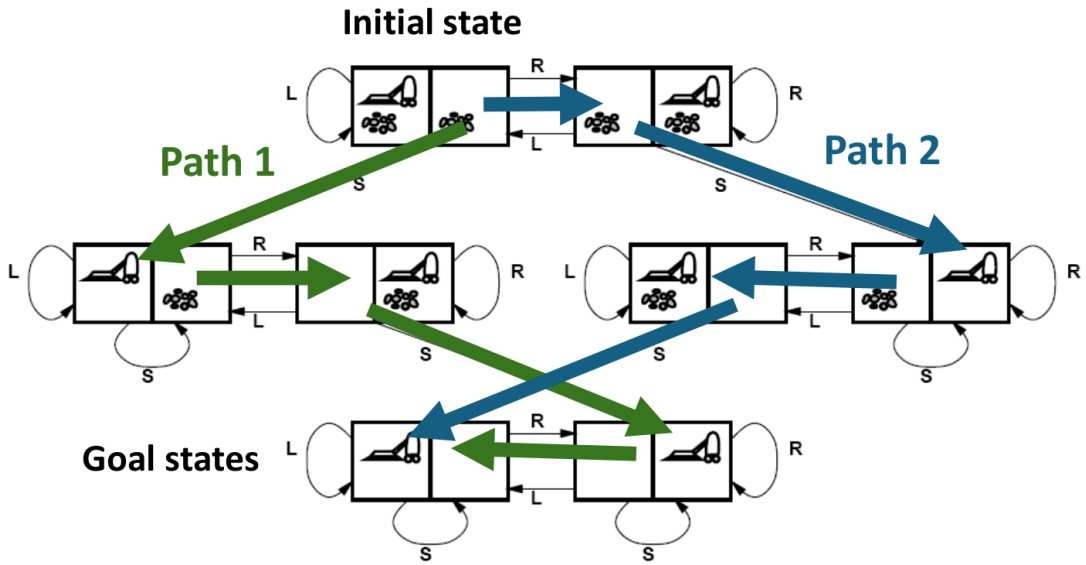
Tree Search: The Transition Model Is a Graph, Not a Tree

- **Cycles**: paths that come back to the same state
- Non-cycle **redundant paths**: several different paths that reach the same state
- Both have to be dealt with before a graph can be searched as a tree

Cycles - the orange loops return to the same state

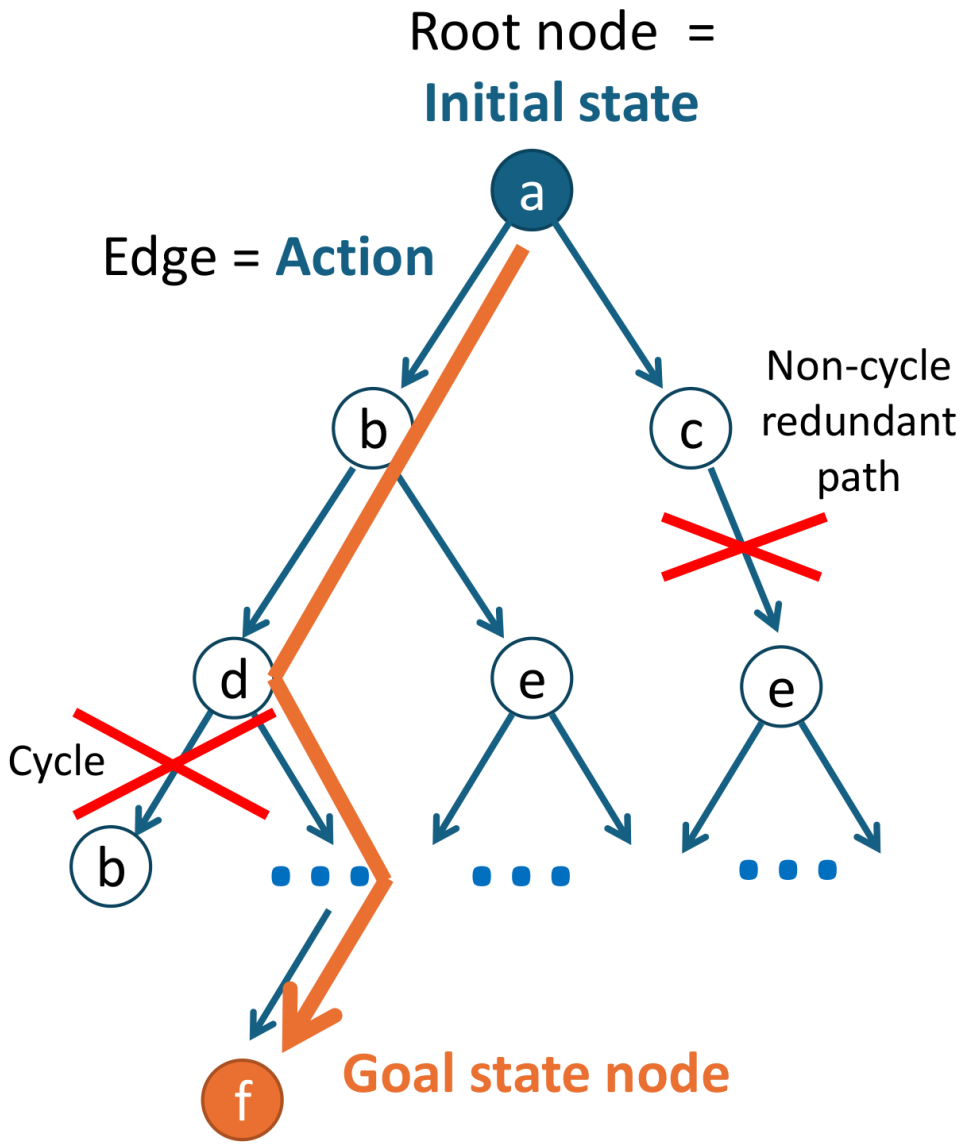


Redundant paths - Path 1 and Path 2 reach the same goal



Tree Search: Generating the Search Tree

- Overlay a "***what-if tree***" of possible actions on the state-space graph.
- The root node is the initial state.
- A child is reached along an **edge** representing an **action**, and the **transition model** says which state that child is.
- A tree cannot contain a **cycle**
  - A cycle in the search space becomes an infinite loop, so it must be cut.
- A tree cannot contain **two paths to the same state**
  - That is a redundant path; removing it improves efficiency
- A path through the tree is a sequence of actions; a solution is a path ending at a goal node

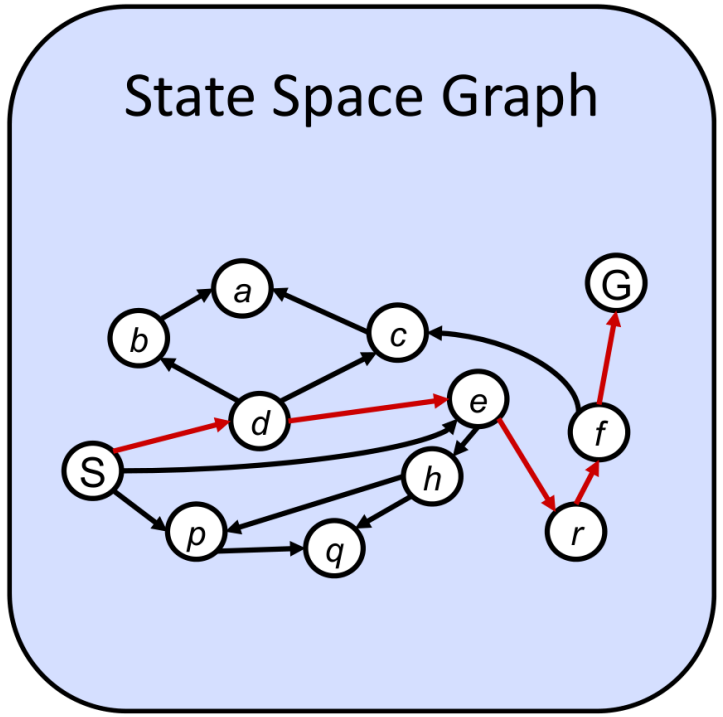


Tree Search: Nodes Are Not States

- This is the distinction students lose most often
- Each node of the tree stands for one state of the system. But it is not the same thing as that state
- But when redundant paths cannot be fully prevented, the same state appears as several different nodes.
- State space: the world. One node per state. Fixed. A graph
- Search tree: the record of what we tried. One node per PATH.
  Built as we go. Always a tree. Can be infinite
- The vacuum world has 8 states and an infinite search tree

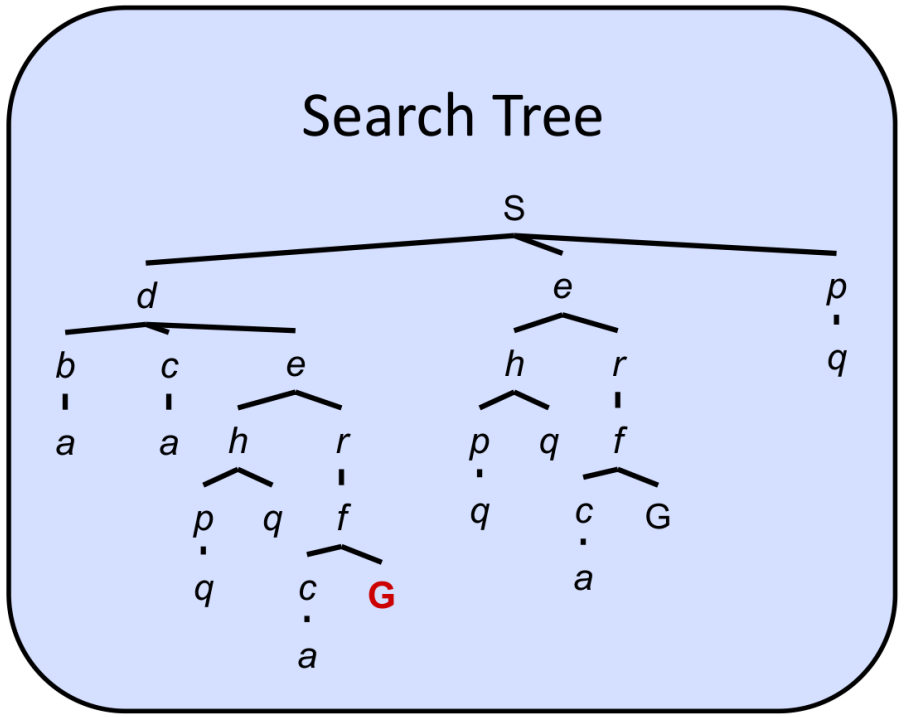
State Space Graphs vs. Search Trees

- The same problem drawn both ways:
  - the state space graph on one side, the search tree unrolled from S on the other
- State e appears once in the graph but many times in the tree once for every path that reaches it. Count them



Each NODE in in the search tree is an entire PATH in the state space graph.

We construct both on demand – and we construct as little as possible.



Tree Search: Classical Tree Search vs AI Search

Classical tree search	AI tree / graph search
The tree is given, and held in memory	The search tree is far too large to hold in memory • Part of the tree is built from the initial state using the transition function • Memory management is critical
	The search space is usually a very large, complex graph
The tree has no redundant or cyclic paths	Memory-efficient cycle checking matters, to avoid infinite loops and limit re-searching the same region
	Checking for redundant paths often costs too much memory, so we accept searching the same part more than once

The difference is whether the tree already exists.

Tree Search: Summary of the Algorithm

1. Initialise the FRONTIER with the start state (root node)
   (the frontier is the set of known but not-yet-expanded nodes)

2. While the frontier is not empty:

   a) Choose the next node to expand from the frontier,
      according to the SEARCH STRATEGY

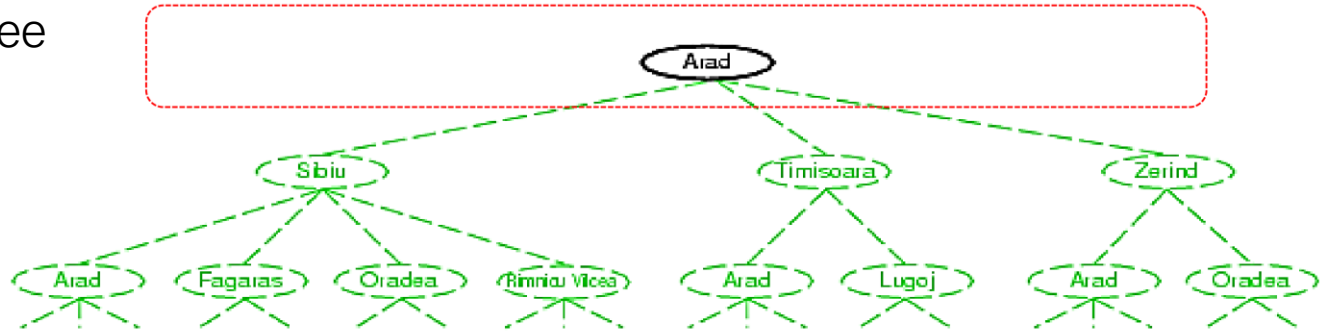
   b) If that node represents a goal state,
      return it as the solution

   c) Otherwise EXPAND the node – apply every possible action
      using the transition model – and add the child nodes
      for the newly reached states to the frontier

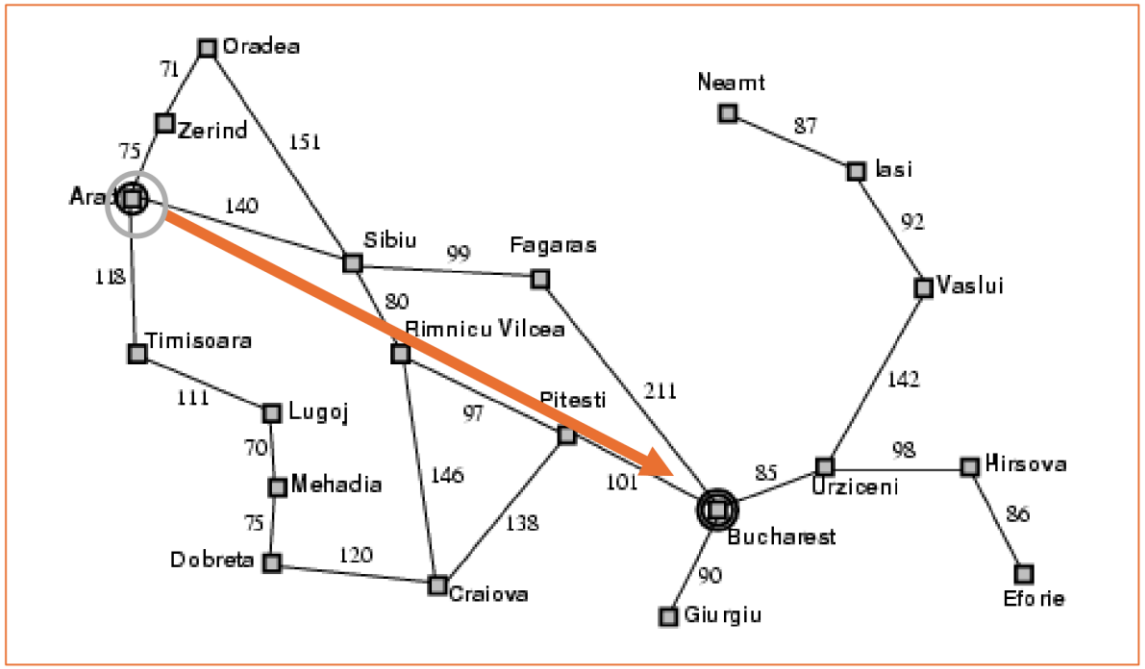
Example: Travelling in Romania (1)

- The frontier holds Arad alone
- Nothing has been expanded yet
- The dashed nodes below do not exist now. They are only drawn to show where children will go.

Tree

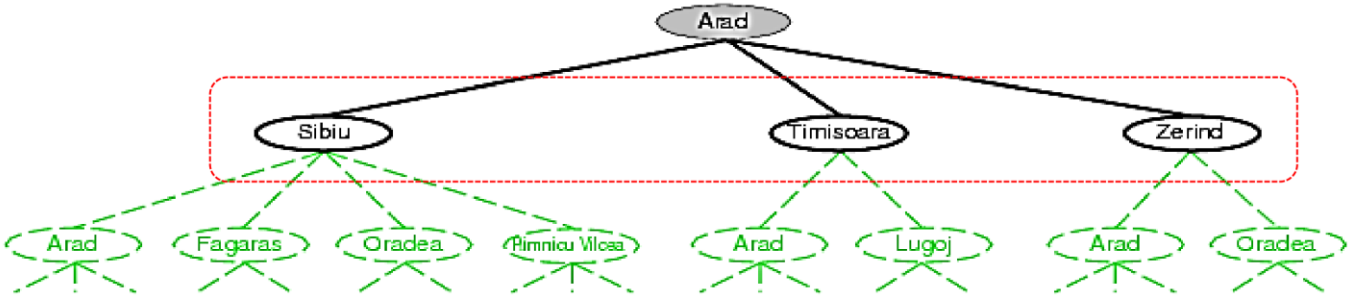


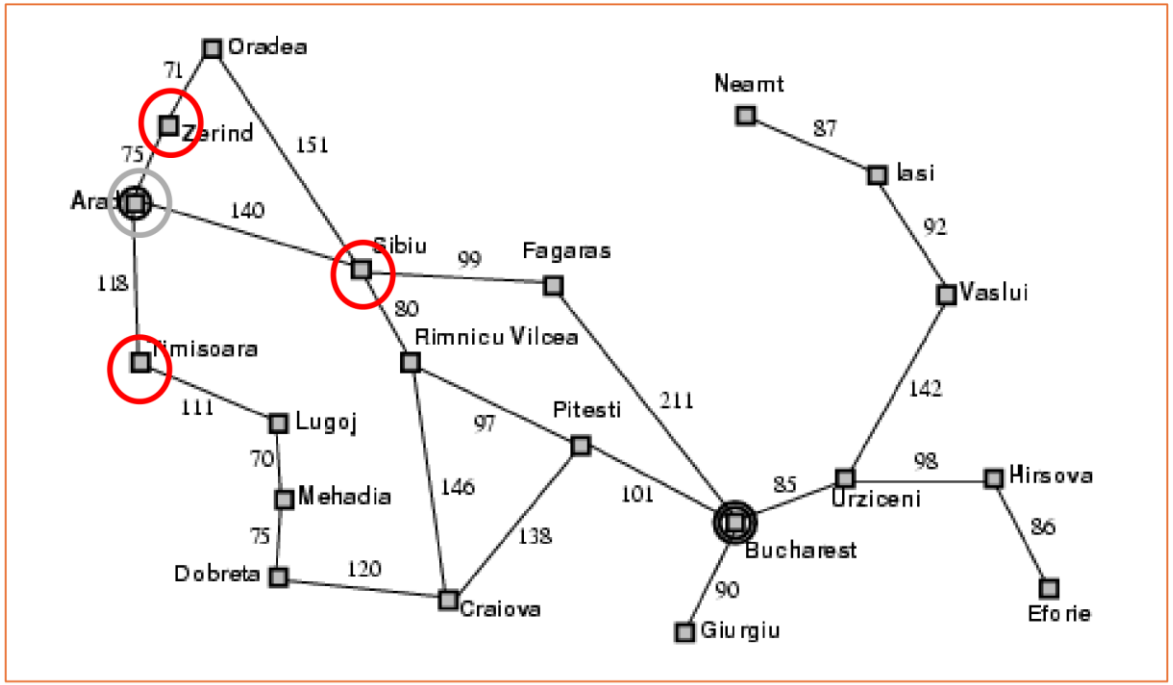
Graph



Example: Travelling in Romania (2)

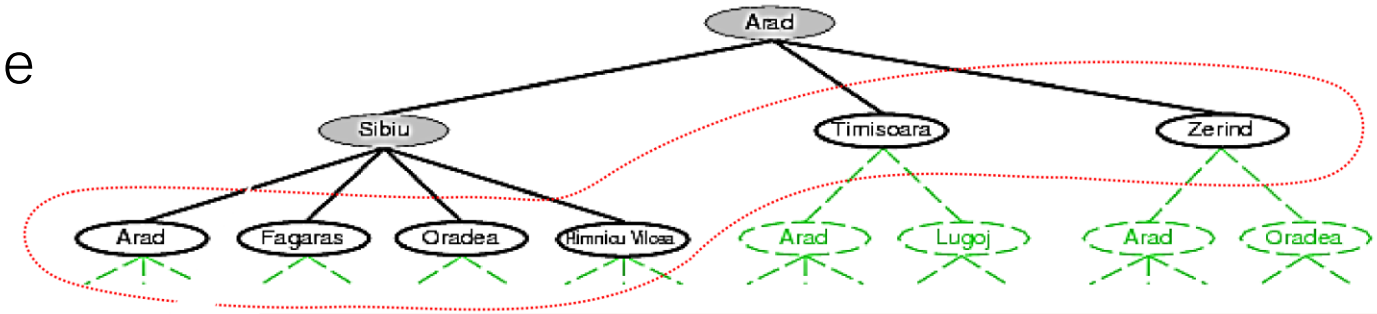
- Expand Arad
- Its three children (Sibiu, Timisoara, Zerind) are generated and join the frontier
- Arad leaves the frontier

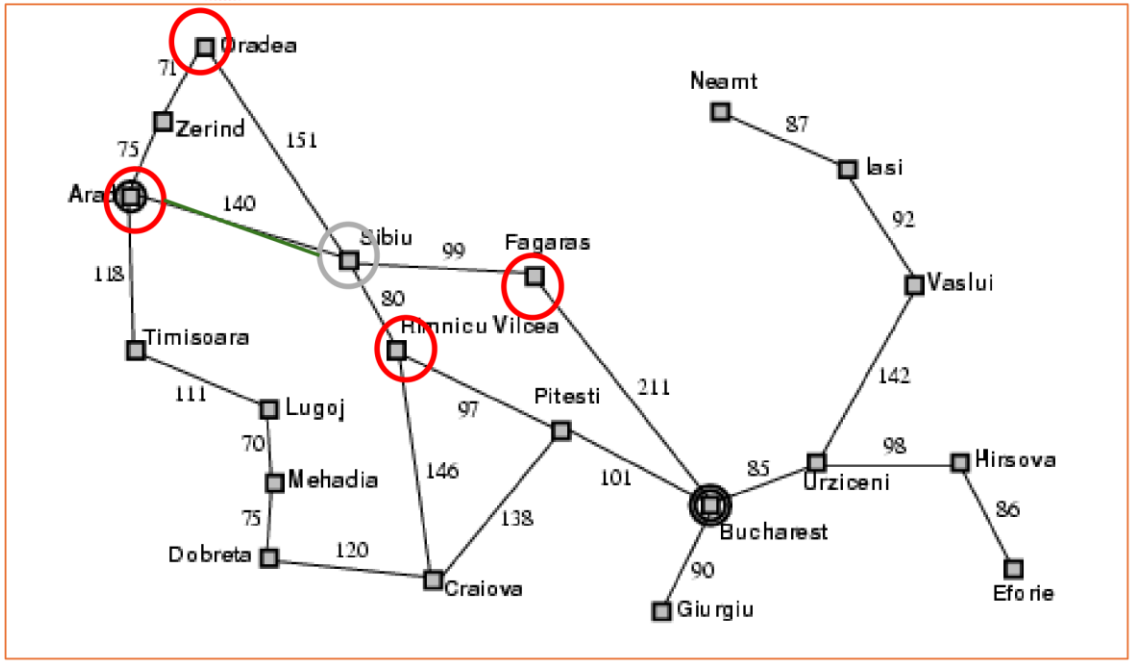




Example: Travelling in Romania (3)

- Expand Sibiu
- Arad has reappeared one level down (the same state) reached by a different path. That is a cycle.
- We could just as well have expanded Timisoara or Zerind instead





Search Strategies: Properties

- A **search strategy** decides the order in which nodes are expanded
- Strategies are evaluated along four dimensions:
  - **Completeness** - does it always find a solution when one exists?
  - **Cost optimality** - does it always find the lowest-cost solution?
  - **Time complexity** - how long does it take?
  - **Space complexity** - how much memory does it need?
- We will discuss several strategies and compare them on these four

3. Search Space

How large is the thing we are searching, and how would we know?

- b, m and d — branching factor, maximum depth, depth of the optimal solution
- Counting the state space by multiplying out its independent parts
- Counting the search tree instead, which is the number that governs cost
- The same four problems measured twice: maze, 8-queens, 8-puzzle, tic-tac-toe
- Memory runs out well before time does

Search Space: Time and Space Complexity

- Both depend on the number n of tree nodes searched before the goal is found.

$$O(n)$$

  - Estimate the size of the reachable state space. The larger it is, the harder the search.
  - Or estimate the number of searched tree nodes directly
- Estimating complexity tells us:
  - How hard is this problem?
  - Which algorithm will fit in memory?
  - Can we find a solution fast enough?
  - Can we find the optimal solution, or must we use a heuristic?

Search Space: Big-O Notation

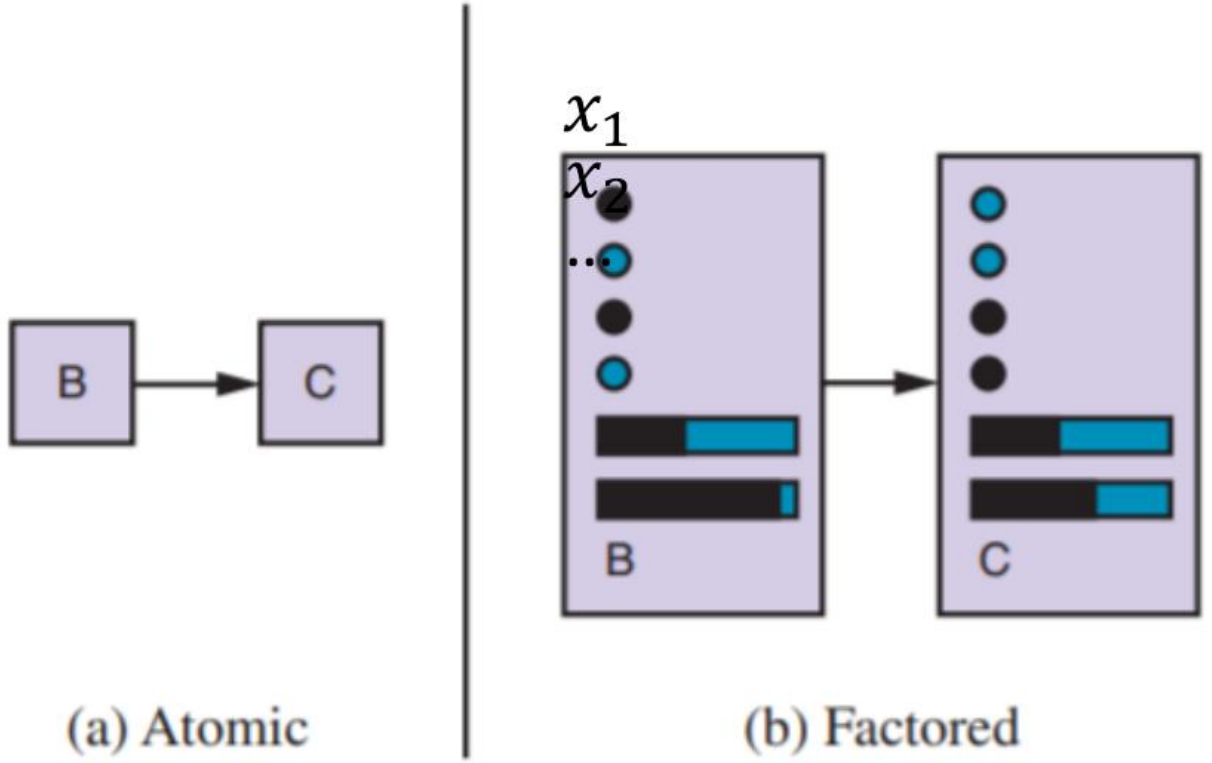
- As the input grows, the highest-order term dominates and the rest stop mattering

- Example: $T(n) = n^2 + n + 1$
  - n = 1: T = 1 + 1 + 1 = 3 (n^2 is 33.3%)
  - n = 10: T = 100 + 10 + 1 = 111 (90%)
  - n = 100: T = 10,000 + 100 + 1 = 10,101 (99%)
  - n = 1,000: T = 1,000,000 + 1,000 + 1 = 1,001,001 (99.9%)

$$\Rightarrow O(T(n)) = O(n^2 + n + 1) = O(n^2)$$

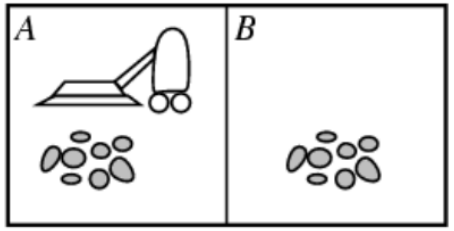
Search Space: Estimating the State-Space Size

- **The state space** is the set of different states the agent and its environment can be in
- **Reachable states** are defined by the initial state and the transition model: only those matter for search
- Even when the algorithm treats states as atomic, we can use a factored representation internally to estimate the size.
- With l fluents (variables) in the factored representation:
  - $n = |X_1| \times |X_2| \times \cdots \times |X_l|$
  - where $|\cdot|$ is the number of values that variable can take



Example: State-Space Size?

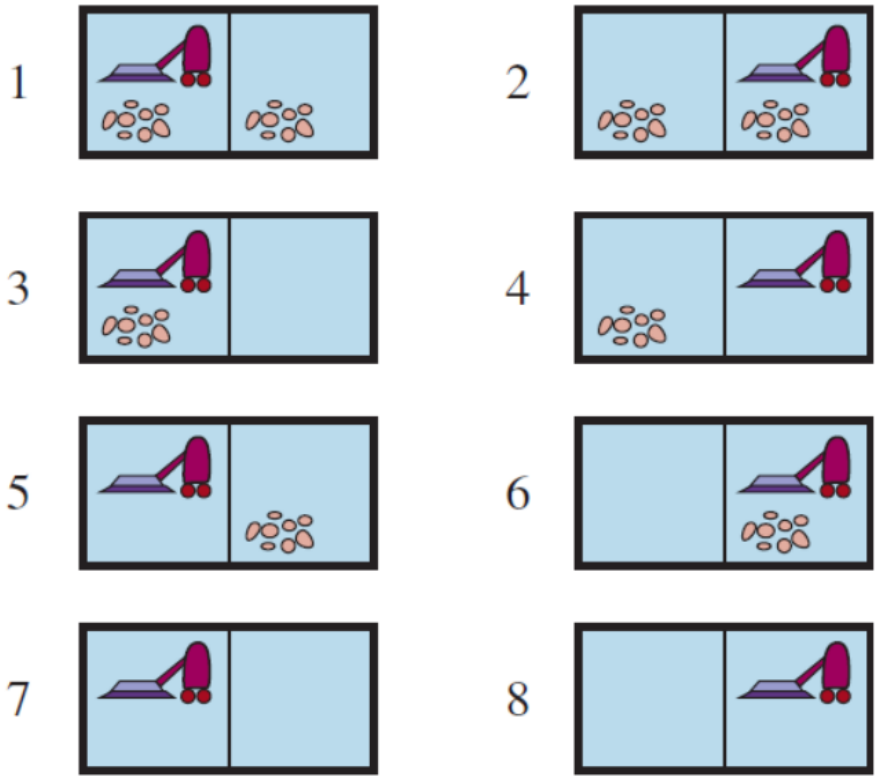
1. Dirt
   - A and B are different rooms; order does not matter
   - Dirt can be in either room, clean or dirty: 2 choices each



$r^n = 2^2$ where r is the room conditions and n is the number of rooms

2. Robot location
   - The robot is in one of the 2 rooms $\rightarrow$ 2

3. Total: $\mathrm{N}_{state} = 2 \times 2^2 = 2^3 = 8$



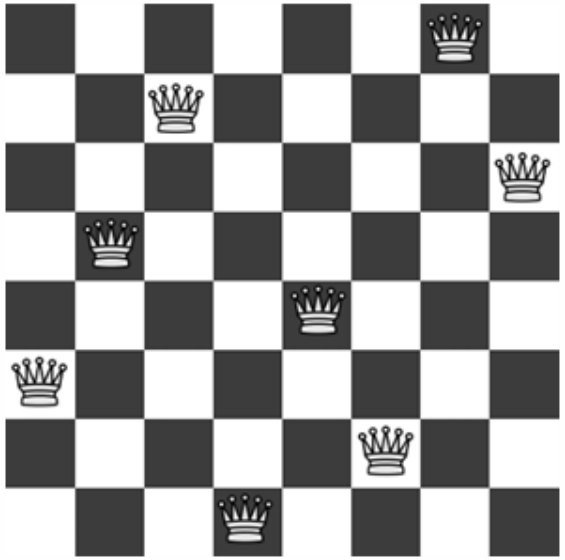
Example: State-Space Size? (2)

The complexity of a search problem can be estimated by computing a rough upper limit.



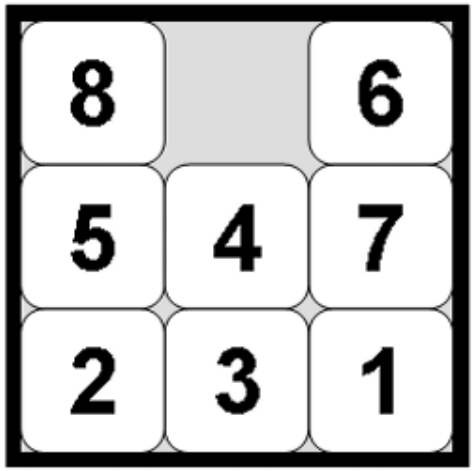
Maze

- The positions the agent can occupy
- n = number of grid cells the agent can be in



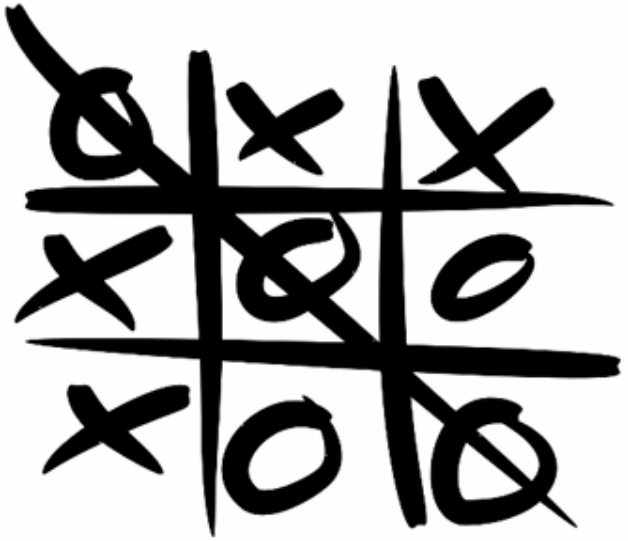
8-queens problem

- Place 8 queens on 64 (8×8) squares
- $n = C(64, 8) \approx 4.4 \times 10^9$



8-puzzle problem

- Arrange the 8 tiles in order
- 9 cells to fill, blank included, so $n \leq 9!$
- Half are unreachable:
- $n = 9! / 2 = 181,440$

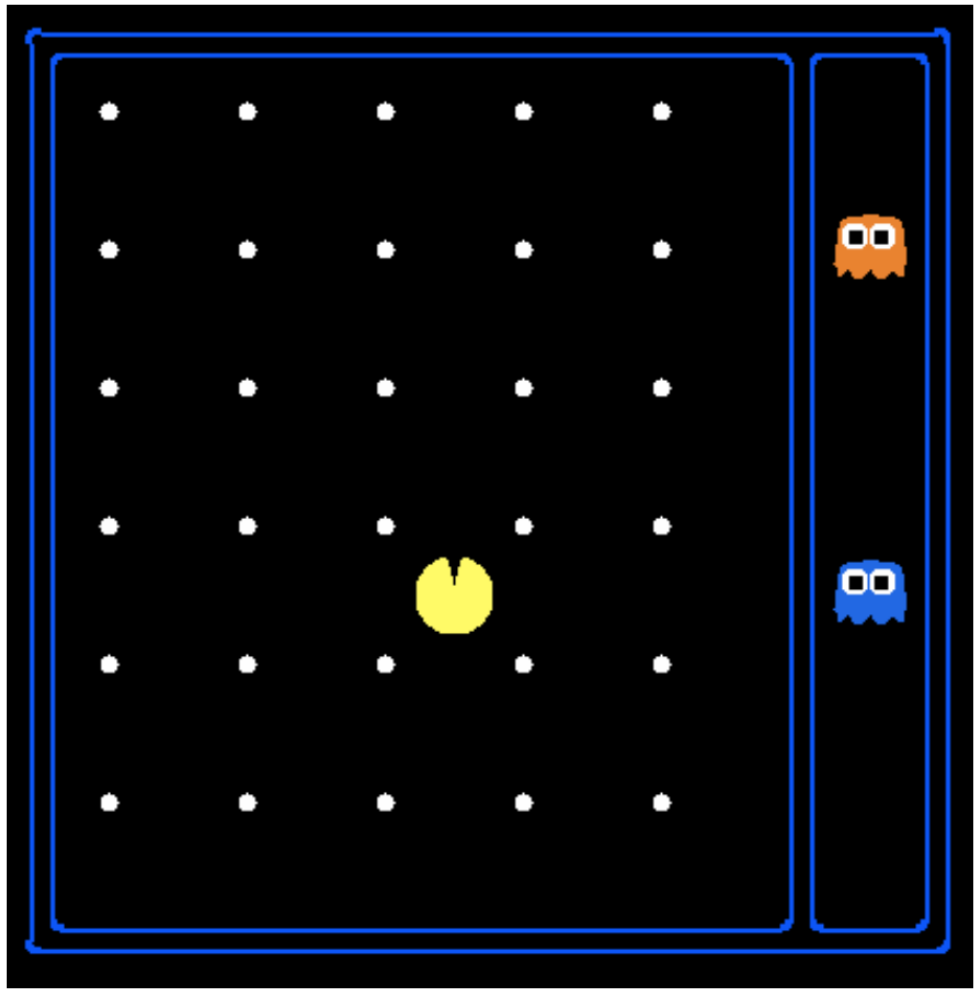


Tic-tac-toe

- Possible board states
- $n < 3^9 = 19,683$

Example: State-Space Size in Pac-Man

- World state: agent position on a 12x10 grid.
- Status of **30 dots (eaten or not)**, two ghosts each on a **12x1** corridor, and the agent's direction (NSEW).
- How many world states?
  - $120 * 2^{30} * 12^2 * 4 = 74{,}217{,}034{,}874{,}880$
- How many states for pathing? 120
- How many states for eat-all-dots? $120 * 2^{30}$

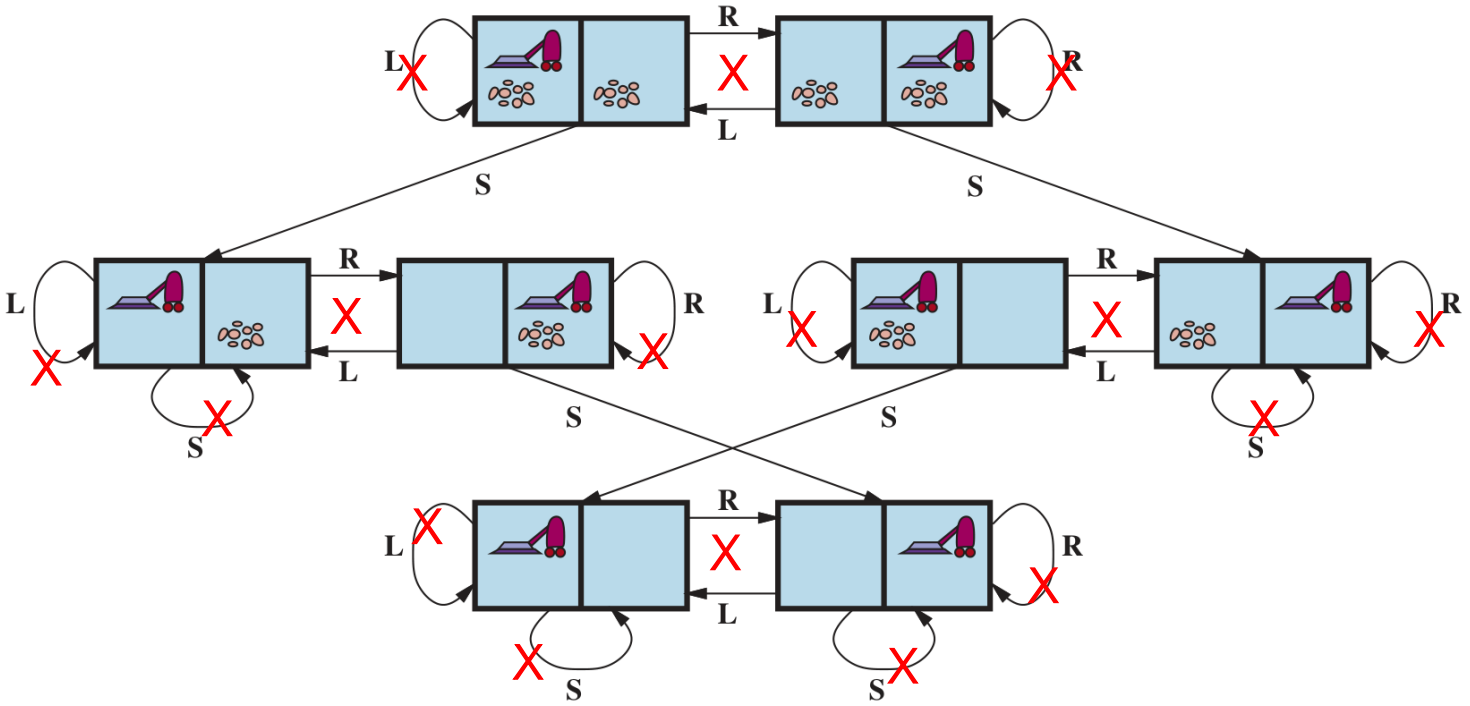


Search Space: Estimating the Search-Tree Size

- Often **more useful** than estimating the state space
- It matters especially where **redundant paths** let one state be represented by many nodes.
- Estimated from the problem formulation — initial state, actions, transition function
- The metrics we use:
  - b — maximum branching factor = the largest number of possible actions
  - m — maximum tree depth = the length of the longest path once cycles are removed
  - d — depth of the optimal solution = the shortest path length from the initial state to a goal
- The number of searched nodes is $n = f(b, m, d) \Rightarrow O(f(b, m, d))$

Search Space: Reading b, m and d Off a Graph

- b — maximum branching factor = largest number of actions (L, R, S) → 3
- m — maximum tree depth = longest path with cycles removed → 5
- d — depth of the optimal solution = shortest path from the initial state to a goal → 3
- Duplicate edges are dropped to turn the transition-model graph into a tree



X : You must remove the cycles before you can count m.

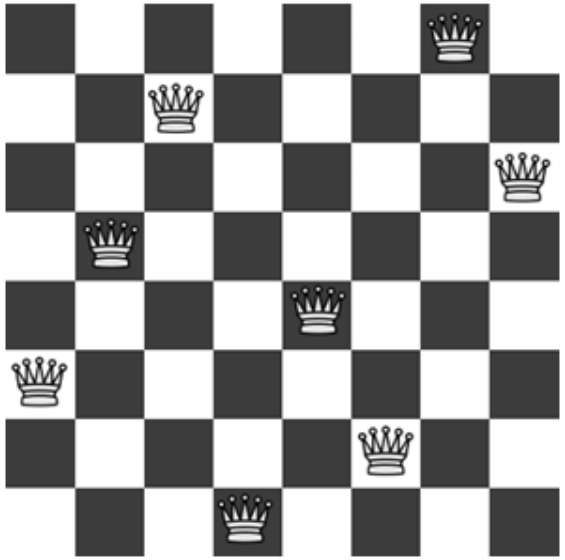
Example: Search Complexity

b: maximum branching factor · m: maximum depth of tree · d: depth of the optimal solution



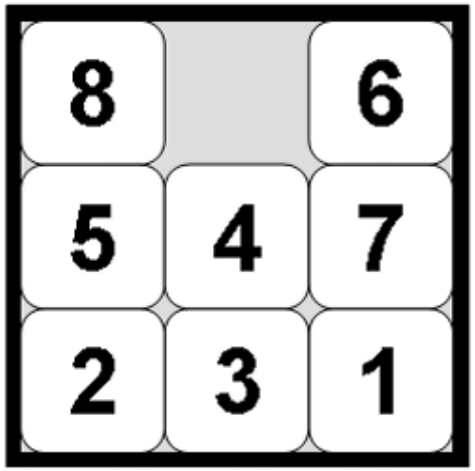
Maze

- b = 4 actions
- m = longest path to the goal (bounded by the x × y grid)
- d = shortest path to the goal (same bound)



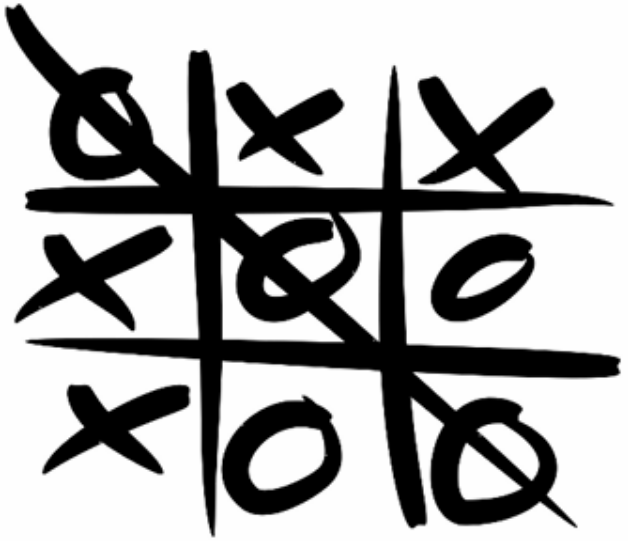
8-queens problem

- b = 8, eight rows in a column
- m = 8, there are eight columns
- d = 8, one queen placed per column



8-puzzle problem

- b = move the blank tile in 4 directions
- m = try all, O(9!/2)
- d ≤ 31

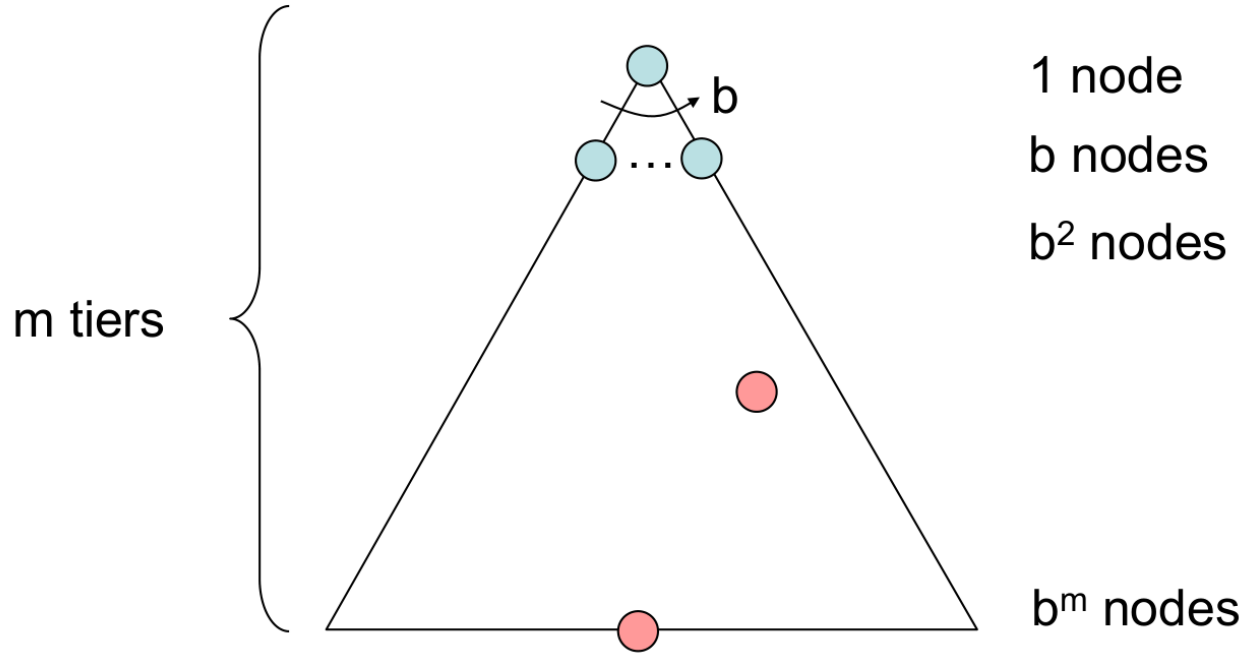


Tic-tac-toe

- b = 9 actions
- m = 9
- d = 5 (xoxox -> x player get straight)

How Many Nodes Are in the Whole Tree?

- b is the branching factor and m the maximum depth — the same two just measured
- Each tier holds b times as many nodes as the one above it: $1, b, b^2, \ldots, b^{\mathrm{m}}$
- $1 + b + b^2 + \cdots + b^{\mathrm{m}} = O(b^{\mathrm{m}})$. :The whole tree costs about what its bottom row costs.
- The red nodes are goals, and they sit at various depths — which is exactly why d and m are two different numbers
- Every time and space bound in Part II (the next lecture) is a piece of this sum.



Introduction to Artificial Intelligence

Chapter 3 - Part II: Uninformed Search

Namyup Kim

Soongsil University



4. Uninformed Search

Search with nothing but the problem definition — no idea which way the goal lies

- Queue, stack and priority queue: the frontier rule is the strategy
- Breadth-first and uniform-cost — complete, optimal, and hungry for memory
- Depth-first — linear space, and what that costs you
- Iterative deepening: BFS guarantees at DFS memory
- All four scored on completeness, cost optimality, time and space

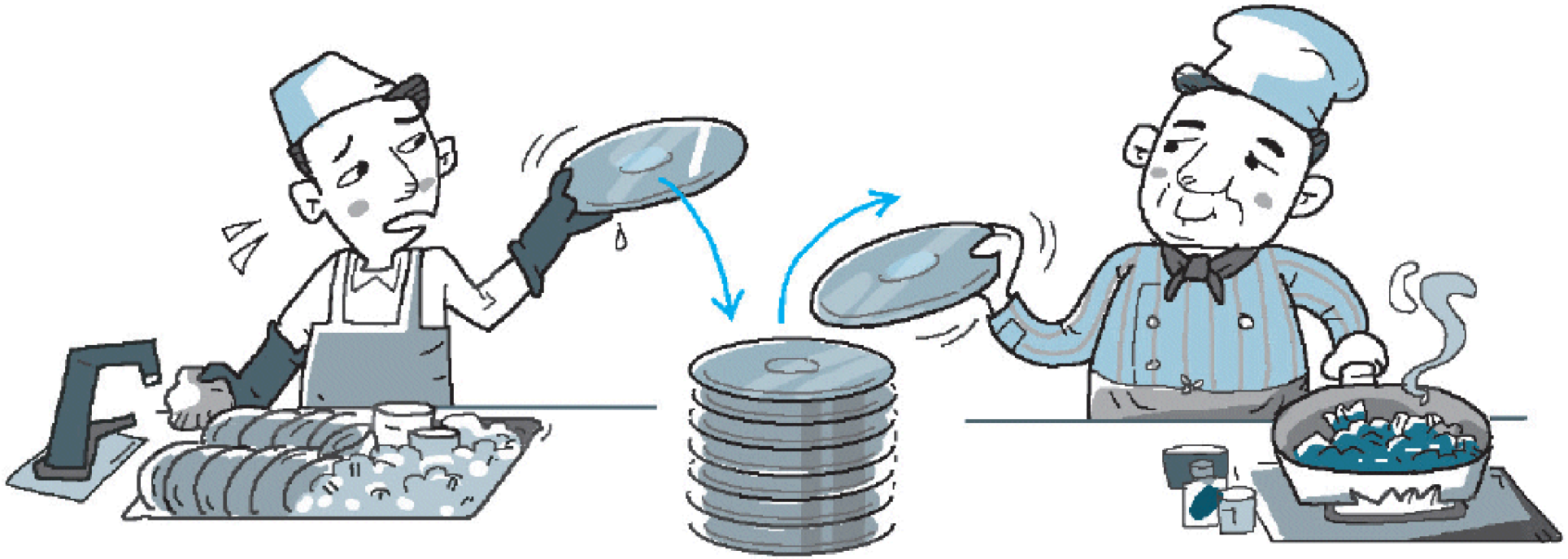
Uninformed Search: The Queue

- The data type where whatever went in first comes out first
  - First In, First Out (FIFO)
  - Examples: a queue of people, a bank ticket number



Uninformed Search: The Stack

- The data type where whatever went in last comes out first
  - Last In, First Out (LIFO)
  - In principle only the top item can be added or removed



Uninformed Search: Queue as an Abstract Data Type

Queue — FIFO

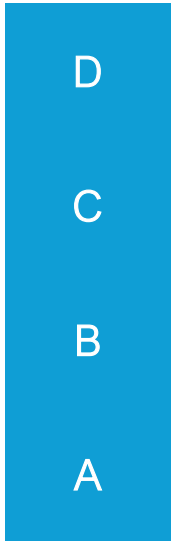


out in



Take the oldest.
Explores level by level → breadth-first search

Stack — LIFO



in / out



Stack of bowls

Take the newest.
Dives down one branch → depth-first search

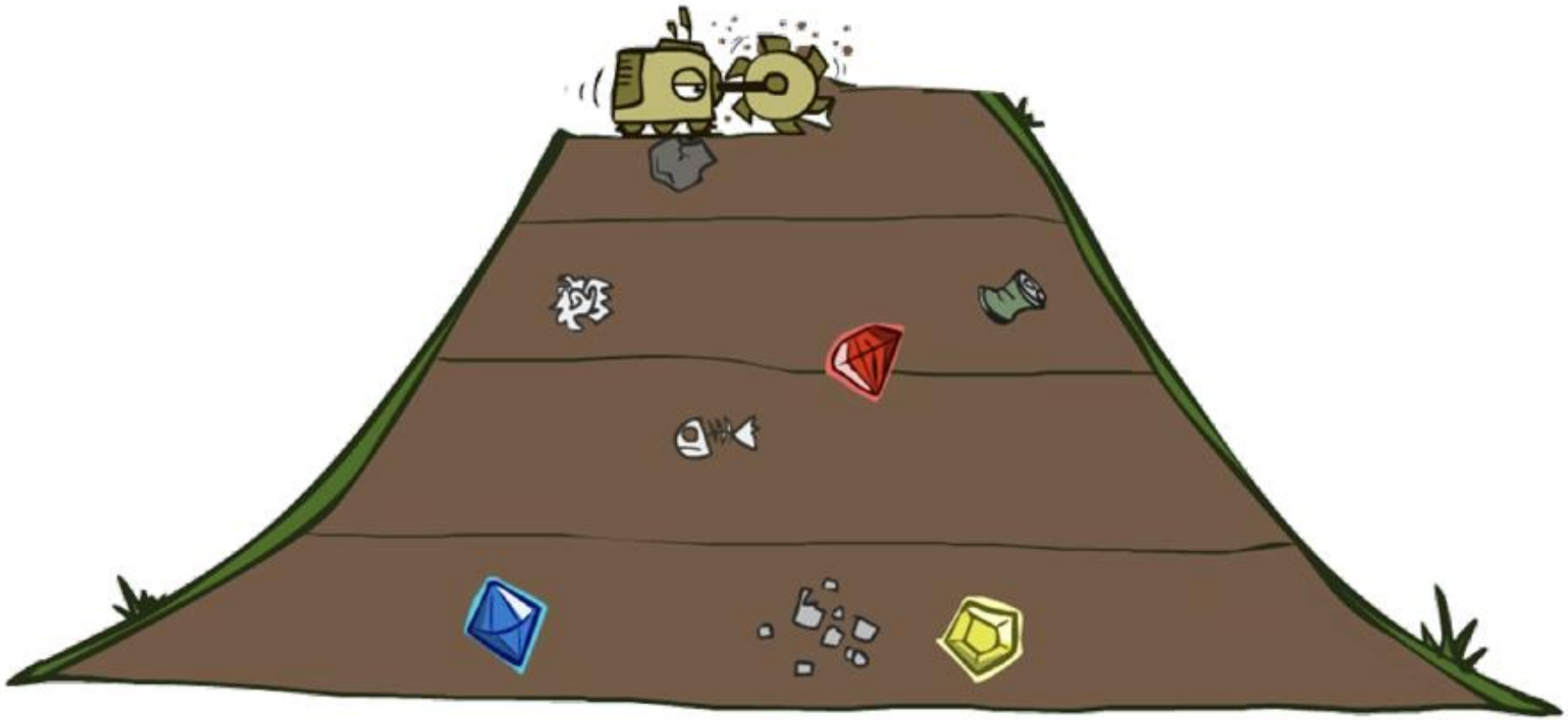
Insertion at the rear, deletion at the front.
The four operations — IS-EMPTY, POP, TOP, ADD — are the same whichever rule the frontier uses; only ADD changes.

Uninformed Search Strategies

- The search algorithm (the planning agent) is given **no information about how close a state is to the goal**.
- It uses only the labels of the atomic states and the transition function
- It searches blindly, following a simple strategy, until it reaches a goal state
- The strategies:
  - Breadth-first strategy — BFS and Uniform-Cost Search (UCS)
  - Depth-first strategy — DFS and Iterative Deepening Search (IDS)

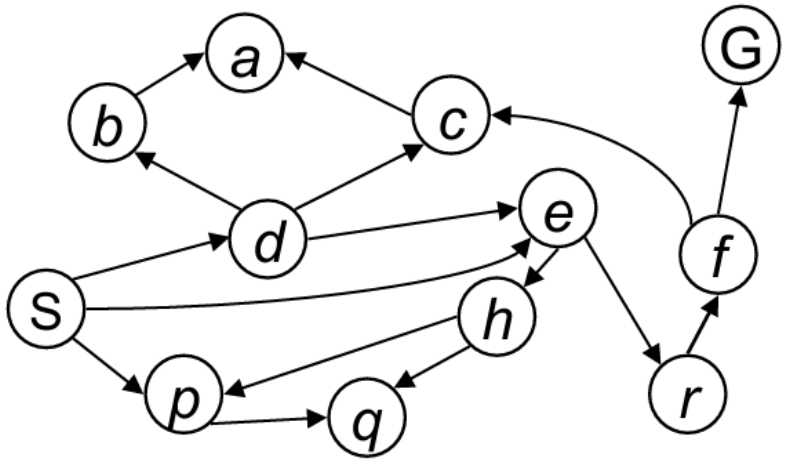
Uninformed Search: Breadth-First Search (BFS)

- Expansion rule: Expand the shallowest unexpanded node on the frontier.

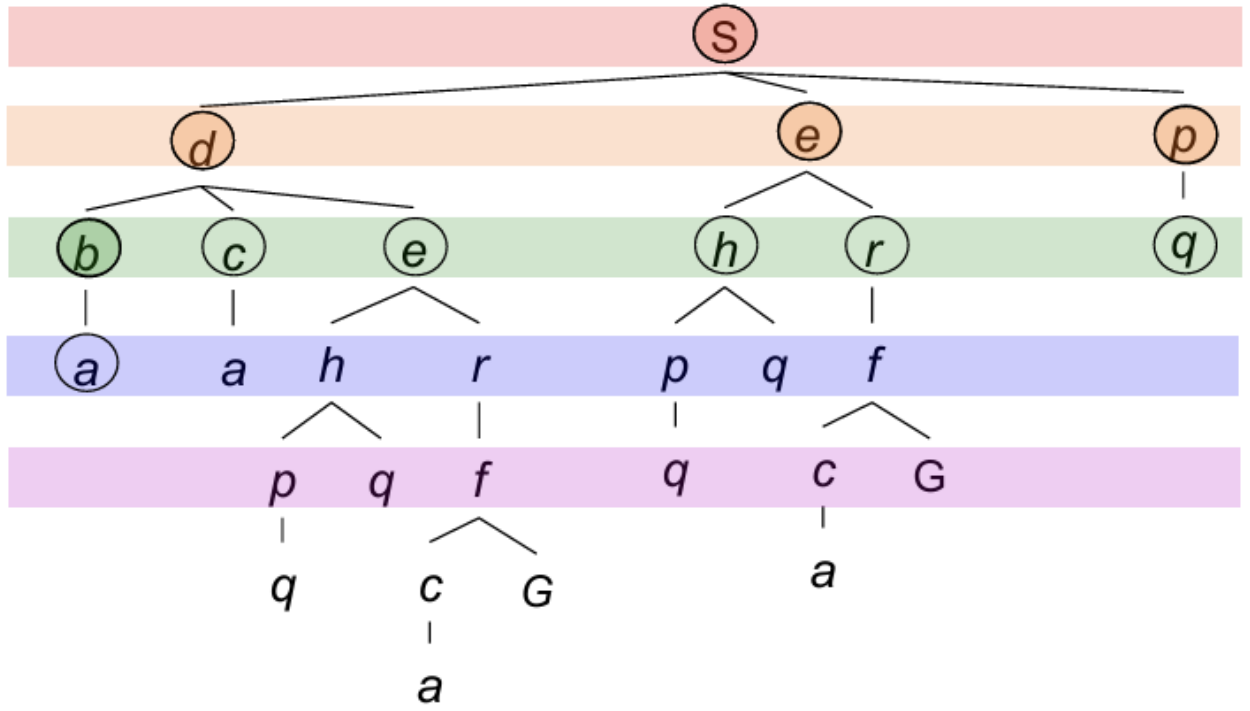


Uninformed Search: Breadth-First Search (BFS)

- Expansion rule: Expand the shallowest unexpanded node on the frontier.
  - Frontier: known, not yet expanded. (**a FIFO queue**)



Search Tiers



Uninformed Search: Breadth-First Search (BFS)

- Expansion rule: Expand the shallowest unexpanded node on the frontier.
  - Frontier: the green nodes: known, not yet expanded. (a FIFO queue)
  - Reached: every node visited, which is what stops a state being visited twice
  - Together they build the **full tree**, with every node holding the parent–child link that lets the solution path be recovered.

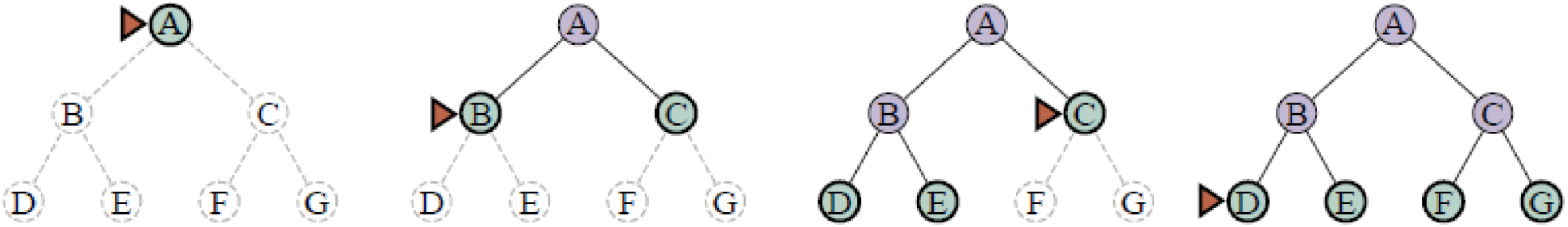


Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

Implementation: Breadth-First Search

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

Expansion adds the node's lower level to the frontier.

Reached stops a node being visited twice — a cycle, or any other redundant path.

reached stops a node being visited twice — a cycle, or any other redundant path. EXPAND adds the level below to the frontier.

Implementation: Expanding the Search Tree

- Tree search in AI builds **the search tree as it goes**.
- The **EXPAND** function applies the transition model (**RESULT**) to try every action available in the current node's state. It returns the list of new child nodes to be added to the frontier.

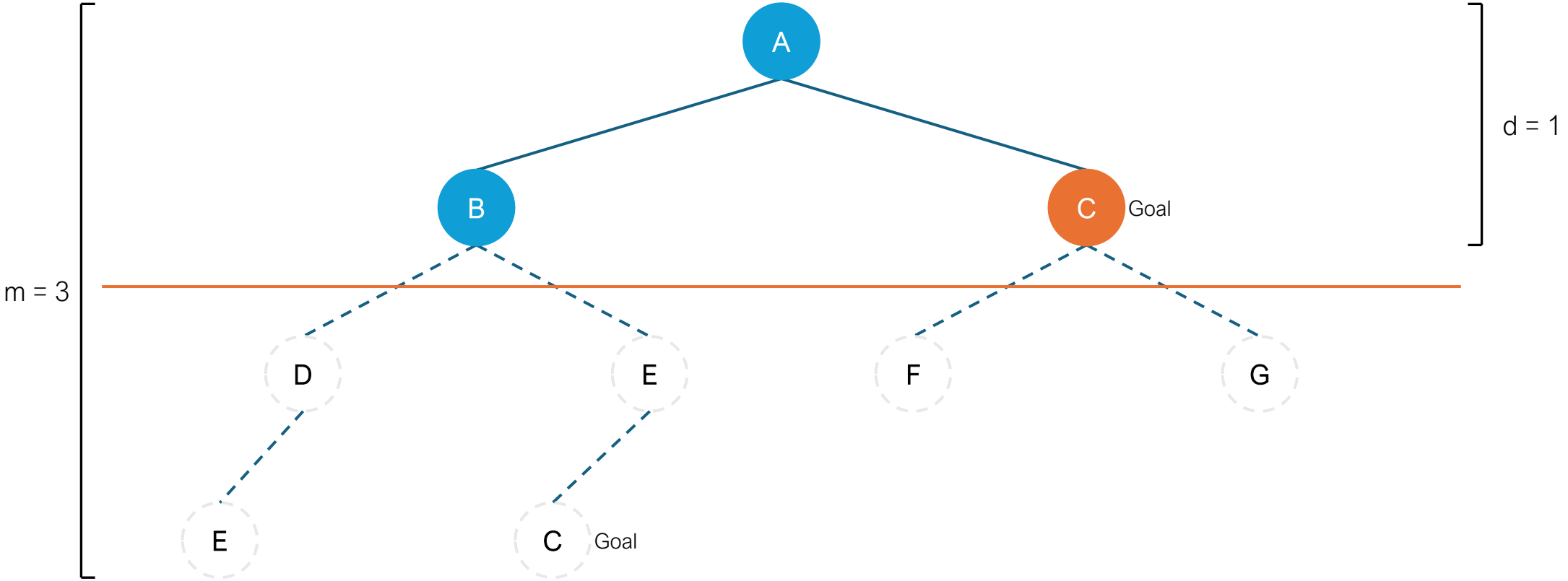
```
function EXPAND(problem, node) yields nodes
    s ← node.STATE
    for each action in problem.ACTIONS(s) do
        s' ← problem.RESULT(s, action)
        cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
        yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Transition function

A node records four things: **its state**, **its parent**, **the action that produced it**, and the **path cost** so far.
yield may equally be implemented as returning a list of nodes.

Time and Space Complexity: Breadth-First Search

b: maximum branching factor · m: maximum depth of tree · d: depth of the optimal solution



$$1 + b + b^2 + \ldots + b^{\mathrm{d}} \Rightarrow O(b^{\mathrm{d}}) \quad \text{(here } b = 2, d = 1, m = 3\text{)}$$

Everything above the orange line has been expanded. If every path down to the goal depth is expanded, the search tree has 1 + b + b² + … + bᵈ nodes.

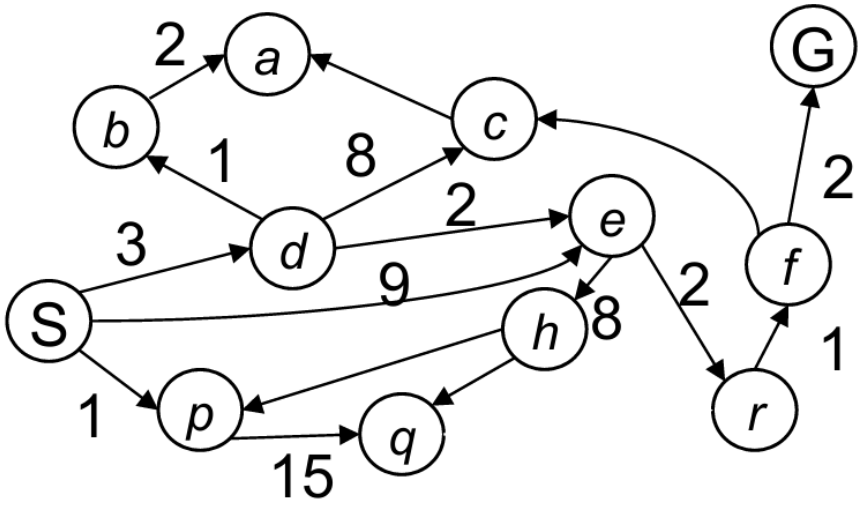
Properties of Breadth-First Search

- Completeness: does a solution exist and will it be found? Yes
- Cost optimality: will it find the lowest-cost solution?
  - Yes, when every step costs the same. Otherwise, use uniform-cost search.
- Time complexity: nodes generated - $O(b^d)$
- Space complexity: nodes stored - $O(b^d)$

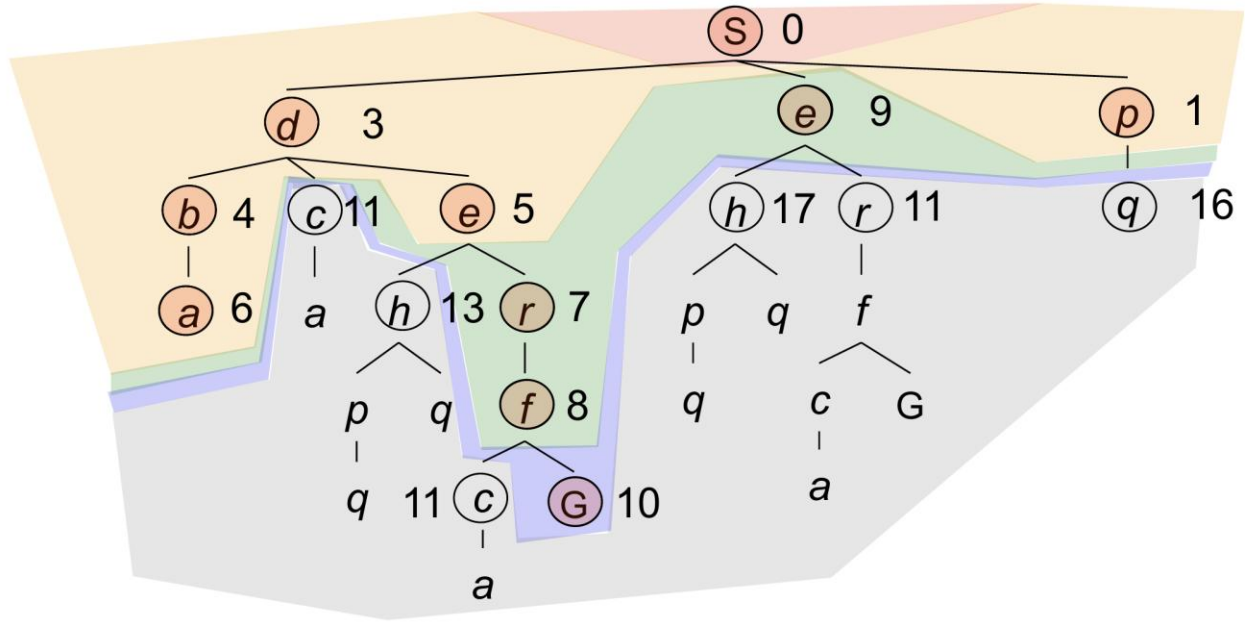
- Note: In AI, a large space complexity is usually a bigger problem than time.

Uniform-Cost Search (= Dijkstra's Algorithm)

- The word "**uniform**" means it searches consistently by lowest cost
- Expansion rule: expand a cheapest node first.

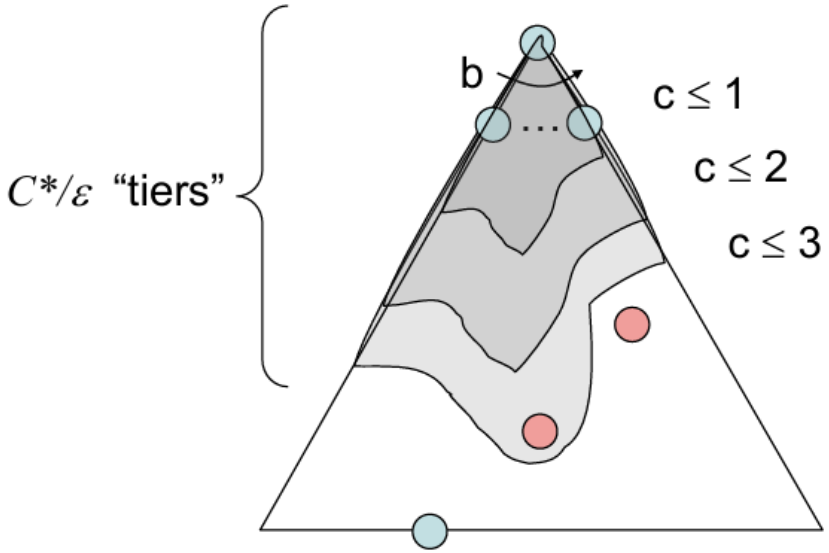


Cost contours



Uniform-Cost Search (= Dijkstra's Algorithm)

- The word "**uniform**" means it searches consistently by lowest cost
- Expansion rule: expand the frontier node **with the lowest path cost** from the initial state
- Implementation: **best-first search**, with the frontier as **a priority queue** ordered by
$$f(n) = g(n) = \ path\ cost$$
- Completeness: yes, provided every step cost exceeds some small positive constant $\boldsymbol{\varepsilon}$.
- Cost optimality: yes; nodes are expanded in order of increasing path cost (Proof next lecture via A*)
- Time and space complexity: $O(b^{(C^*/\varepsilon)})$,
  - expanding every node with cost $c \leq C^*$ (the cost of the optimal solution)
  - **This can be larger than BFS's $O(b^d)$**: it may explore many cheap long paths before an expensive short one.



*Breadth-first search is **the special case of UCS where every step costs the same.***
= Flatten the contours

Implementation: Uniform-Cost Search

```
function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
  return BEST-FIRST-SEARCH(problem, PATH-COST)
```

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and v
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
```

The order of expansion is set by f(n) = the path cost from the initial state to n.

If a cheaper path to a state turns up later, we take it. (≠ BFS)

The order of expansion is set by f(n) = the path cost from the initial state to n. The extra check is what BFS does not have: revisit a node if a better — cheaper — path reaches it.

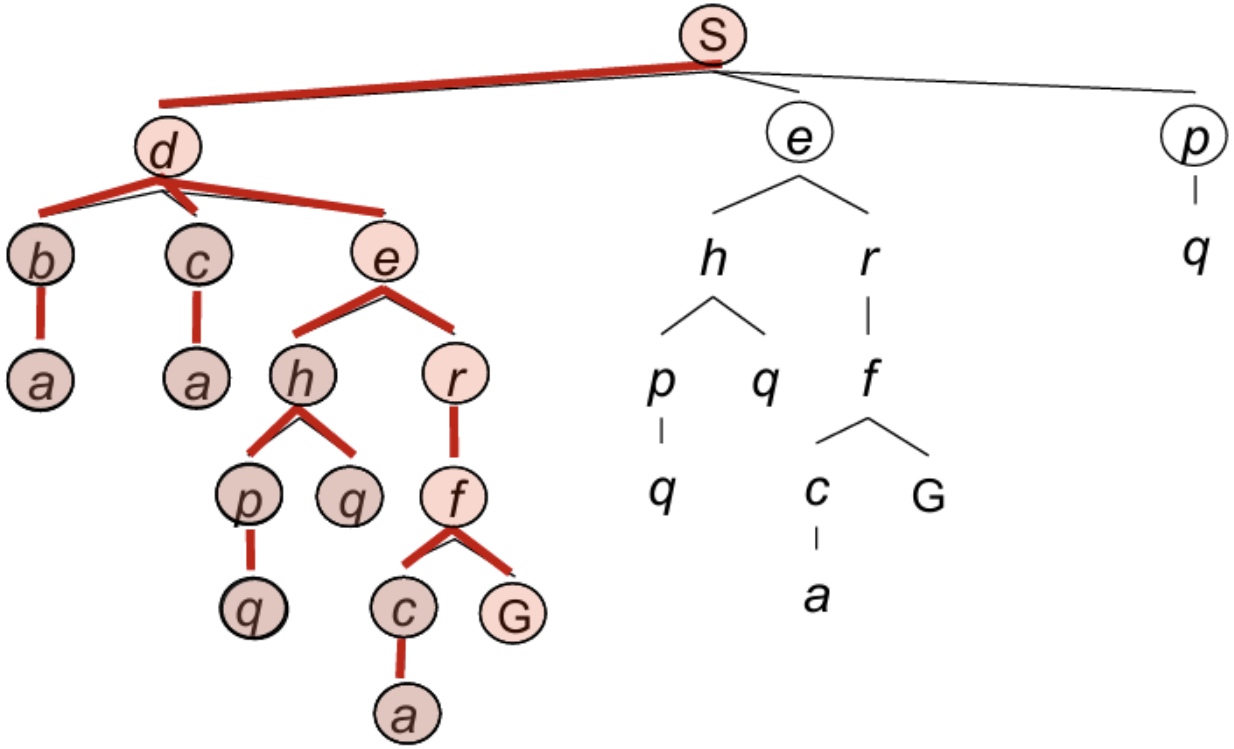
Uninformed Search: Depth-First Search (DFS)

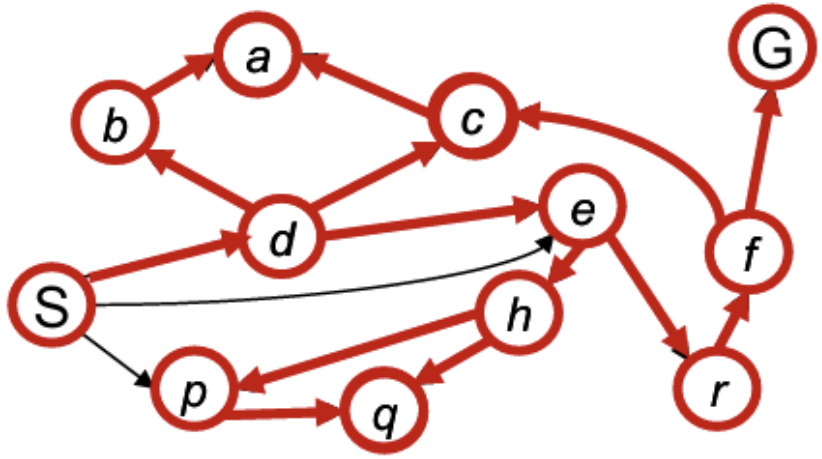
- Expansion rule: expand the deepest unexpanded node on the frontier



Uninformed Search: Depth-First Search (DFS)

- Expansion rule: expand the deepest unexpanded node on the frontier
- Frontier: a stack (LIFO)





Uninformed Search: Depth-First Search (DFS)

- Expansion rule: expand the deepest unexpanded node on the frontier
- Frontier: a stack (LIFO)
- No reached data structure is needed
- Redundant paths are not checked, so the same region may be searched more than once
- Cycle checking is done only along the current path

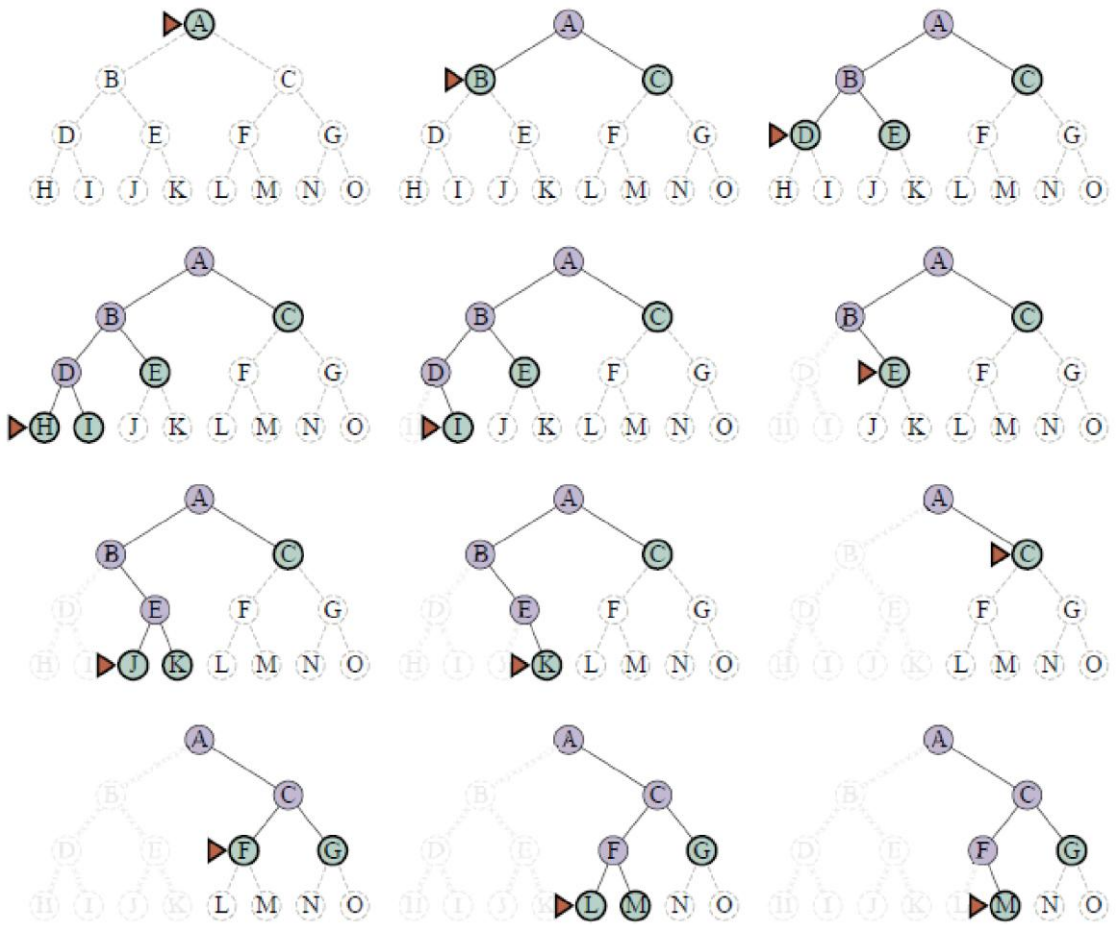


Figure 3.11 A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M. The frontier is in green, with a triangle marking the node to be expanded next. Previously expanded nodes are lavender, and potential future nodes have faint dashed lines. Expanded nodes with no descendants in the frontier (very faint lines) can be discarded.

Implementation: Depth-First Search

- DFS identical to BFS or best-first except that **it takes the last element off the frontier**.
- Implementation option
  - Iterative: abandoned branches are freed as the tree is built, and cycles are checked only along the current path — much like backtracking search.
  - Recursive: the current path lives on the call stack where the function cannot read it, so cycle checking needs a separate structure.

```
function DEPTH-LIMITED-SEARCH(problem, ℓ) returns a node or failure or cutoff
  frontier ← a LIFO queue (stack) with NODE(problem.INITIAL) as an element
  result ← failure
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if DEPTH(node) > ℓ then
      result ← cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        add child to frontier
  return result
```

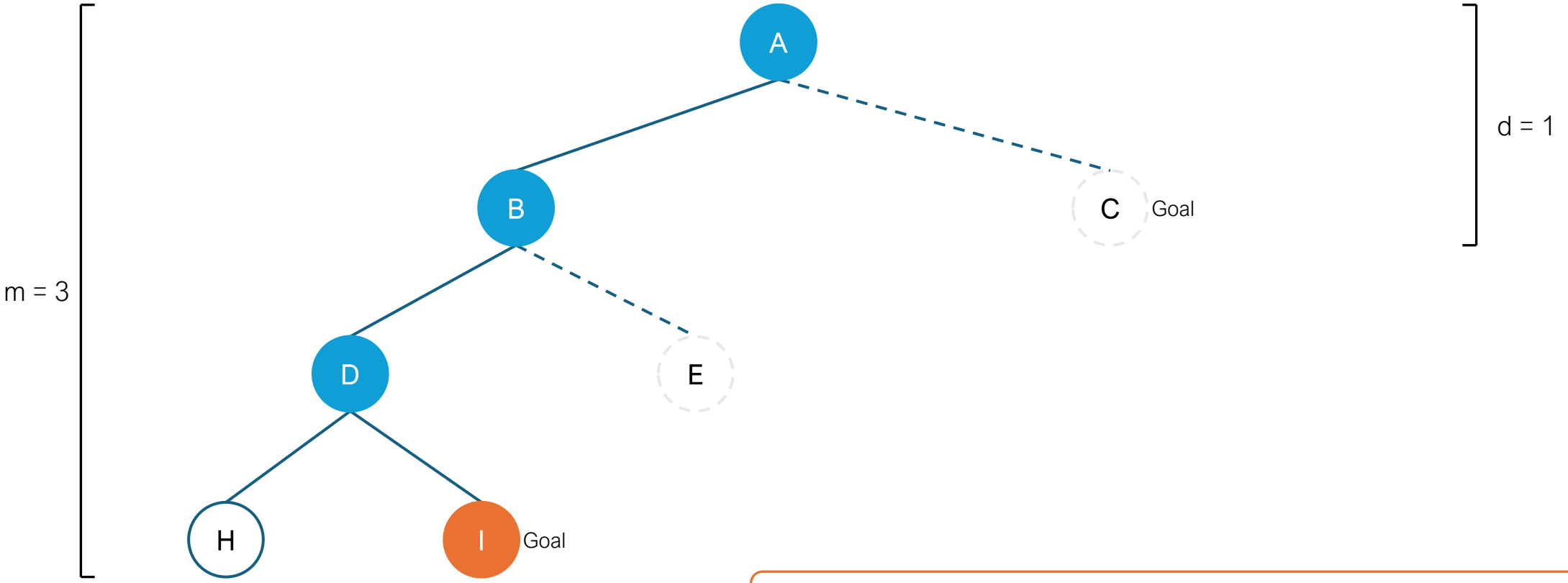
l could be ∞ in DFS

No reached data structure:
Memory management.

IS − CYCLE also stops the same state sitting on the frontier. twice. But, Redundant paths cannot be prevented.

Time and Space Complexity: Depth-First Search

b: maximum branching factor · m: maximum depth of tree · d: depth of the optimal solution



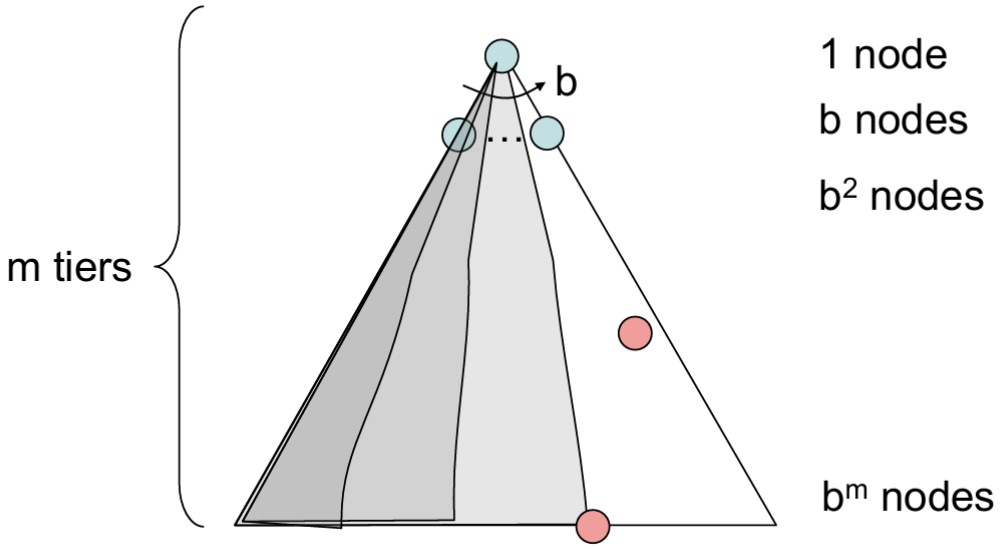
DFS returns the FIRST goal it reaches — not the cheapest

Time $O(b^m)$, Space(=memory) $O(bm)$ (here b = 2, d = 1, m = 3)

Time complexity: Worst case, it expands every path.

Properties of Depth-First Search

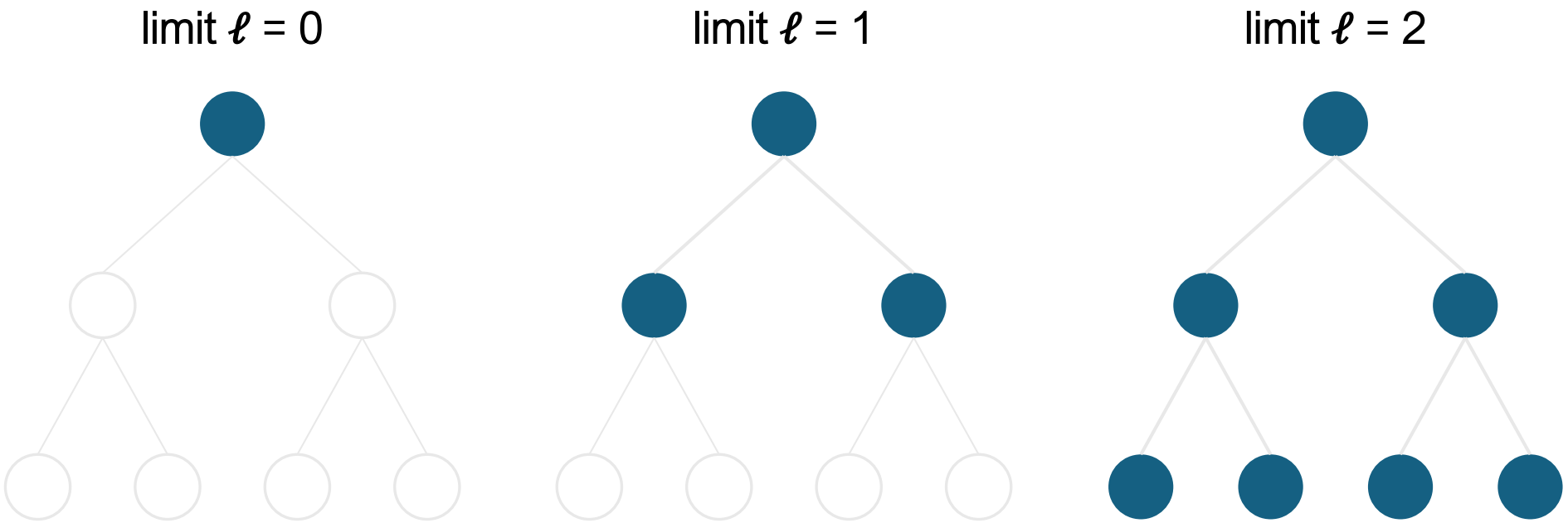
- Completeness?
  - m could be infinite, so only if we prevent cycles (more later)
- Cost optimality — no. It returns the first solution it finds.
- Time complexity — $O(b^{\mathrm{m}})$
  - worst case, it reaches maximum depth m on the last path
  - if $m \gg d$, this is inefficient compared to BFS
- Space complexity — $O(bm)$, linear in the maximum tree depth.
  - but only if no reached structure is used.
  - cycles can be cut; **redundant paths** cannot be checked.



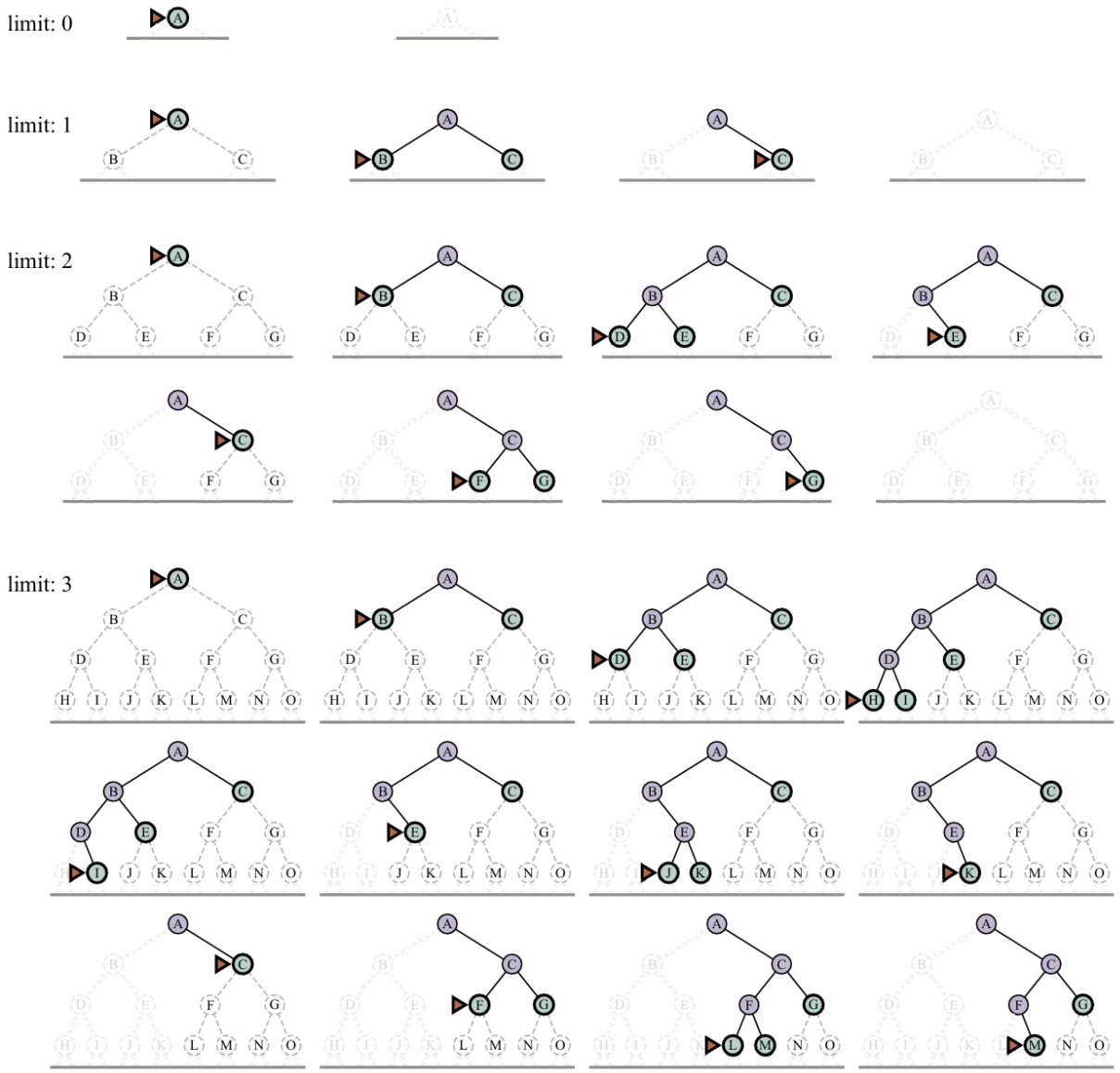
Iterative Deepening Search (IDS)

- Can we have
  - DFS's low memory use,
  - avoid its infinite loops, and
  - keep BFS's optimality guarantee?

- Check the root; then DFS for a path of length 1; then length 2; then length 3; …



Iterative Deepening Search



Implementation: Iterative Deepening Search

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
  for depth = 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result
```

```
function DEPTH-LIMITED-SEARCH(problem, ℓ) returns a node or failure or cutoff
  frontier ← a LIFO queue (stack) with NODE(problem.INITIAL) as an element
  result ← failure
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if DEPTH(node) > ℓ then
      result ← cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        add child to frontier
  return result
```

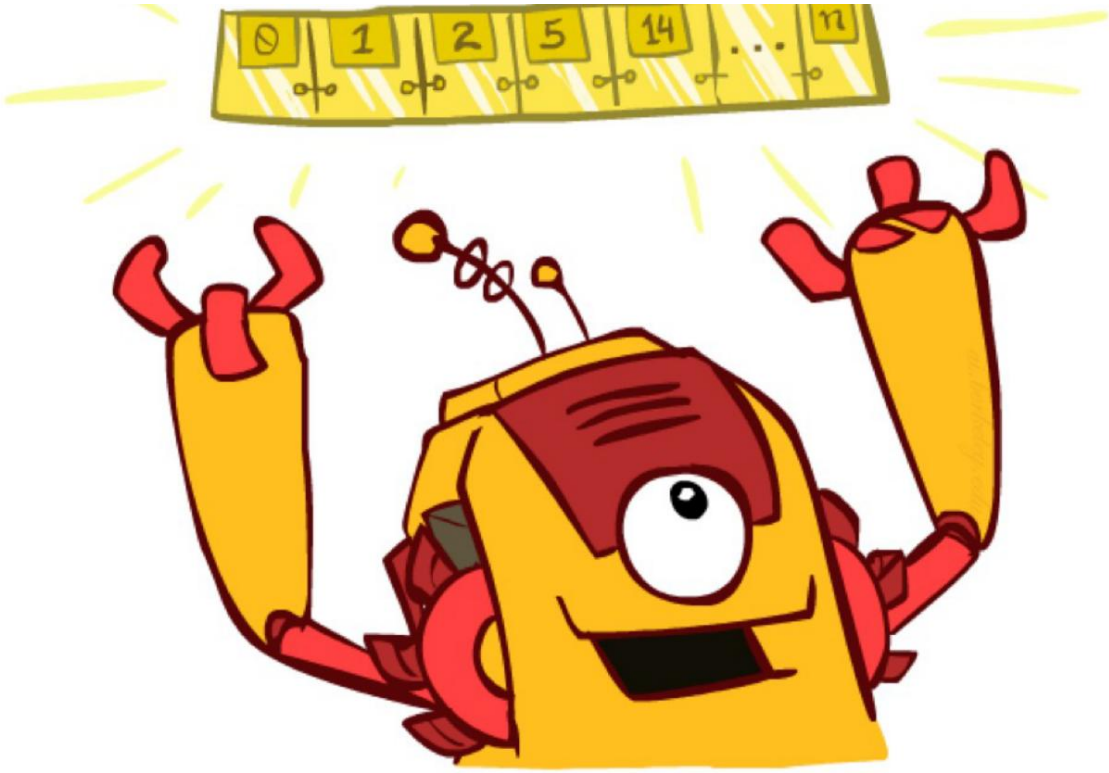
- cutoff means "no solution within this limit"; failure means "no solution at all". The outer loop needs the difference to know when to stop.

Properties of Iterative Deepening Search

- Completeness — yes
- Cost optimality — yes
- Time complexity — it rebuilds the tree up to d times:
  - $db + (d-1)b^2 + \ldots + 1 \cdot b^{\mathrm{d}} = O(b^{\mathrm{d}})$
  - slower than BFS, but in the same complexity class.
- Space complexity — $O(bd)$, linear.
  - Even less than DFS, since $d \leq m$.
  - cycles have to be handled inside the depth-limited DFS
- IDS gives the same result as BFS, at a longer running time but a far better space complexity
  - This is why IDS and DFS are the workhorses of AI

The One Queue: Every Strategy Is One Algorithm

- Breadth-first, uniform-cost, depth-first and iterative deepening are one algorithm.
  - Only the frontier rule differs: FIFO queue, priority queue, LIFO stack, stack with a depth cutoff
- Conceptually every frontier is a priority queue: nodes with priorities attached
- Practically, depth-first and breadth-first avoid the $\log(n)$ overhead of a real priority queue by using a stack and a plain queue
- One implementation with the queuing object passed in gives all four — and Part III changes nothing but the priority: f = g + h



Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

Introduction to Artificial Intelligence

Chapter 3 - Part III: Informed Search

Namyup Kim

Soongsil University



5. Informed Search

Now the agent gets a **hint** (informed). One function changes everything.

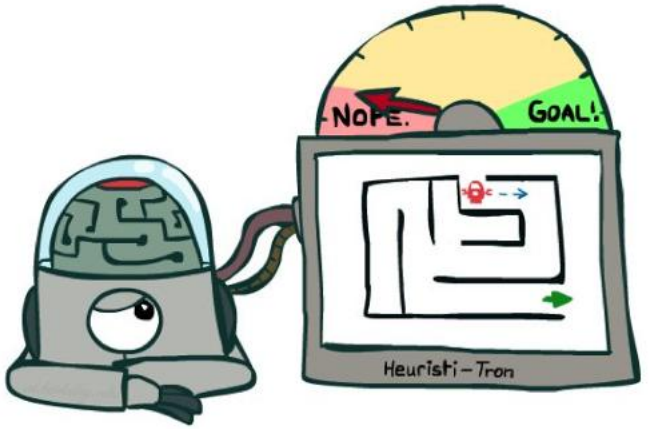
- $h(n)$: **the estimated cost** from a node to the goal
- Greedy best-first search: use h alone, and watch it miss the shorter route by 32 miles
- A* search: $f(n) = g(n) + h(n)$, with the proof that it is optimal
- Admissibility and consistency - what a heuristic has to promise
- Designing heuristics: relaxed problems, subproblems, dominance, pattern databases
- Weighted A*, and the one parameter that contains every algorithm in this chapter

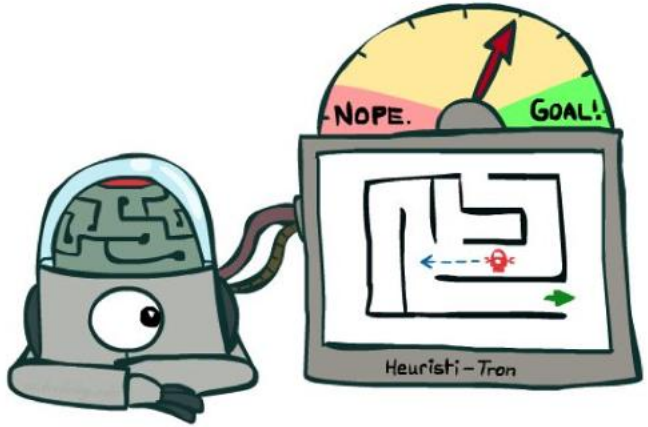
Informed Search

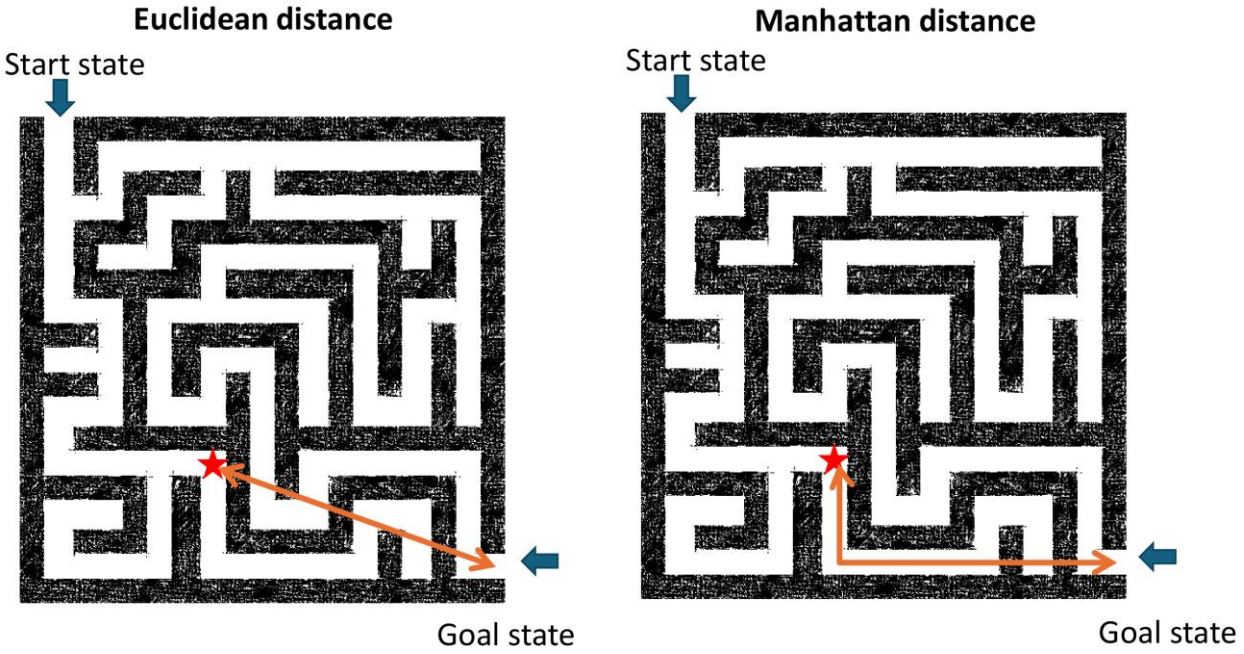
- AI search problems generally have very large search spaces.
  → We want to improve efficiency by expanding as **few** nodes as possible
- The agent can use extra information — a "hint" — to explore the more promising states first
  - information the agent already has (e.g. a map with the goal marked)
  - derived from percepts arriving through its sensors (e.g. a GPS sensor and the goal coordinates)
- The agent uses a heuristic function $h(n)$ to rank the nodes on the frontier, and a best-first search strategy always expands the most promising node on it.
- Informed search algorithms:
  - Greedy best-first search
  - A* search

Heuristic Function

- The heuristic function $h(n)$ estimates how close a state is to a goal.
- Examples:
  - Euclidean distance: straight line, ignoring obstacles
  - Manhattan distance; sum of the horizontal and vertical steps

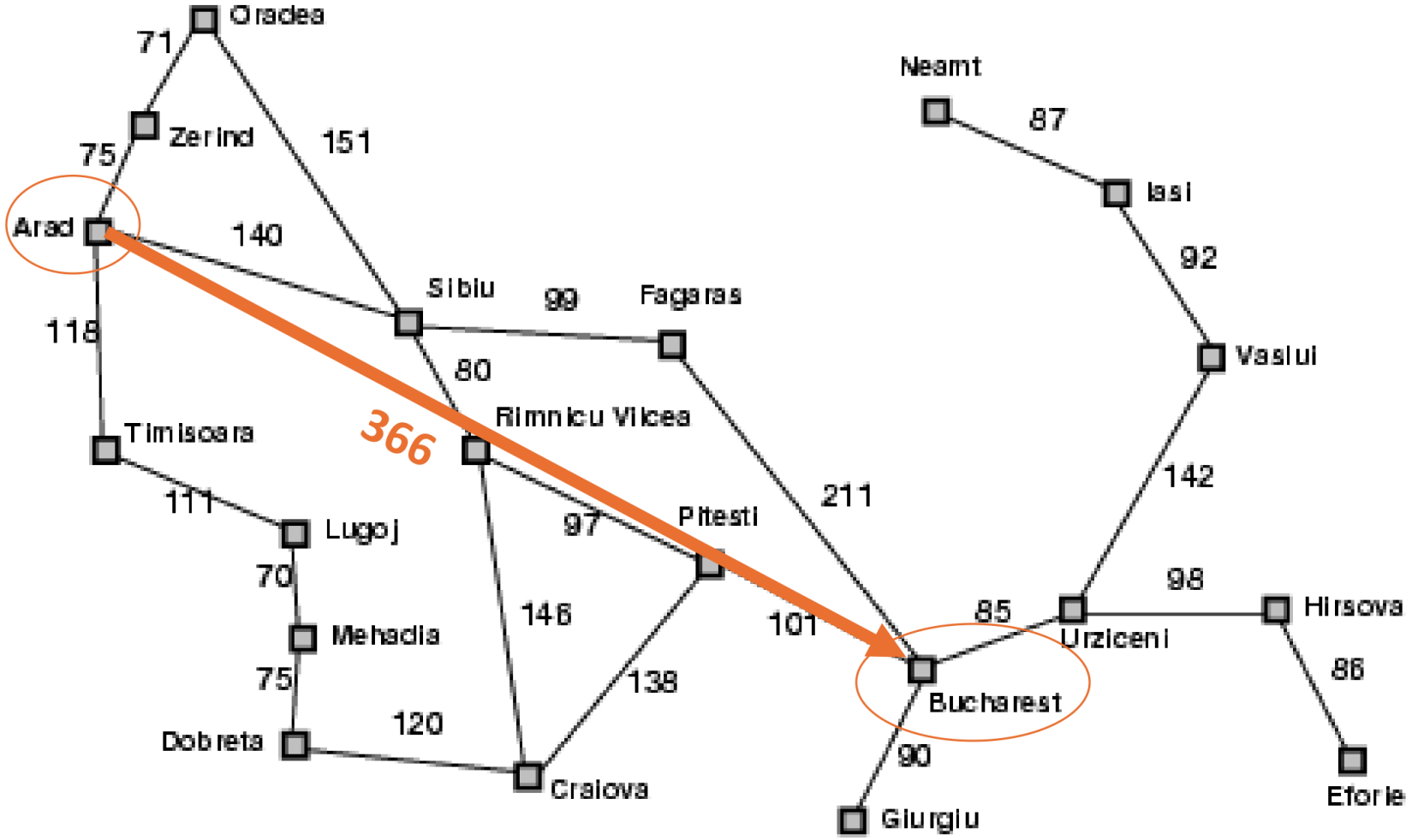






Example: A Heuristic for the Romania Problem

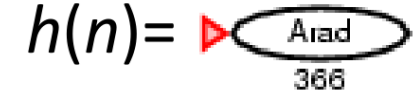
- $h(n)$: estimate the driving distance from Arad to Bucharest by the straight-line distance (Euclidian)

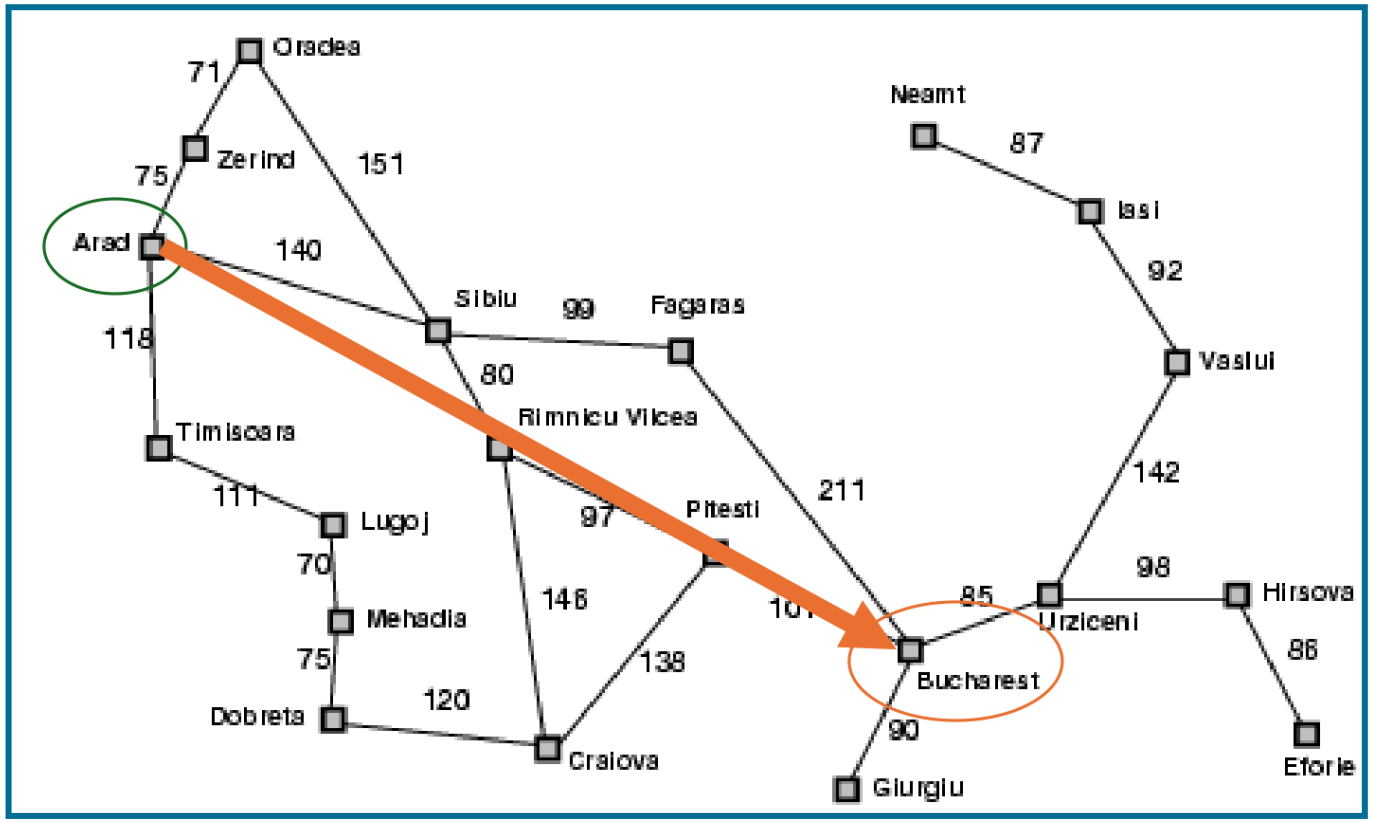


Straight-line distance to Bucharest	$= h(n)$
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: Greedy Best-First Search

- **Expansion rule**: expand the node with the lowest heuristic value $h(n)$
- Start at Arad, h = 366

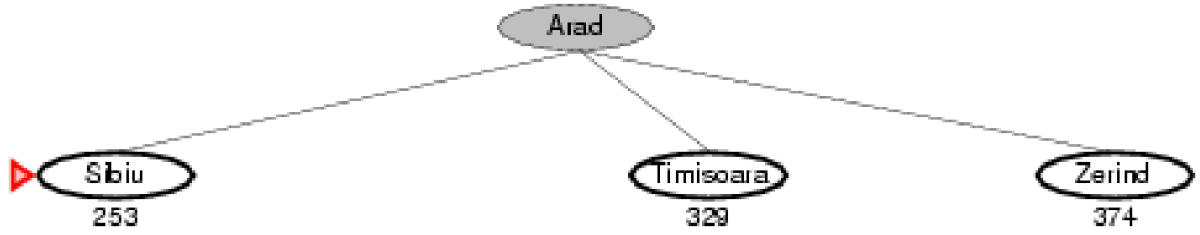
$h(n)=$ 

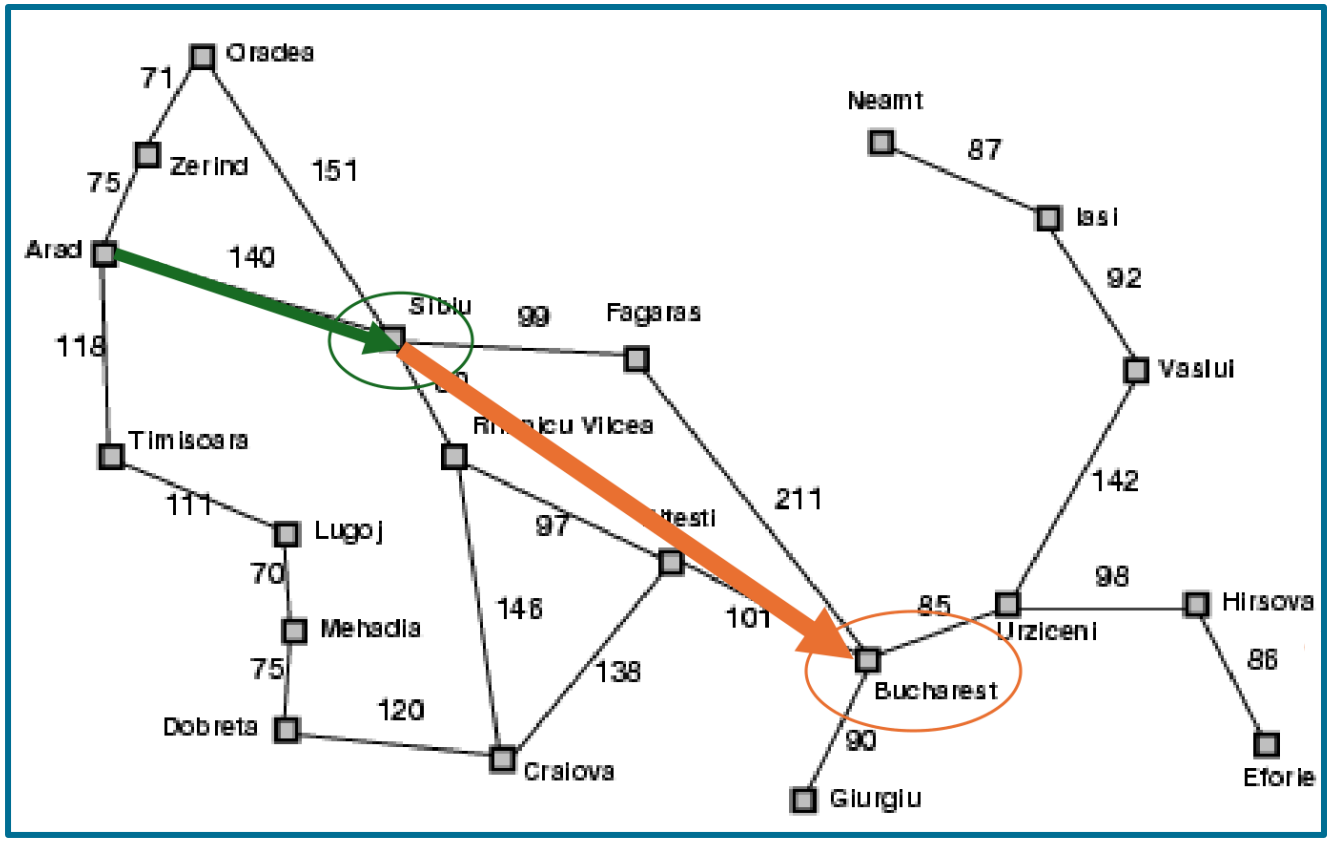


Straight-line distance to Bucharest	$= h(n)$
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: Greedy Best-First Search

- Expand Arad. Of Sibiu (253), Timisoara (329) and Zerind (374), Sibiu has the lowest h



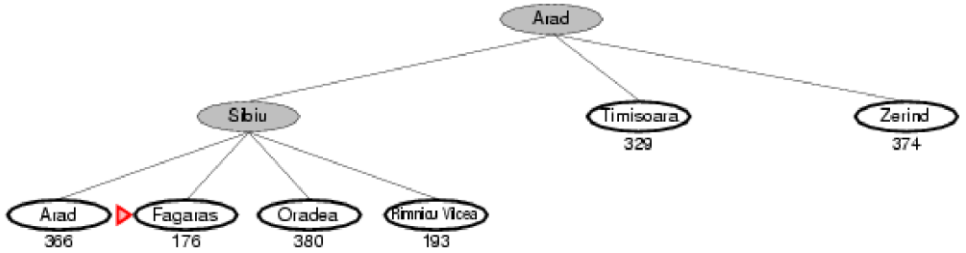


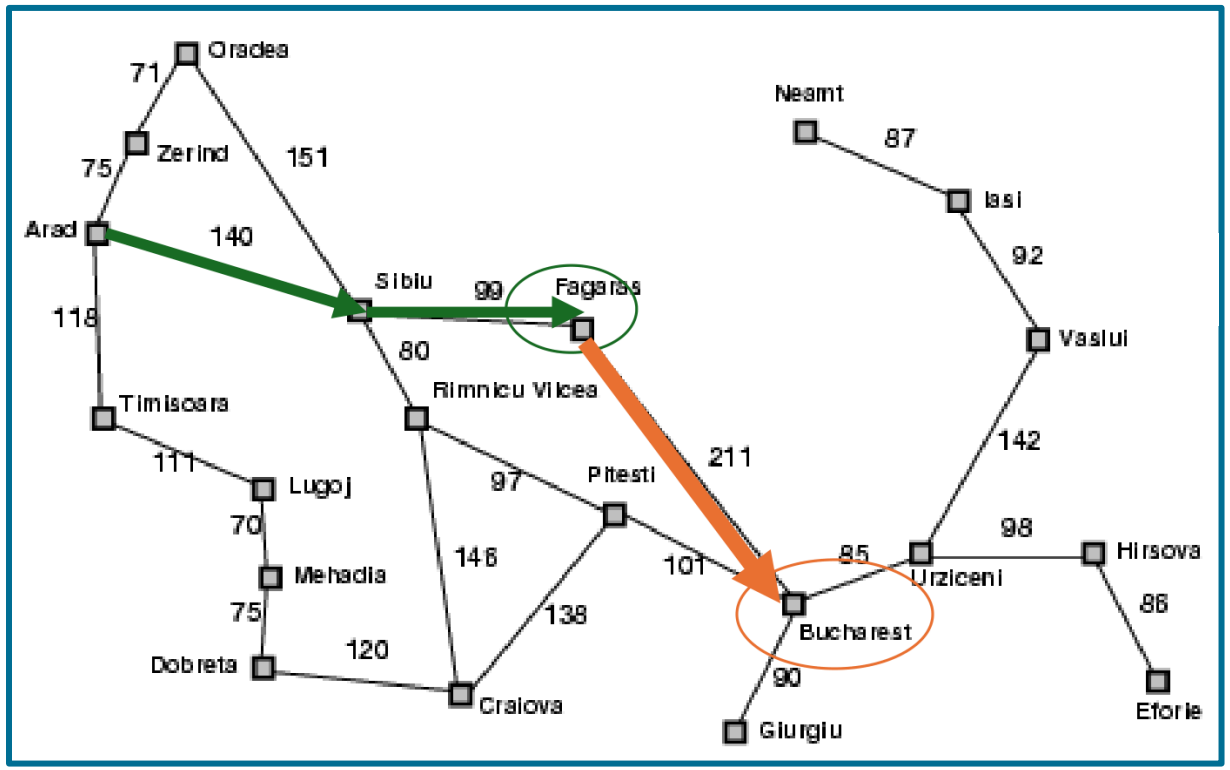
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: Greedy Best-First Search

- Expand Sibiu. Fagaras has $h = 176$, the lowest among its children
- Note that Rimnicu Vilcea (193) is passed over



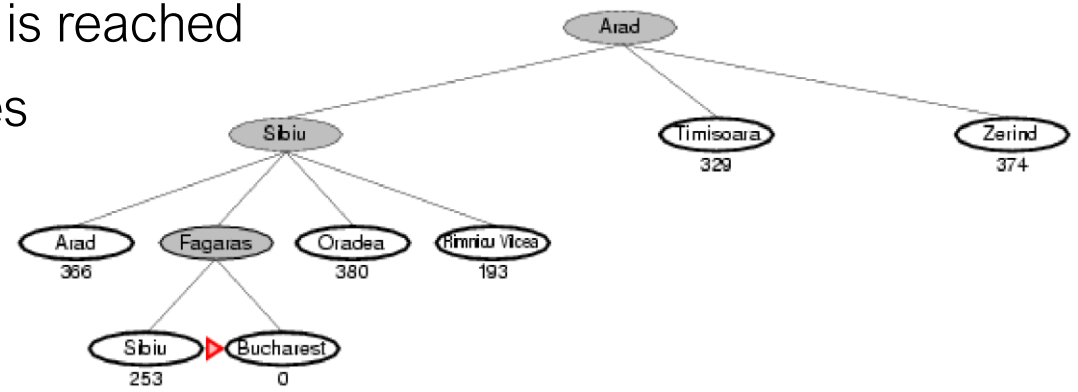


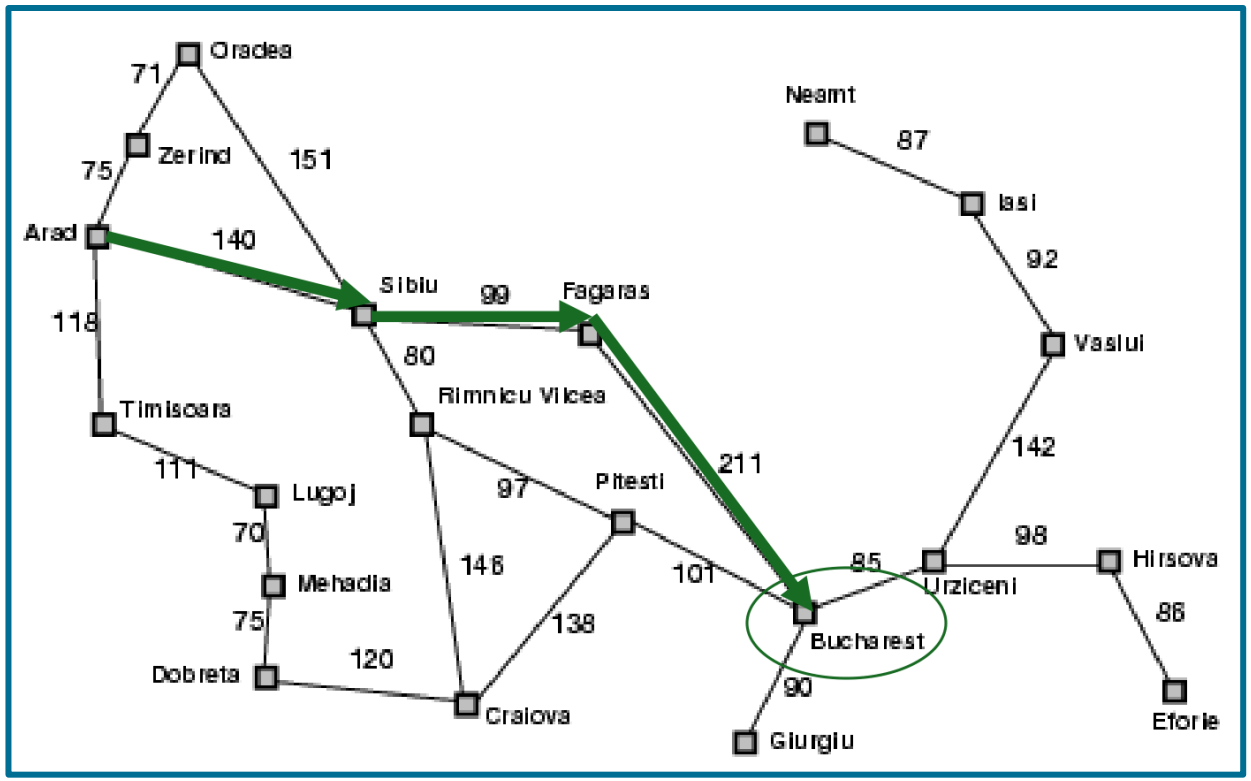
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: Greedy Best-First Search (4)

- Expand Fagaras, and Bucharest is reached
- Total: 140 + 99 + 211 = 450 miles



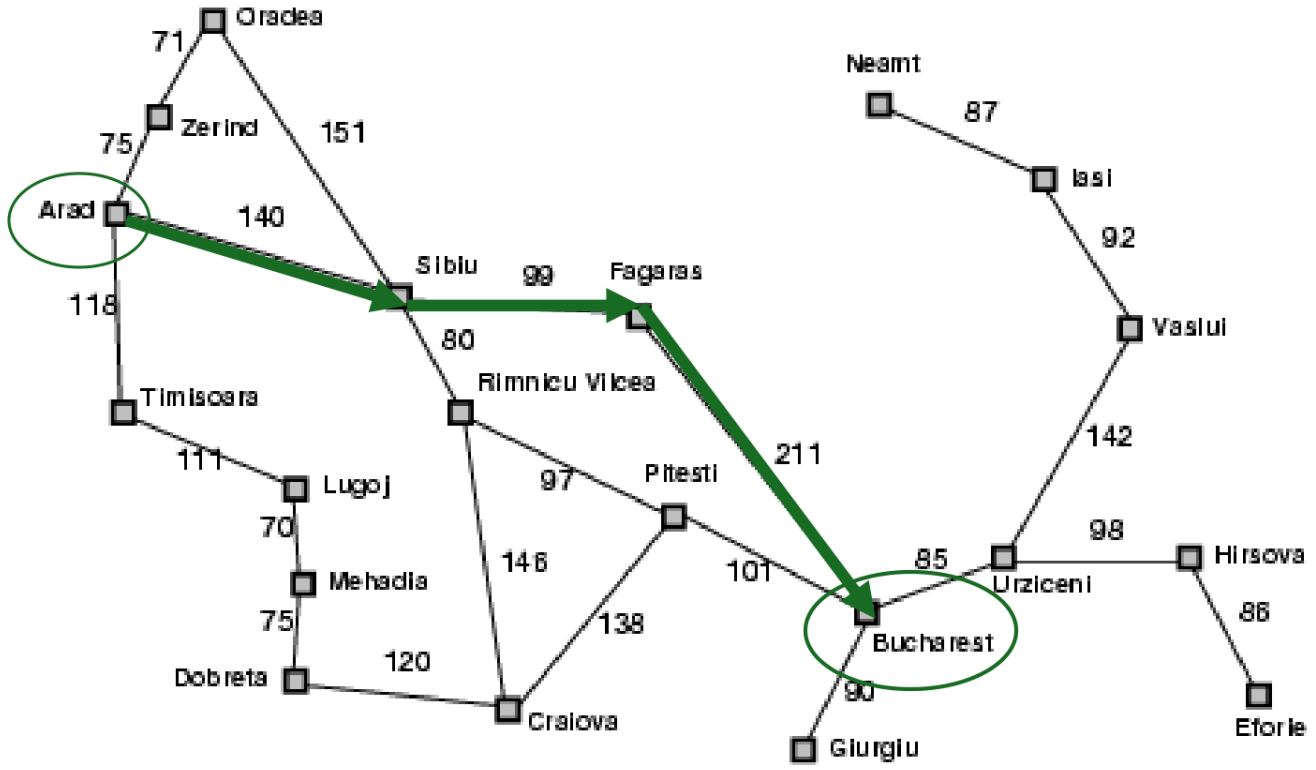


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Properties of Greedy Best-First Search

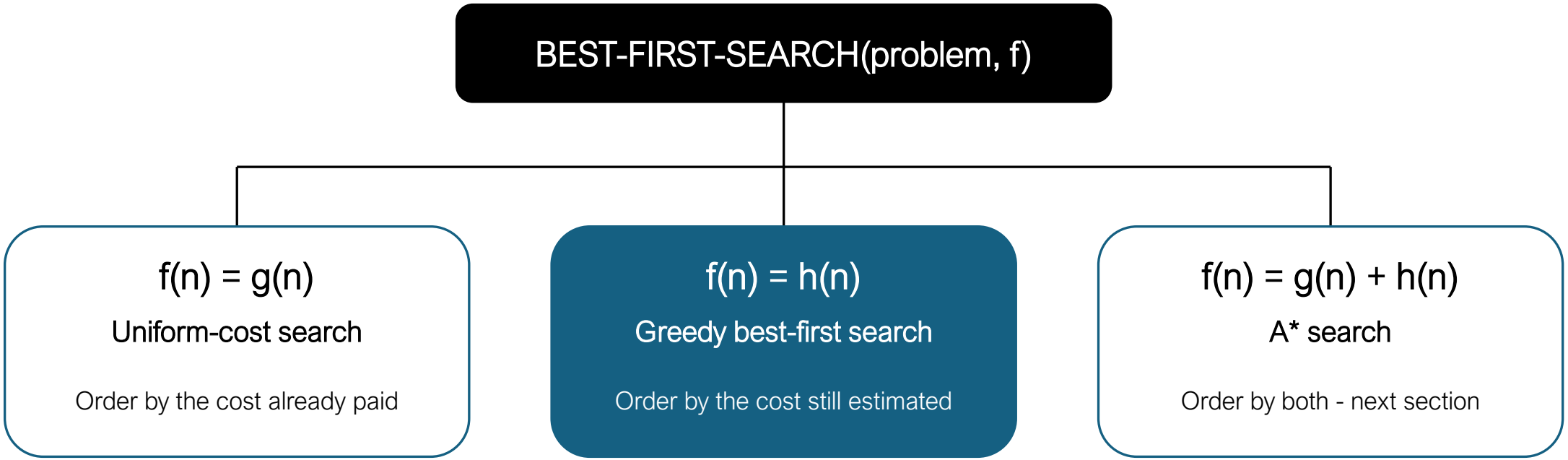
- Completeness? Yes — best-first search is complete in a finite space
- Cost optimality? No
  1. the path it found: 140 + 99 + 211 = 450 miles
  2. another path, through Rimnicu Vilcea: 140 + 80 + 97 + 101 = 418 miles



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Implementing Greedy Best-First Search

- It is the best-first search we already have
- Expand the frontier using $f(n) = h(n)$
- Nothing else changes — same loop, same frontier, same EXPAND



Three strategies, one loop (same). Only the key the frontier is sorted on changes.

Implementation: Greedy Best-First Search

Because $f(n) = h(n)$, it expands the node with the lowest estimated cost.

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

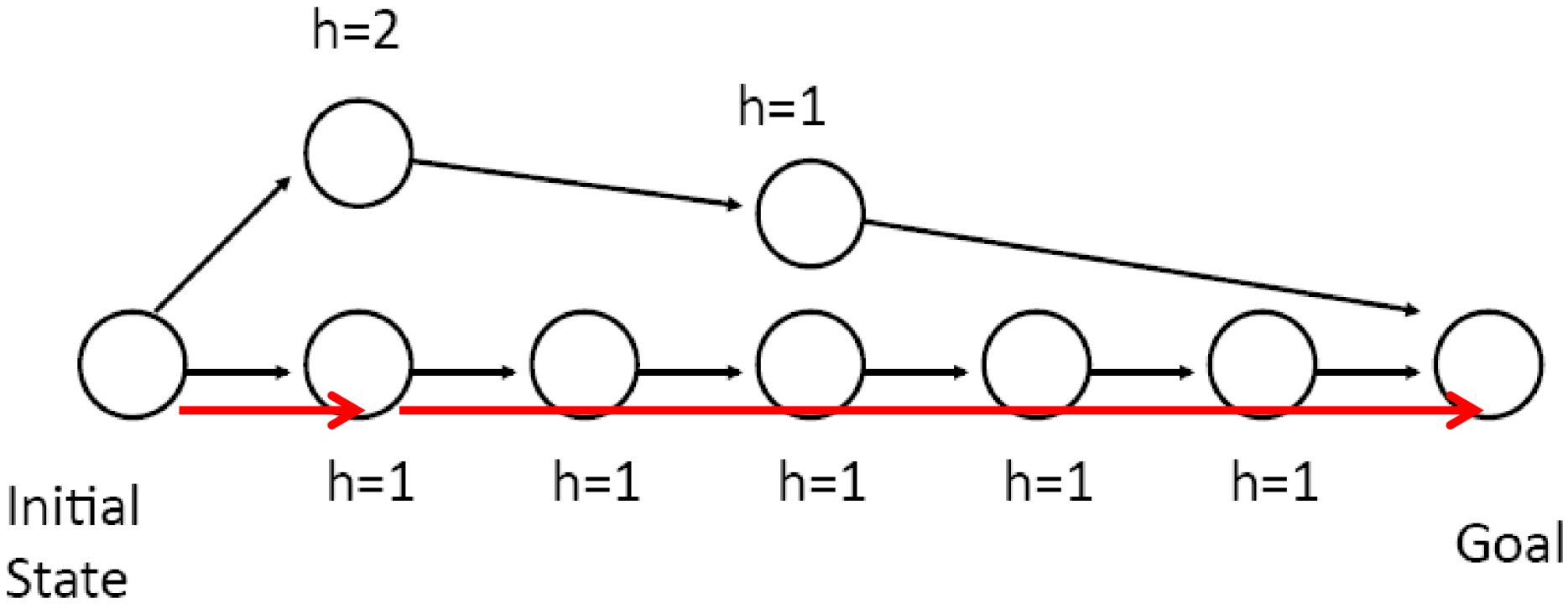
The order in which the frontier is expanded is decided by f(n).

Properties of Greedy Best-First Search (2)

- Completeness? Yes — best-first search is complete in a finite space
- Cost optimality? No
- Time complexity?
  - worst case: $O(b^m)$ — similar to DFS
  - best case: O(bm) — if h(n) were 100% accurate it would expand a single path
- Space complexity? The same as its time complexity
- Do you remember?
  - b: maximum branching factor, m: maximum depth of tree, d: depth of the optimal solution

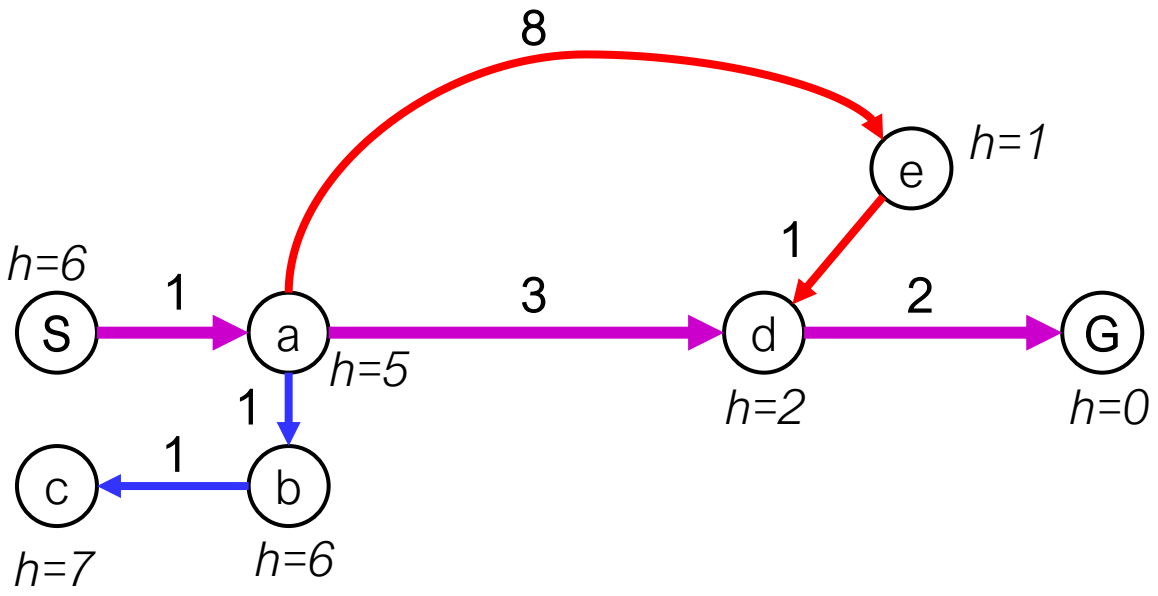
Why Greedy Best-First Search Is Not Cost Optimal

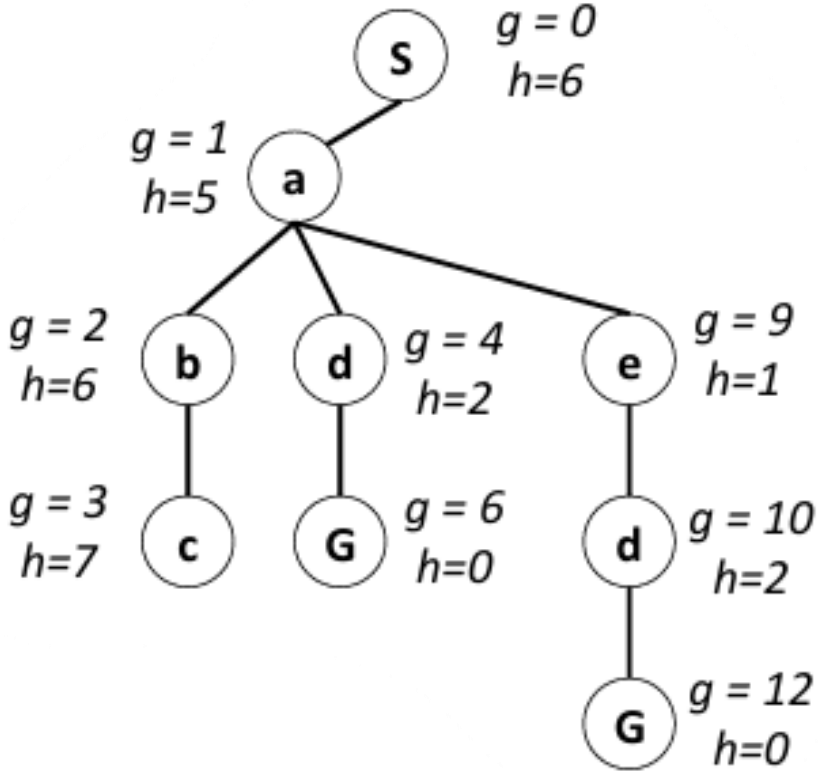
- $h = 1$ looks better than $h = 2$
- Greedy best-first search follows that path and never reconsiders it.
- It weighs only the estimated cost to the goal: never the cost already paid.



A* Search

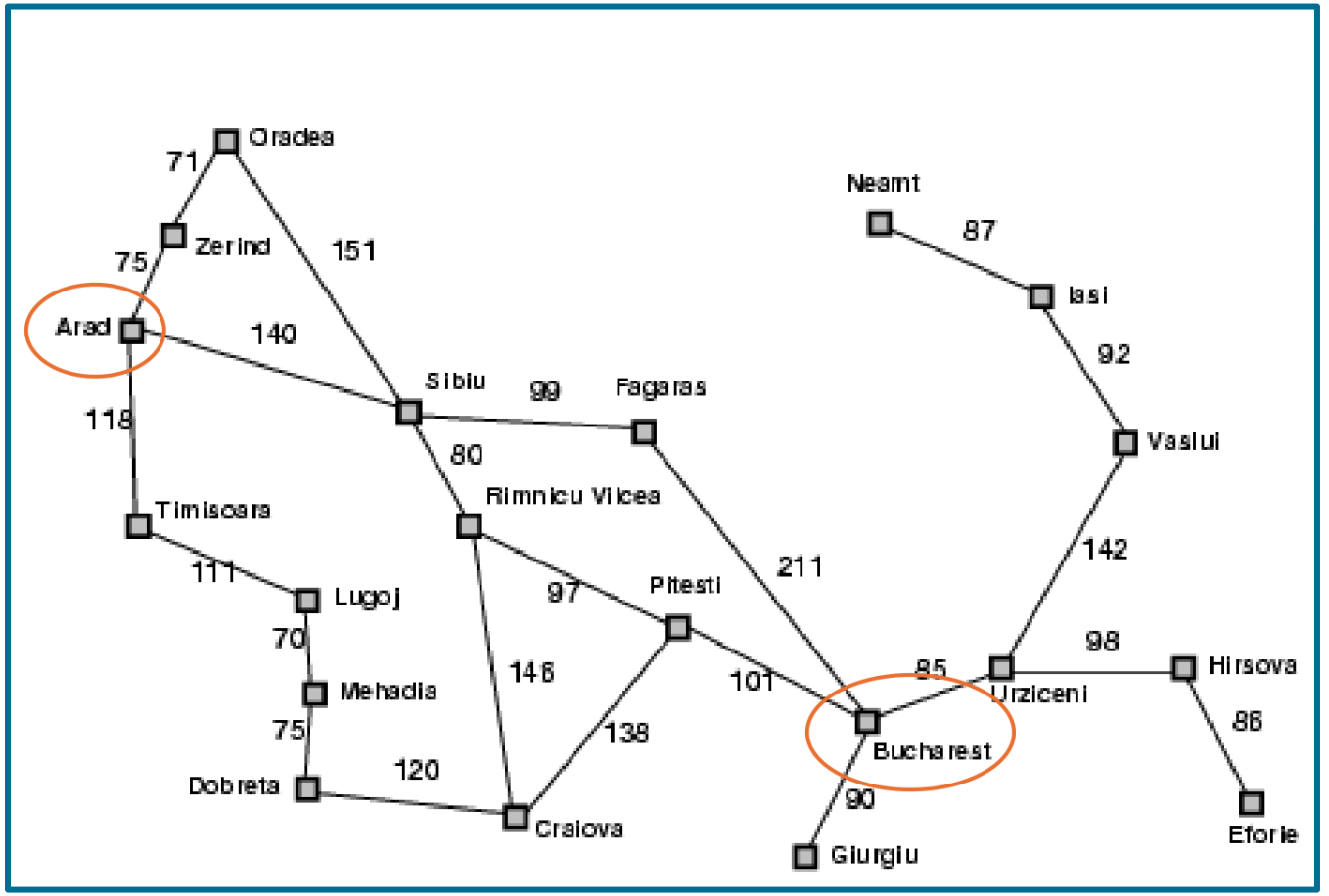
- Idea: to avoid expanding paths that are already very expensive, let $g(n)$ be the *path cost* to node n, and take it into account.
- The evaluation function $f(n)$ is the estimated total cost of the path through n to the goal:
  - $g(n)$: the cost so far, to reach node n (path cost)
  - $h(n)$: the estimated cost from n to the goal (heuristic)
  - $f(n) = g(n) + h(n)$





Example: A* Search

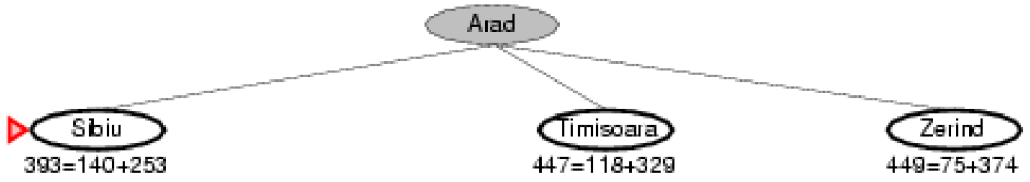
- Expansion rule: $f(n) = g(n) + h(n)$
- Expand the node with the smallest $f(n)$.
  - Start at Arad: $g = 0, h = 366$, so $f = 0 + 366 = 366$
  - $h(n)$ is the straight-line distance to Bucharest.

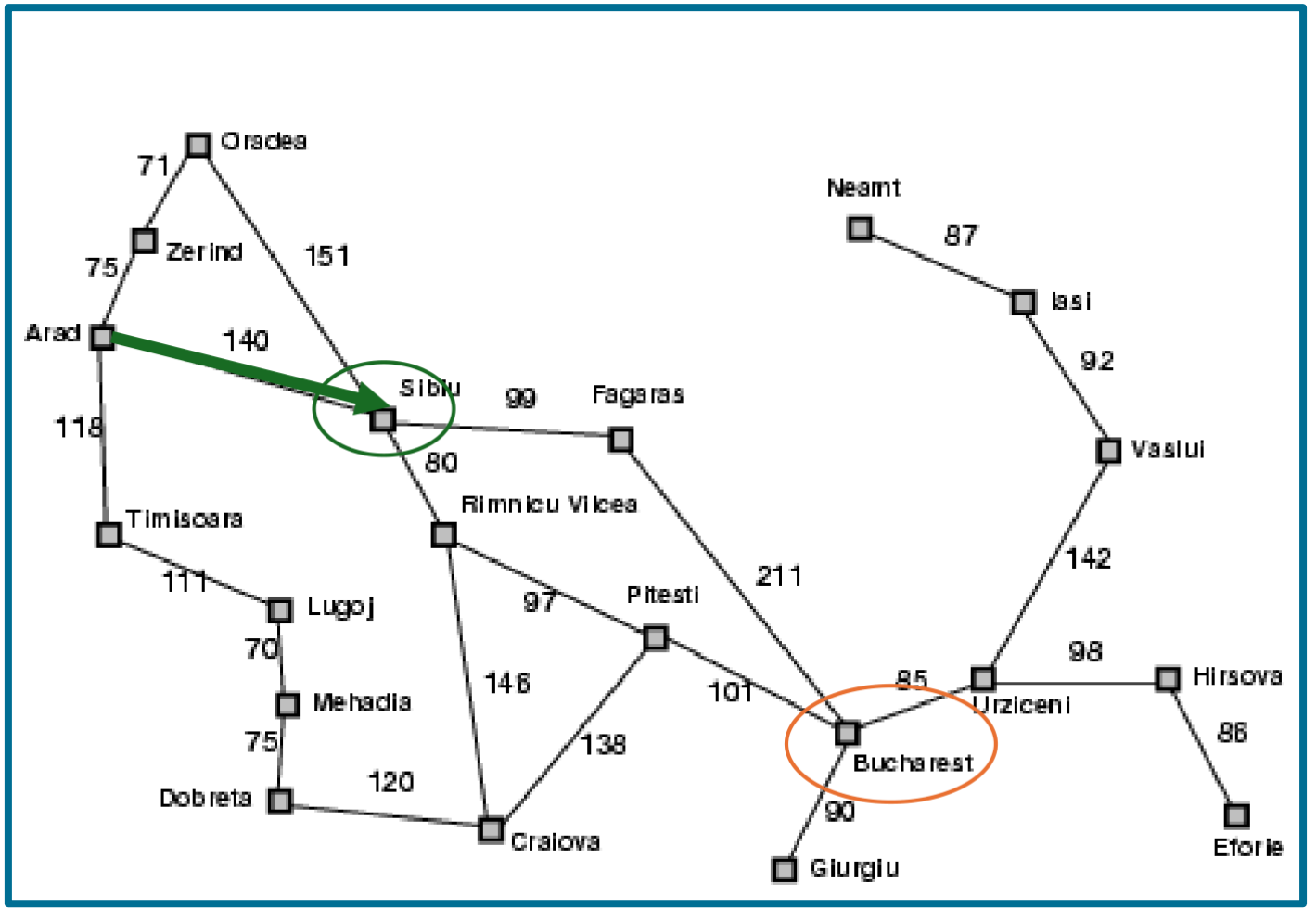


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: A* Search (2)

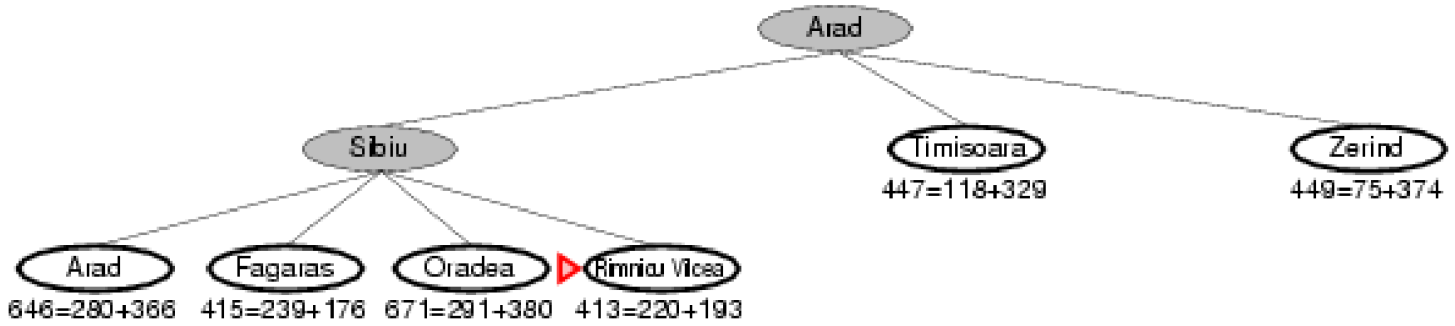


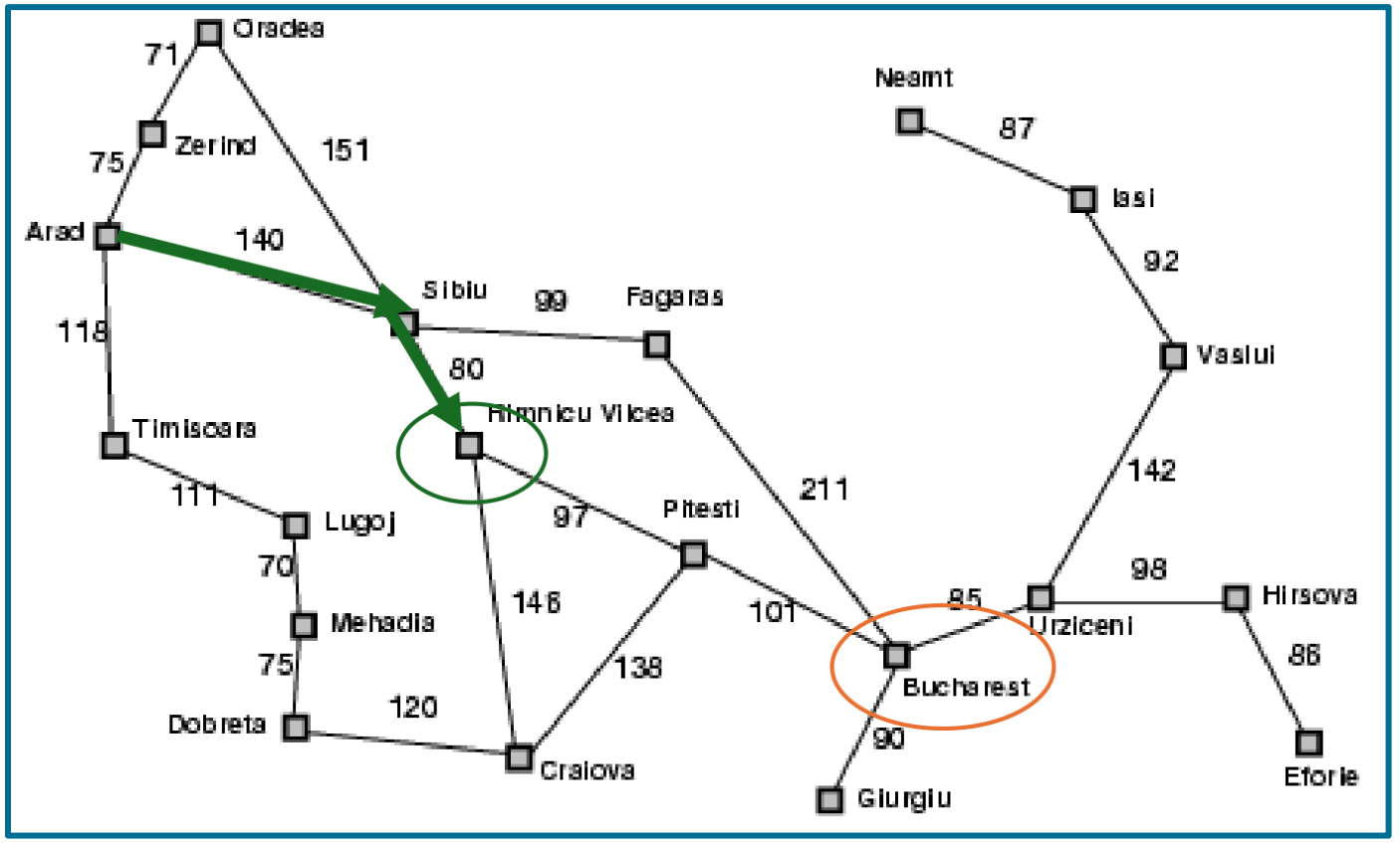


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: A* Search (3)

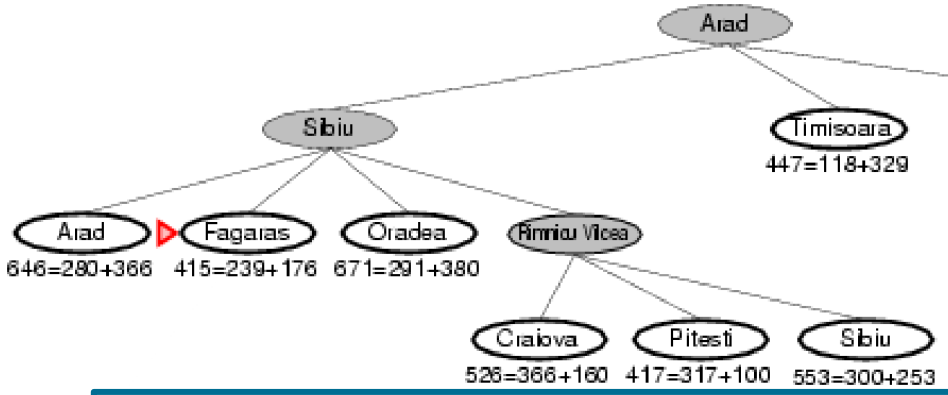


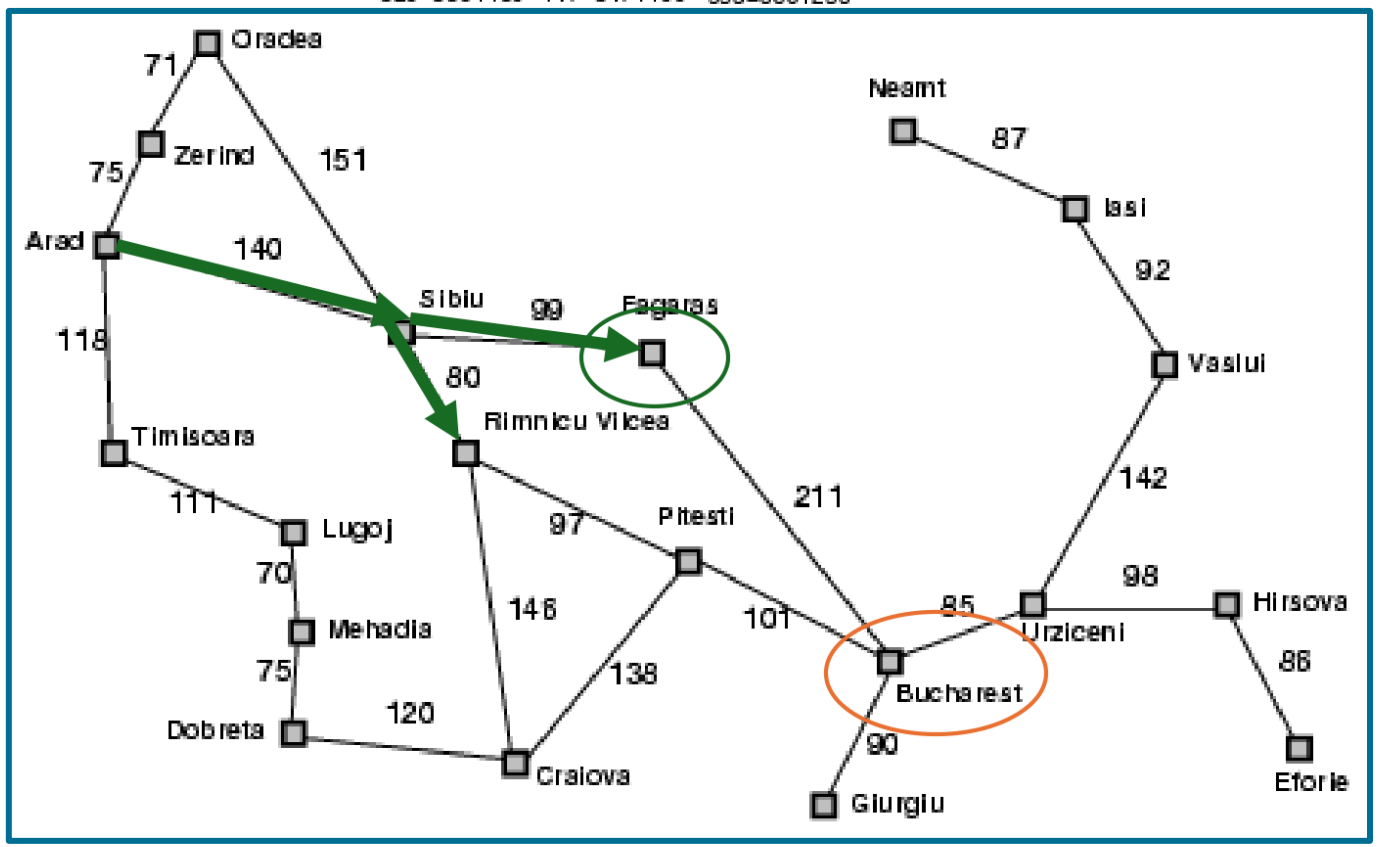


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: A* Search (4)

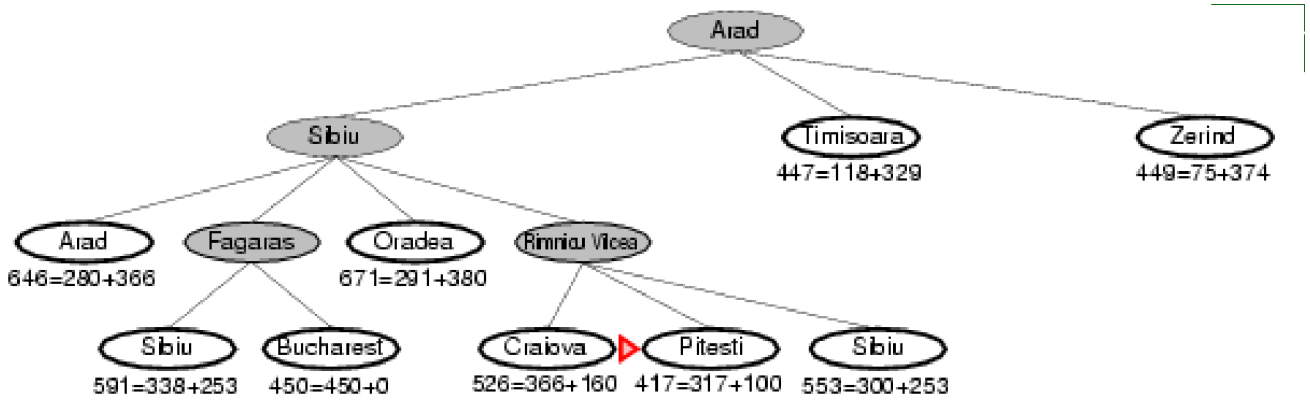


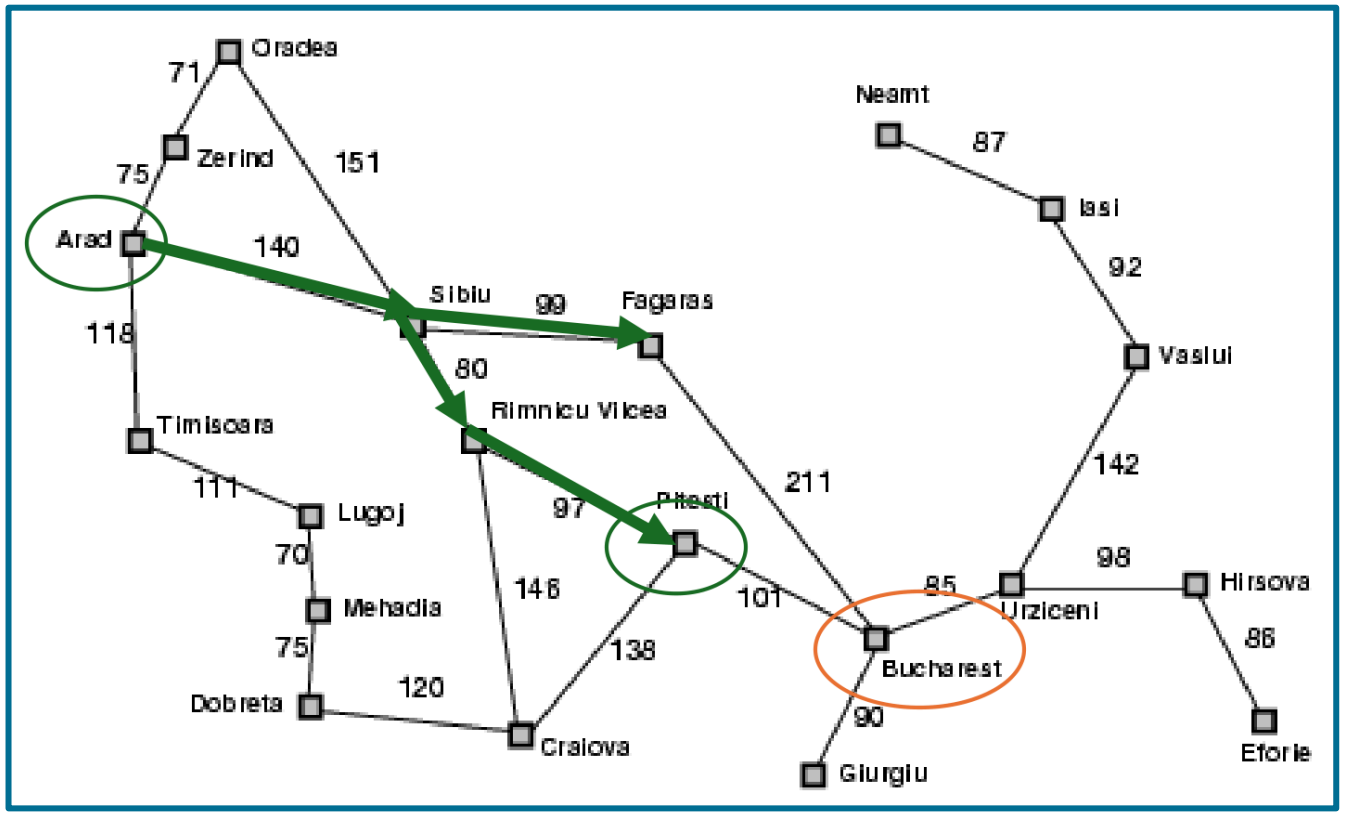


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: A* Search (5)

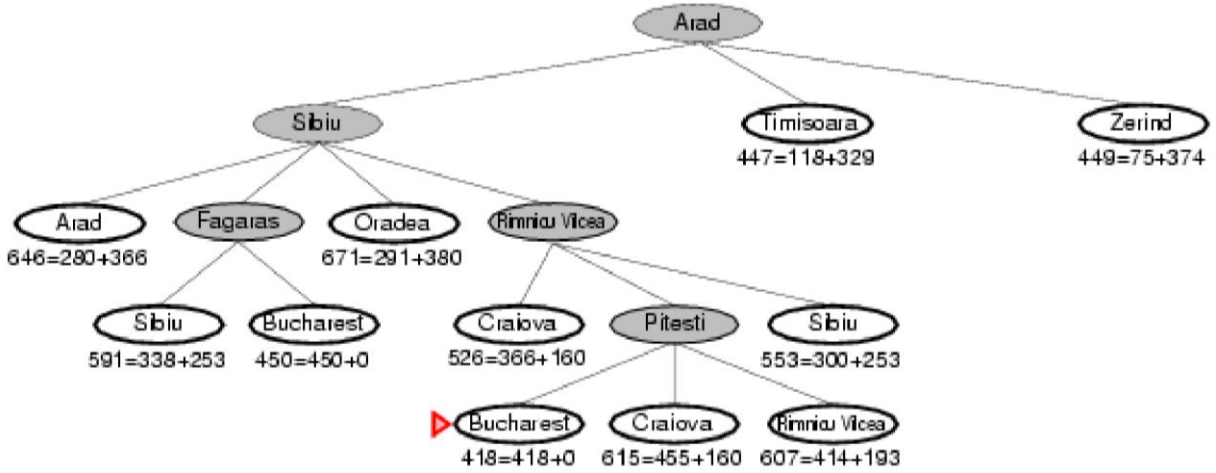


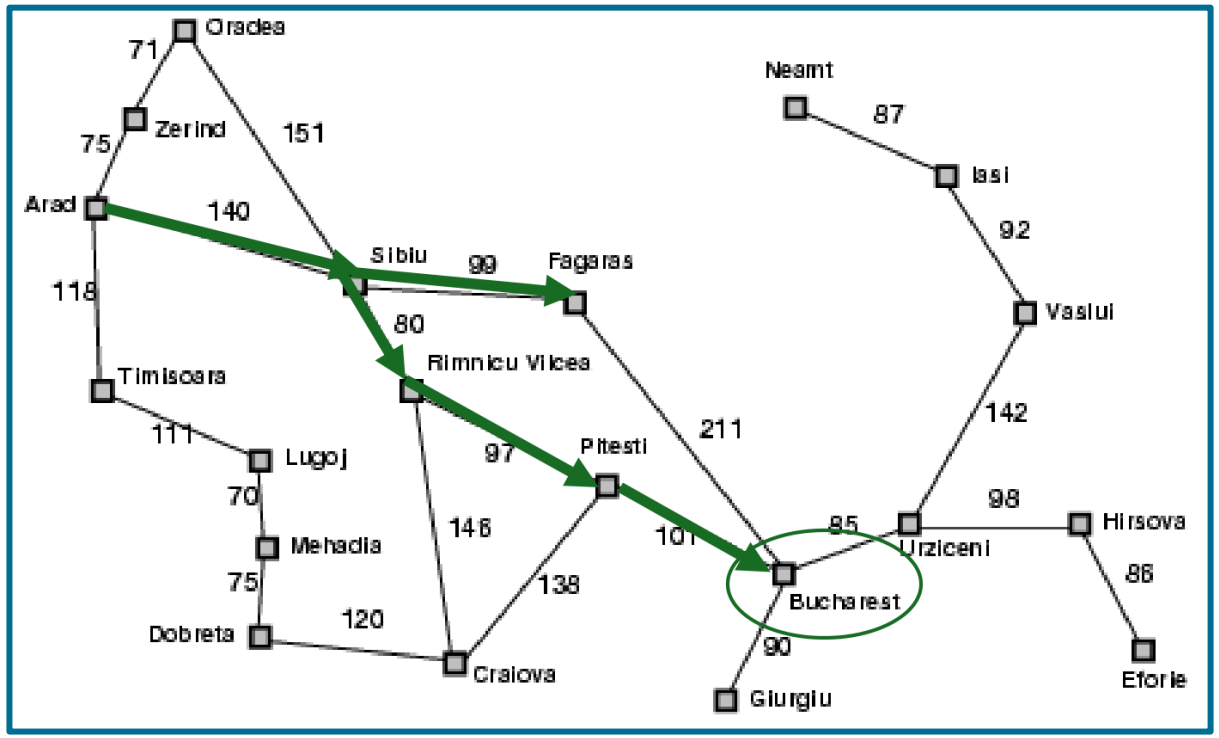


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: A* Search (6)



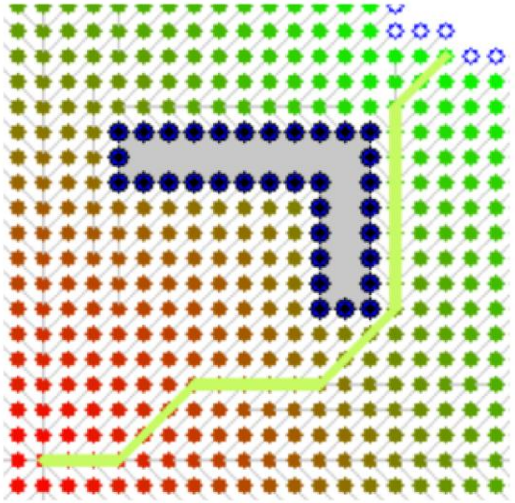


Straight-line distance to Bucharest

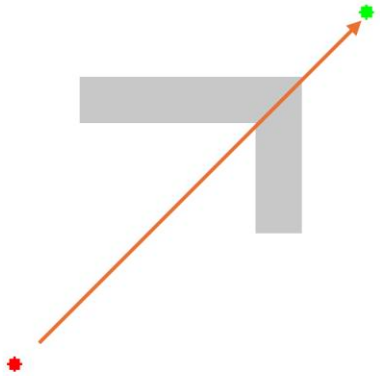
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Breadth-First Search vs A* Search

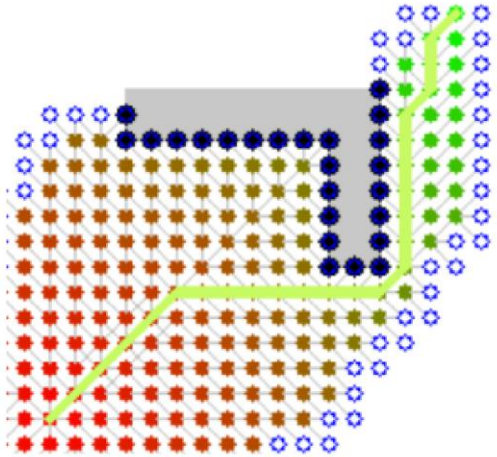
BFS



A*



A*



A* Search expands
fewer nodes than BFS!

Source: Wikipedia(https://en.wikipedia.org/wiki/A*_search_algorithm)

Implementation of A* Search

- Path cost to n + heuristic from n to the goal = estimate of the total cost. $g(n) + h(n)$

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

The order in which the frontier is expanded is decided by f(n).

Admissible Heuristics and Optimality

- Definition: a heuristic h is **admissible** when, for every node n,
  - $h(n) \leq h^*(n)$
  - where $h^*(n)$ is the true cost from n to a goal state
- An admissible heuristic is a lower bound: it never overestimates the true cost of reaching the goal
- Example: **straight-line distance** never overestimates actual road distance
- Theorem: if h is admissible, A* is optimal

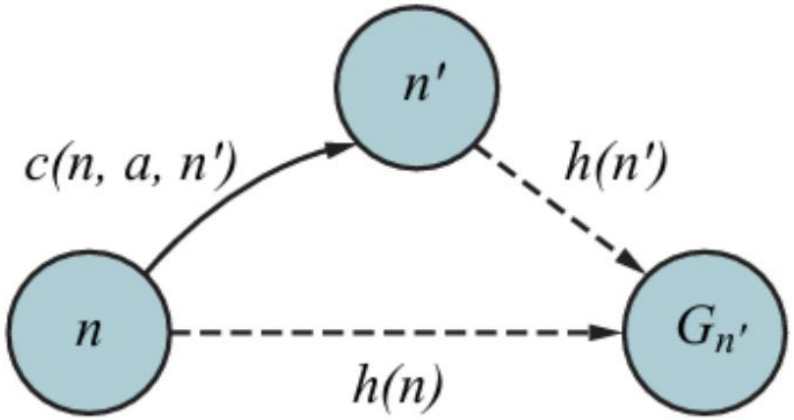
Proof of A* Optimality

- By contradiction (reductio ad absurdum)
- Let C* be the optimal path cost
- Suppose some node n on the optimal path is never expanded. If every node on the optimal path had been expanded, the algorithm would have returned the optimal solution.
- (1) $f(n) > C^*$ otherwise n would have been expanded
- (2) $f(n) = g(n) + h(n)$ by definition
- (3) $f(n) = g^*(n) + h(n)$ since n lies on the optimal path
- (4) $f(n) \leq g^*(n) + h^*(n)$ by admissibility, $h(n) \leq h^*(n)$
- (5) $f(n) \leq C^*$ by definition, $C^* = g^*(n) + h^*(n)$
- Line (1) and line (5) contradict each other. $\therefore$ $f(n) \leq C^*$

Proof of A* Optimality: Consistency

- Consistency (also called monotonicity): A heuristic $h(n)$ is consistent when, for every node n and every successor n' generated by an action a:

$$h(n) \leq c(n, a, n') + h(n')$$



- This is a form of the triangle inequality: one side of a triangle can never be longer than the sum of the other two
- Consistency implies admissibility. But, admissibility cannot imply consistency

Properties of A* Search

- Completeness? Yes
- Cost optimality? Yes
- Time complexity?
  - In the worst case the number of nodes with $f(n) \leq C^*$ is $O(b^d)$, as for BFS
- Space complexity?
  - The same as its time complexity. This is usually far too large unless we know a very good heuristic

Introduction to Artificial Intelligence

Chapter 3 - Part III: Informed Search

Namyup Kim

Soongsil University

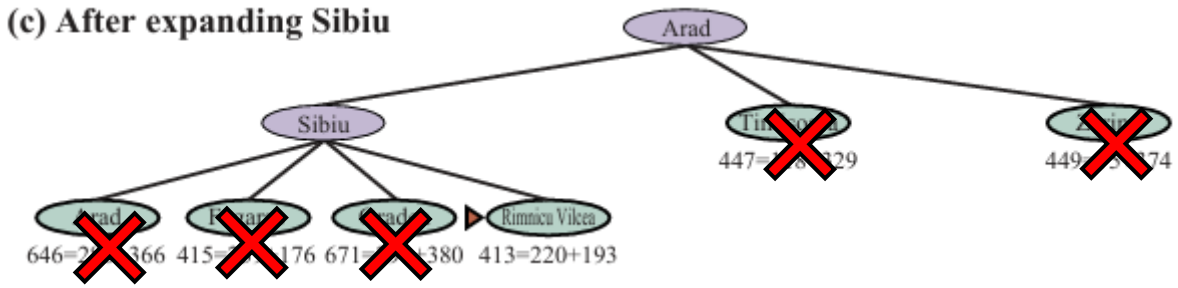


Iterative-Deepening A* Search (IDA*)

- Idea: A* performed *without a reached data structure*, in the way iterative deepening search relates to DFS. → Adress the space complexity issue.
  - Note: ordinary IDS is uninformed and raises the cutoff by 1 each iteration

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
  for depth = 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result
```

- IDA* uses the node cost f = g + h as the cutoff (), raising the cost cutoff a little each iteration. If the increment is small and h is admissible, it is optimal.
- Problems: how much to raise the cutoff each iteration, and having to rebuild the tree many times. → **Raise the cutoff to the smallest f that exceeded the last cutoff.**
- Example: f-cost cutoff is 414



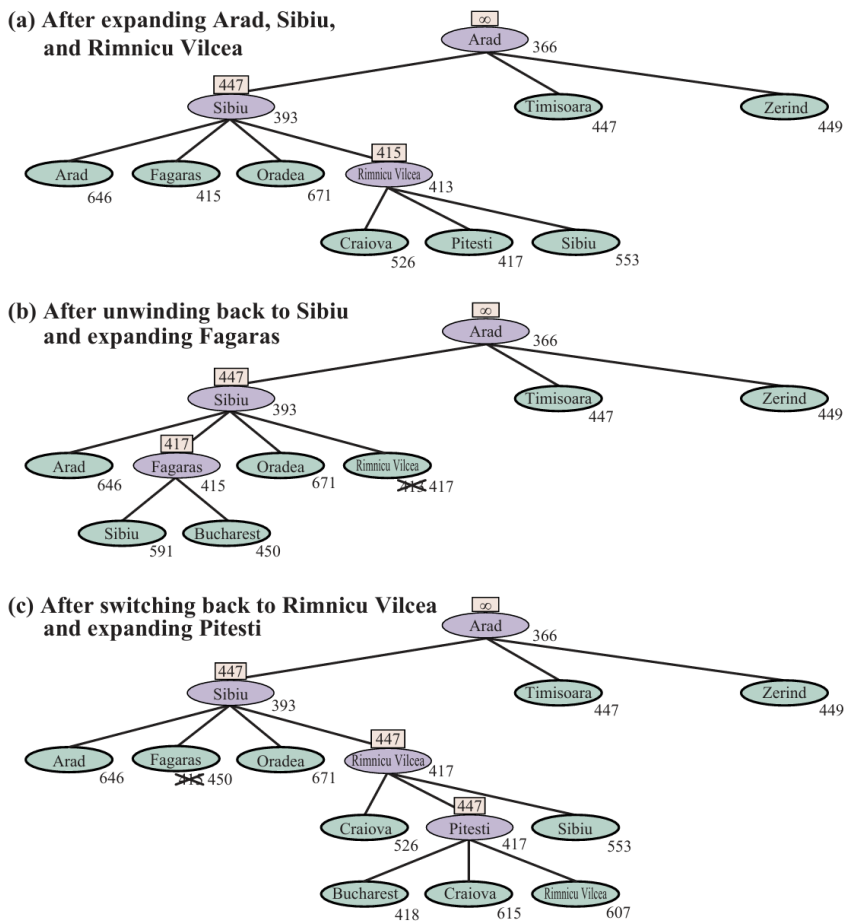
Other memory-bounded A* variants

- Recursive best-first search (RBFS) — adds an $f - limit$ to the depth-first behavior of best-first search

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution or failure
    solution, fvalue ← RBFS(problem, NODE(problem.INITIAL), ∞)
  return solution

function RBFS(problem, node, f_limit) returns a solution or failure, and a new f-cost limit
  if problem.IS-GOAL(node.STATE) then return node
  successors ← LIST(EXPAND(node))
  if successors is empty then return failure, ∞
  for each s in successors do        // update f with value from previous search
      s.f ← max(s.PATH-COST + h(s), node.f))
  while true do
      best ← the node in successors with lowest f-value
      if best.f > f_limit then return failure, best.f
      alternative ← the second-lowest f-value among successors
      result, best.f ← RBFS(problem, best, min(f_limit, alternative))
      if result ≠ failure then return result, best.f
```

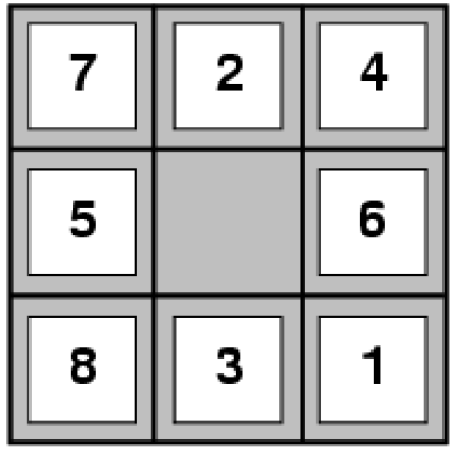
Figure 3.22 The algorithm for recursive best-first search.



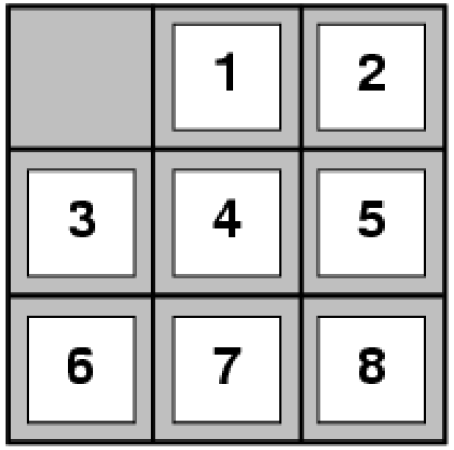
- Simplified memory-bounded A* (SMA*) — run A* until memory is full, then drop the worst (largest f) leaf node; it can be regenerated later if needed.

Designing Heuristic Functions $h(n)$

- Heuristics for the 8-puzzle:



Start State

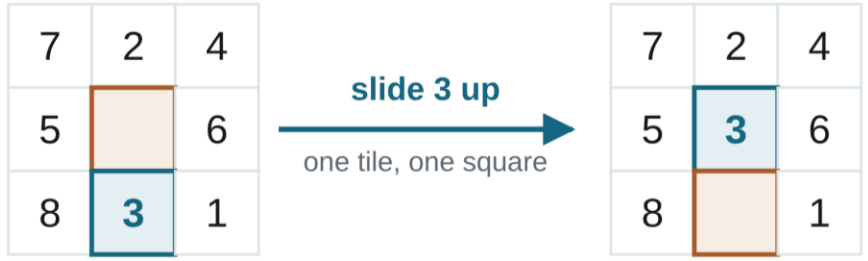


Goal State

  - $h_1(n)$ = the number of misplaced tiles. h_1(start) = 8
  - $h_2(n)$ = the total Manhattan distance. h_2(start) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18
- Are h_1 and h_2 admissible? Yes
  - A heuristic h is **admissible** when, for every node n,
    - $h(n) \leq h^*(n)$
    - where $h^*(n)$ is the true cost from n to a goal state

Designing Heuristic Functions $h(n)$

- Heuristics for the 8-puzzle:
  - $h_1(n)$ = the number of misplaced tiles. h_1(start) = 8
  - $h_2(n)$ = the total Manhattan distance. h_2(start) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18
- Are h_1 and h_2 admissible? Yes

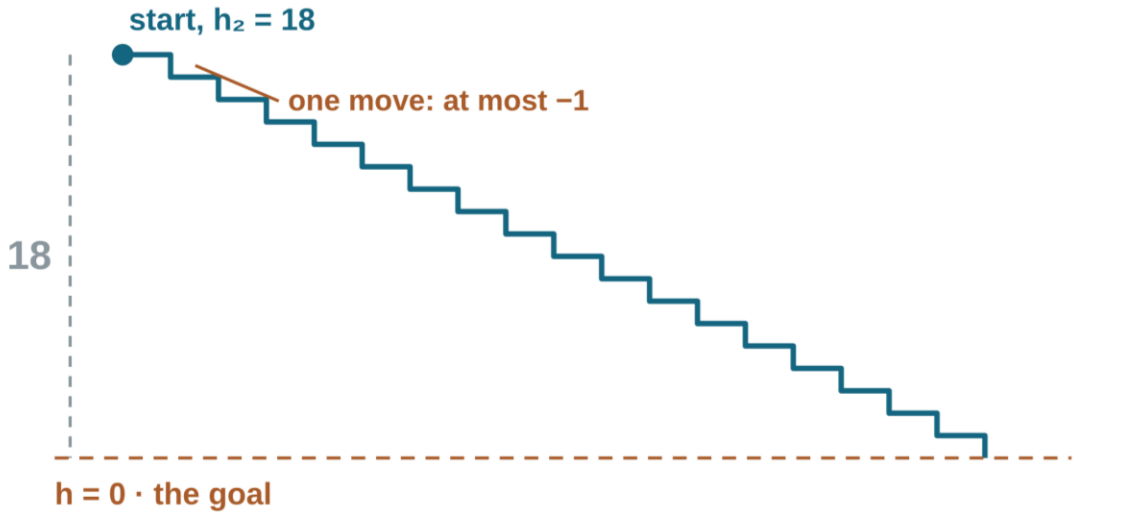


$h_1 = 8 \cdot h_2 = 18$ $h_1 = 8 \cdot h_2 = 17$

h_1 misplaced tiles
the moved tile may land home, stay wrong, or leave home — **−1, 0 or +1**

h_2 Manhattan sum
that tile's distance shifts by exactly one square — **−1 or +1**

Only one tile ever moves. The other seven sit still.
So neither h can fall by more than 1 in a single move.

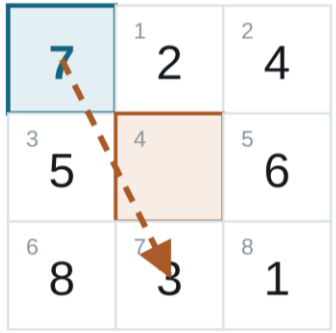


h has to walk from 18 down to 0, one step at a time.

h^*(start) ≥ 18 = h_2(start) → admissible

Heuristics from Relaxed Problems

- A problem with fewer restrictions on its actions is called a relaxed problem
- The optimal solution cost of a relaxed problem is an admissible heuristic for the original problem - it can never come out larger than the true cost.
- Which relaxation is used in h_1 and h_2?
  - Original actions in 8-puzzle: move the ***blank*** tile ***left, right, up or down***.
  - h_1 actions: move the tiles anywhere
    - ➢ optimal solution: move misplaced tiles to be correct
  - h_2 actions: move the tiles to adjacent square
    - ➢ optimal solution: move misplaced tiles to be correct using *left, right, up or down directions.*

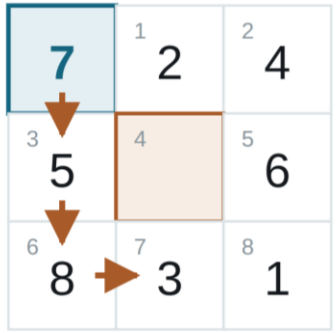


tile 7 : 1 move

One move carries a tile straight to its home square, however far away that square is.

So the cost is simply: how many tiles are not home?

$h_1 = 8$



tile 7 : 3 moves

One move is one square — but nothing blocks it. Tile 7 slides straight through 5 and 8.

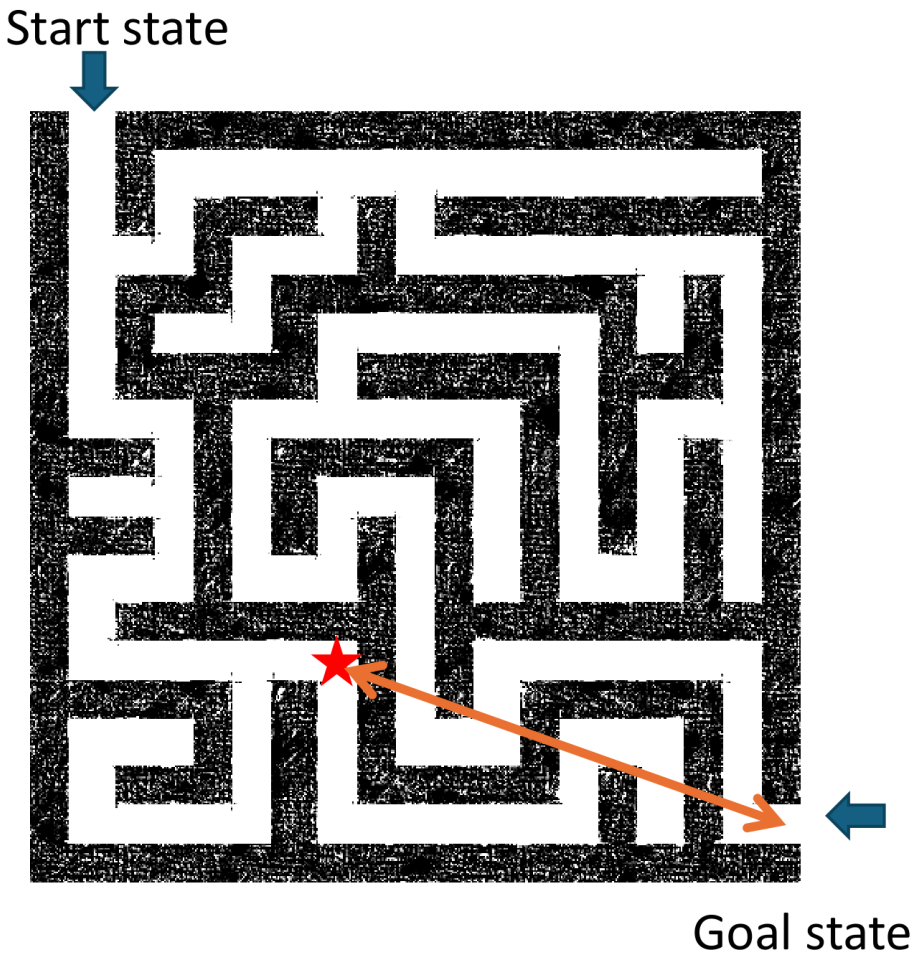
So the cost is: add up every tile's distance.

$h_2 = 18$

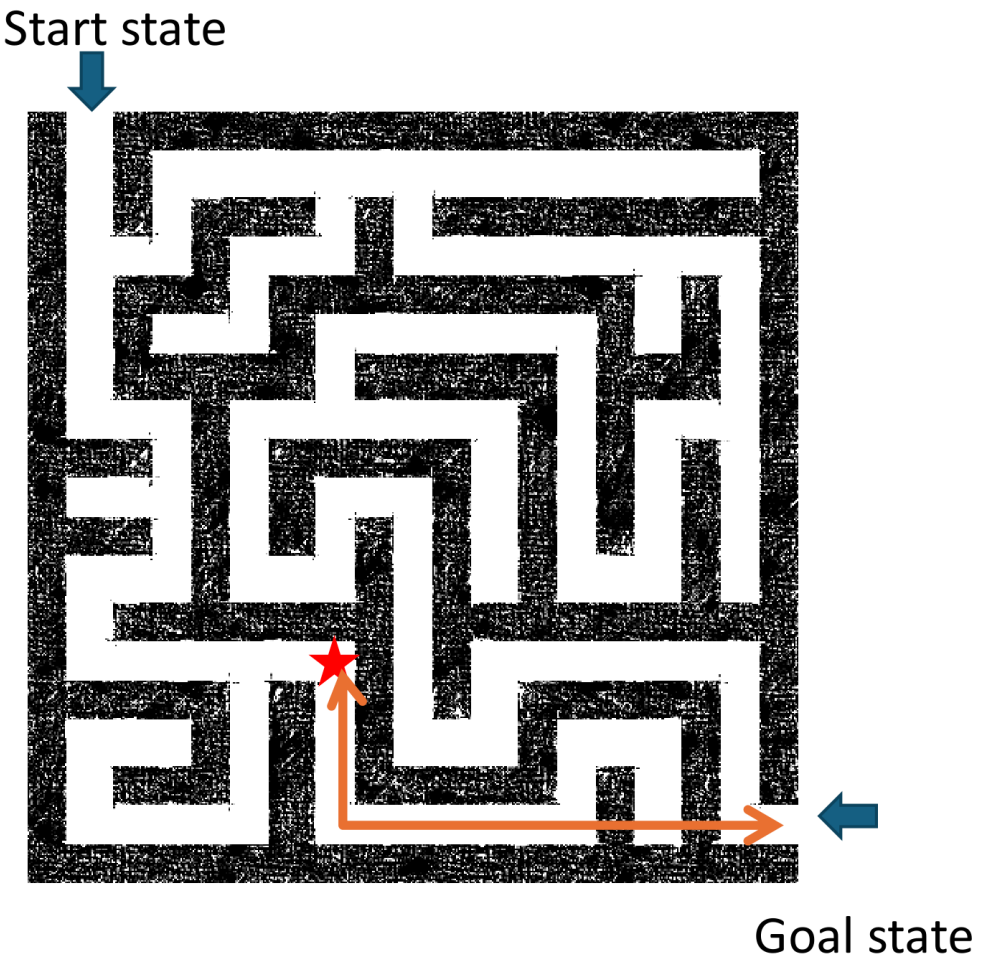
Heuristics from Relaxed Problems (2)

- Why are these two cases relaxed problems?

Euclidean distance

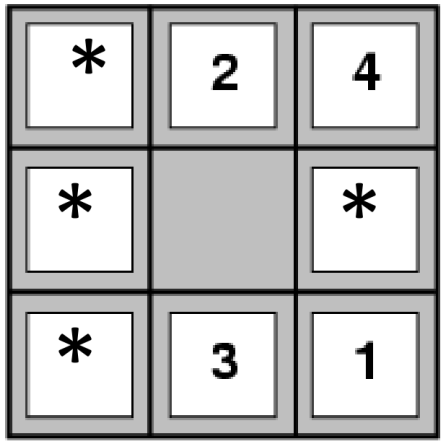


Manhattan distance

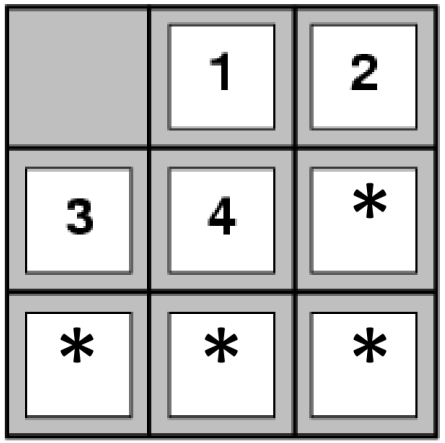


Heuristics from Subproblems

- Let $h_3(n)$ be the cost of getting some of the tiles (ex., 1, 2, 3, 4) into their correct positions. The final order of the other tiles does not matter
- The solution cost of a subproblem is an admissible heuristic.
- Calculation:
  - Very small subproblems are usually easy to solve
  - The exact solution cost for all (or many) possible subproblem instances can be precomputed and stored - this is called a **pattern database**.



Start State



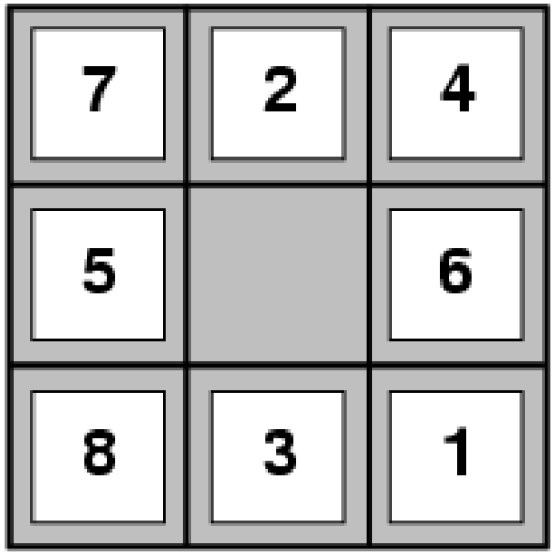
Goal State

Dominance: Which Heuristic Is Better?

- Definition: if h_1 and h_2 are both admissible and $h_2(n) \geq h_1(n)$ for every n, then h_2 dominates h_1.
  - $h_1(n)$ = the number of misplaced tiles. h_1(start) = 8
  - $h_2(n)$ = the total Manhattan distance. h_2(start) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18
- Which is better for A* search, h_1 or h_2?
  - A* expands every node with $f(n) < C^* \Leftrightarrow h(n) < C^* - g(n)$
  - h_2 is never smaller than h_1, so A* using h_2 expands fewer nodes — **it is better.**

The Effect of Information on Search

- Typical search costs for the 8-puzzle:
- Solution at depth d = 12
  - IDS = 3,644,035 nodes
  - A*(h_1) = 227 nodes
  - A*(h_2) = 73 nodes
- Solution at depth d = 24
  - IDS ≈ 54,000,000,000 nodes
  - A*(h_1) = 39,135 nodes
  - A*(h_2) = 1,641 nodes



Combining Heuristics

- Suppose we have admissible heuristics $h_1, h_2, \ldots, h_m$,but none of them dominates any of the others

- Combining them is straightforward:

$$h(n) = \max\{ h_1(n), h_2(n), \ldots, h_m(n) \}$$

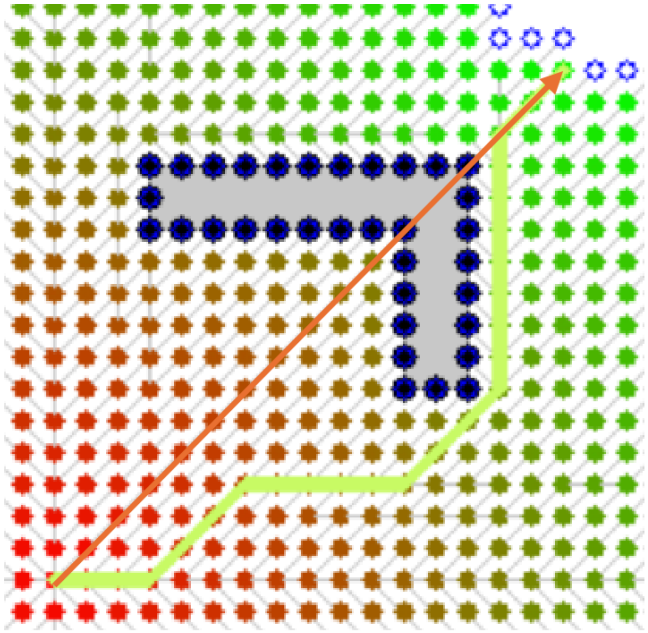
- That is, for each node pick whichever heuristic comes closest to the true cost $h^*(n)$.

Satisficing Search: Weighted A* Search

- Often a "good enough" solution is enough, if it can be found quickly or with far fewer resources - that is, by expanding fewer nodes.
  - Satisficing: satisfy(만족하다) + suffice(충분하다)
- A* can be run with *an inadmissible heuristic*: multiply $h(n)$ by a factor W, so that it sometimes slightly *overestimates* the optimal cost to the goal
  1. This can largely reduce the number of nodes expanded
  2. But it breaks the algorithm's optimality guarantee
- Weighted A* search: $f(n) = g(n) + W \times h(n), \quad 1 < W < \infty$
- The algorithms we have seen are special cases:
  - A* search: $g(n) + h(n)$ (W = 1)
  - Uniform-cost search / BFS: $g(n)$ (W = 0)
  - Greedy best-first search: $h(n)$ (W = ∞)

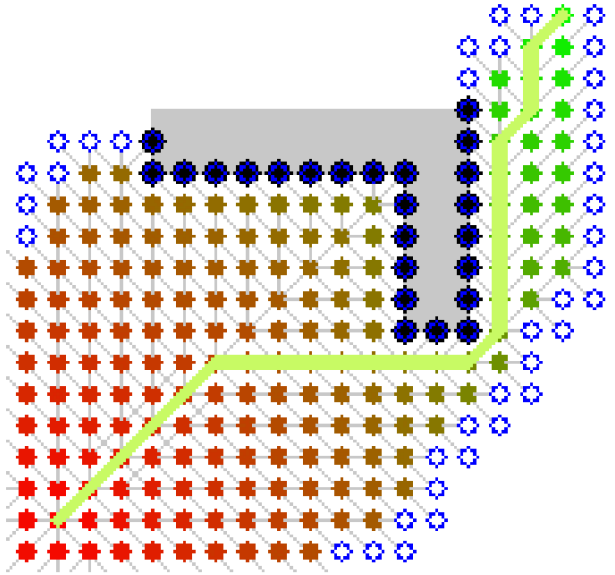
Example: Weighted A* Search

Reduction in the number of expanded nodes



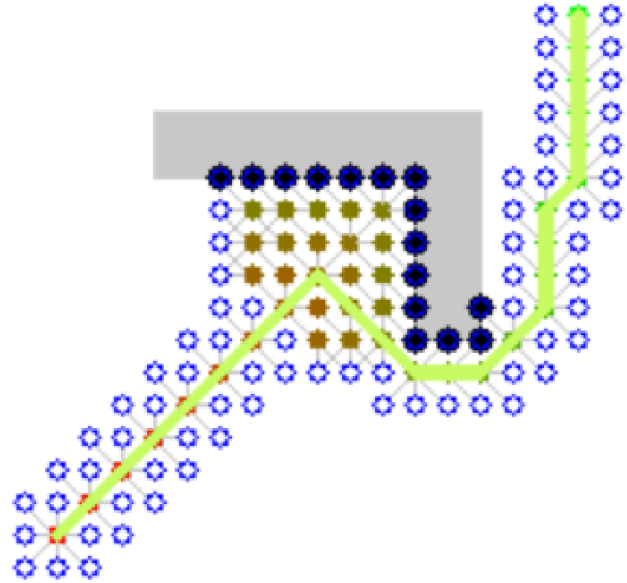
Breadth-first Search (BFS)

$f(n) =$ # actions to reach n



Exact A* Search

$f(n) = g(n) + h_{Eucl}(n)$



Weighted A* Search

$f(n) = g(n) + 5\, h_{Eucl}(n)$

Summary: All Search Strategies

	Algorithm	Complete?	Optimal?	Time complexity	Space complexity
Uninformed Search	BFS (Breadth-first search)	Yes	If all step costs are equal	$O(b^d)$	$O(b^d)$
	Uniform-cost search	Yes	Yes	Number of nodes with $g(n) \leq C^*$	
	DFS	In finite space, Yes (cycles checking)	No	$O(b^m)$	$O(bm)$
	IDS	Yes	If all step costs are equal	$O(b^d)$	$O(bd)$
Informed Search	Greedy best-first search	In finite space, Yes (cycles checking)	No	Depends on heuristic. Best case: $O(bd)$. Worst case: $O(b^m)$	
	A* search	Yes	Yes	Number of nodes with $g(n) + h(n) \leq C^*$	

- with a good heuristic

· b: maximum branching factor of the search tree · d: depth of the optimal solution
· m: maximum length of any path in the state space · C*: cost of the optimal solution